

零声教育 Darren

课程: [埃梯零声 \(0voice.com\)](https://0voice.com)

特别注意:

课程使用的RO2版本必须在 Ubuntu24.04 desktop 桌面版运行

Ubuntu24.04桌面版下载地址: <https://mirrors.aliyun.com/ubuntu-releases/24.04/ubuntu-24.04.2-desktop-amd64.iso>

课件地址: https://gitlab.0voice.com/2503_vip/ros2-doc

源码地址: https://gitlab.0voice.com/2503_vip/ros2-src

4 ROS开发基石



ROS2通信体系



4.1 节点

4.1.1 什么是节点 (What is Node)

节点的定义和概念

- **节点 (Node)**：ROS2系统中的基本执行单元，**是一个独立的进程**
- **功能职责**：每个节点负责特定的功能，如传感器数据处理、运动控制、决策等
- **通信机制**：节点间通过**话题、服务、动作**进行通信

节点的特性

节点特性:

- └─ 独立性 # 每个节点是独立的进程，崩溃不影响其他节点
- └─ 可重用性 # 同一节点可以启动多个实例
- └─ 分布式 # 节点可以运行在不同的机器上
- └─ 命名空间 # 支持命名空间隔离
- └─ 生命周期 # 具有完整的生命周期管理

节点的组成要素

1. 基础标识

- 节点名称和命名空间 - **确保节点在系统中的唯一识别和组织管理**
- **唯一标识符** - 防止节点名称冲突，支持分布式部署

2. 通信接口

- **发布者 (Publishers)** - 异步发送数据到话题，实现一对多广播通信
- **订阅者 (Subscribers)** - 异步接收话题数据，支持多个订阅者同时监听
- **服务服务器 (Service Servers)** - 提供同步请求-响应服务，处理客户端调用
- **服务客户端 (Service Clients)** - 发起同步服务请求，等待服务器响应
- 动作服务器 (Action Servers) - 处理长时间运行任务，提供进度反馈
- 动作客户端 (Action Clients) - 发起异步任务请求，接收执行状态和结果

3. 配置管理

- 参数声明和验证 - 定义节点配置项，确保参数类型和范围正确性
- 参数回调处理 - 监听参数变化，实现动态配置响应
- 动态参数更新 - 运行时修改节点行为，无需重启节点

4. 时间管理

- 周期性定时器 - 按固定间隔执行任务，如传感器数据采集
- 一次性定时器 - 延迟执行单次任务，用于超时处理
- 时间戳处理 - 记录和同步事件时间，确保数据时序正确

5. 日志系统

- 多级别日志记录 - 支持DEBUG/INFO/WARN/ERROR等不同重要性级别
- 格式化输出 - 统一日志格式，便于分析和调试
- 调试信息 - 提供运行时状态信息，帮助问题定位

6. 执行控制

- 回调组管理 - 组织和调度不同类型的回调函数
- 优先级控制 - 确保重要任务优先执行
- 并发处理 - 支持多线程执行，提高系统响应性能

7. 生命周期

- 状态管理 - 跟踪节点当前运行状态

- 配置/激活/停用/清理 - 标准化节点启动、运行、停止流程
- 状态转换 - 确保节点安全可靠地在不同状态间切换

4.1.2 节点编程方法 (Node Programming Methods)

C++节点编程基础

创建C++功能包

```
# 进入工作空间src目录
cd ~/ros2_ws/src

# 创建简单的C++功能包
ros2 pkg create --build-type ament_cmake hello_cpp_node --dependencies rclcpp
```

命令解析：

- `ros2 pkg create`
 - ROS2的包创建工具
 - 用于自动生成标准ROS2包结构
- `--build-type ament_cmake`
 - 指定构建系统类型为ament_cmake
 - **ament_cmake是ROS2推荐的C++包构建系统**
 - 基于CMake，提供ROS2特定的构建功能
- `hello_cpp_node`
 - 包名称
 - **将创建名为"hello_cpp_node"的ROS2包**
 - 包名应遵循小写字母和下划线的命名规范
- `--dependencies rclcpp`
 - 声明包依赖项
 - **rclcpp是ROS2的C++客户端库**
 - 自动在package.xml和CMakeLists.txt中添加此依赖

执行结果：该命令将创建以下目录结构：

```
hello_cpp_node/
├─ CMakeLists.txt      # CMake构建配置文件
├─ package.xml         # 包元数据和依赖声明
├─ src/               # 源代码目录
└─ include/hello_cpp_node/ # 头文件目录
```

编写节点代码

src/hello_cpp_node/src/hello_node.cpp

这个范例启动一个定时器，间隔一秒打印信息。

```
/**
 * @file hello_cpp_node/src/hello_node.cpp
 * @brief ROS2基础示例节点 - 演示定时器和日志输出功能
 * @author
 * @date
 * @description 这是一个简单的ROS2 C++节点示例，演示了如何创建节点、设置定时器和输出日志信息
 */

// 包含ROS2 C++客户端库的头文件
#include <rclcpp/rclcpp.hpp>
// 包含时间相关的头文件，用于定时器功能
#include <chrono>
// 包含字符串处理的头文件
#include <string>

// 使用chrono字面量，方便时间单位的表示（如1s表示1秒）
using namespace std::chrono_literals;

/**
 * @class HelloNode
 * @brief Hello世界示例节点类
 * @description 继承自rclcpp::Node，实现了一个简单的定时打印功能
 */
class HelloNode : public rclcpp::Node
{
public:
    /**
     * @brief 构造函数
     * @description 初始化节点，设置节点名称为"hello_node"，创建定时器
     */
    HelloNode() : Node("hello_node"), count_(0)
    {
        // 创建墙时钟定时器，每1秒执行一次timer_callback函数
        // create_wall_timer使用系统时钟，不受ROS时钟影响
        timer_ = this->create_wall_timer(
            1s, std::bind(&HelloNode::timer_callback, this));

        // 输出节点启动信息到控制台
        RCLCPP_INFO(this->get_logger(), "Hello ROS2 节点已启动!");
    }

private:
    /**
     * @brief 定时器回调函数

```

```

    * @description 每当定时器触发时调用此函数，输出Hello信息和当前计数
    */
void timer_callback()
{
    // 使用RCLCPP_INFO宏输出信息级别的日志
    // %ld是长整数的格式化占位符
    RCLCPP_INFO(this->get_logger(), "Hello ROS2! 计数: %ld", count_);

    // 计数器自增
    count_++;
}

// 定时器对象的智能指针，用于管理定时器的生命周期
rclcpp::TimerBase::SharedPtr timer_;

// 计数器变量，记录定时器执行次数
size_t count_;
};

/**
 * @brief 主函数
 * @description 程序入口点，负责初始化ROS2环境、创建和运行节点
 */
int main(int argc, char * argv[])
{
    // 初始化ROS2通信中间件
    // 必须在使用任何ROS2功能之前调用
    rclcpp::init(argc, argv);

    // 创建HelloNode节点实例的智能指针
    // 使用shared_ptr确保内存安全管理
    auto node = std::make_shared<HelloNode>();

    // 进入ROS2事件循环，开始处理回调函数
    // 这个函数会阻塞，直到节点被关闭
    rclcpp::spin(node);

    // 清理ROS2资源，关闭通信中间件
    rclcpp::shutdown();

    // 程序正常退出
    return 0;
}

```

设置编译选项

CMakeLists.txt配置:

src/hello_cpp_node/CMakeLists.txt

```
#
# @file hello_cpp_node/CMakeLists.txt
# @brief ROS2 C++节点包的构建配置文件
# @description 定义了hello_cpp_node包的编译、依赖和安装规则
#

cmake_minimum_required(VERSION 3.8)

# 定义项目名称
# 这个名称应该与package.xml中的包名保持一致
project(hello_cpp_node)

# 编译器设置 - 启用编译警告
# 为GCC或Clang编译器添加常用的警告选项，提高代码质量
if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
    add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# 查找ROS2依赖包
# ament_cmake: ROS2的构建系统核心包
find_package(ament_cmake REQUIRED)
# rclcpp: ROS2的C++客户端库，提供节点、发布者、订阅者等功能
find_package(rclcpp REQUIRED)

# 创建可执行文件
# 从src/hello_node.cpp源文件编译生成hello_node可执行文件
add_executable(hello_node src/hello_node.cpp)

# 设置目标的依赖关系
# 将rclcpp库链接到hello_node可执行文件
# 这样可执行文件就能使用ROS2的C++功能
ament_target_dependencies(hello_node rclcpp)

# 安装可执行文件
# 将编译好的可执行文件安装到ROS2的标准目录
# lib/${PROJECT_NAME}是ROS2包可执行文件的标准安装路径
install(TARGETS hello_node
        DESTINATION lib/${PROJECT_NAME})

# 完成ament包的配置
# 这是ROS2包配置的最后一步，生成必要的配置文件
ament_package()
```

编译运行流程

```
# 1. 编译功能包
cd ~/ros2_ws
# 构建指定包--packages-select hello_cpp_node
colcon build --packages-select hello_cpp_node

# 2. 设置环境变量
source install/setup.bash

# 3. 运行节点
# hello_cpp_node 告诉ROS2在哪个包中查找可执行文件
# hello_node 实际运行的二进制文件
ros2 run hello_cpp_node hello_node
```

```
lqf@lqf-ros2:~/ros2_ws$ ros2 run hello_cpp_node hello_node
[INFO] [1749110620.925656219] [hello_node]: Hello ROS2 节点已启动!
[INFO] [1749110621.926056509] [hello_node]: Hello ROS2! 计数: 0
[INFO] [1749110622.926097619] [hello_node]: Hello ROS2! 计数: 1
[INFO] [1749110623.926391254] [hello_node]: Hello ROS2! 计数: 2
[INFO] [1749110624.925900610] [hello_node]: Hello ROS2! 计数: 3
```

4.1.3 节点的命令行操作

在hello_node运行时操作命令

节点信息查看

```
# 查看所有运行的节点
ros2 node list
```

比如显示:

```
/hello_node
```

```
# 查看节点详细信息
ros2 node info /my_node
# 比如
ros2 node info /hello_node

# 实时查看节点信息
watch -n 1 "ros2 node list"

# 然后可以停止hello_node，然后启动hello_node，观察watch信息
```


4.2 通信机制-话题

4.2.1 什么是话题

定义和概念

- **话题 (Topic)** : ROS2中节点间进行异步数据传输的命名通道
- **作用**: 作为数据传递的桥梁, 连接发布者和订阅者
- **特点**: 基于发布-订阅模式, 支持一对多、多对一、多对多通信

话题的组成要素

话题系统:

└─ 话题名称	# 唯一标识符, 如 <code>"/sensor_data"</code>
└─ 消息类型	# 数据格式定义, 如 <code>std_msgs/String</code>
└─ 发布者	# 发送数据的节点
└─ 订阅者	# 接收数据的节点

话题vs其他通信方式

- **话题**: 异步、多对多、持续数据流
- **服务**: 同步、一对一、请求-响应
- **动作**: 异步、长时间任务、带反馈

4.2.2 话题通信模型

多对多通信

通信模式示例:



异步通信特点

- **发布者**: 发送消息后立即返回, 不等待接收确认
- **订阅者**: 通过回调函数异步处理接收到的消息
- **解耦性**: 发布者和订阅者不需要知道对方的存在

消息接口

- **标准消息类型**: std_msgs, geometry_msgs, sensor_msgs等
 - std_msgs - 基础数据类型

```
// 基本数据类型
std_msgs::msg::String text_msg;
text_msg.data = "Hello ROS2";

std_msgs::msg::Int32 number_msg;
number_msg.data = 42;

// 数组类型
std_msgs::msg::Float64MultiArray array_msg;
array_msg.data = {1.0, 2.0, 3.0, 4.0};
array_msg.layout.dim.push_back(std_msgs::msg::MultiArrayDimension());
array_msg.layout.dim[0].size = 4;
array_msg.layout.dim[0].stride = 4;
array_msg.layout.dim[0].label = "values";
```

- geometry_msgs - 几何和空间信息
 - 位置信息 geometry_msgs::msg::Point
 - 四元数表示旋转 geometry_msgs::msg::Quaternion
 - 位姿信息 (位置+旋转) geometry_msgs::msg::Pose
 - 带时间戳的位姿 geometry_msgs::msg::PoseStamped
 - 速度信息 geometry_msgs::msg::Twist
 - 路径信息 nav_msgs::msg::Path
- sensor_msgs - 传感器数据
 - 图像数据 sensor_msgs::msg::Image
 - 激光扫描数据 sensor_msgs::msg::LaserScan
 - 点云数据 sensor_msgs::msg::PointCloud2
 - IMU数据 sensor_msgs::msg::Imu
- **自定义消息**: 用户可定义特定的消息格式
 - 消息定义文件 (.msg) **自定义数据结构**

```
# 在msg/目录下创建 RobotStatus.msg
# msg/RobotStatus.msg
string robot_name
geometry_msgs/Pose current_pose
float64 battery_level
bool is_moving
int32[] sensor_readings
RobotMode mode # 自定义枚举

# 常量定义
int32 STATUS_IDLE = 0
int32 STATUS_MOVING = 1
int32 STATUS_CHARGING = 2
```

- **QoS (服务质量)**：控制消息传输的可靠性和延迟 (**kafka**)
 - QoS配置参数节选

```
#include <rclcpp/qos.hpp>

class QoSExampleNode : public rclcpp::Node {
public:
    QoSExampleNode() : Node("qos_example") {
        // 1. 可靠性设置
        auto reliable_qos = rclcpp::QoS(10)
            .reliability(RMW_QOS_POLICY_RELIABILITY_RELIABLE) // 可靠传输
            .durability(RMW_QOS_POLICY_DURABILITY_TRANSIENT_LOCAL); // 持久化

        auto best_effort_qos = rclcpp::QoS(10)
            .reliability(RMW_QOS_POLICY_RELIABILITY_BEST_EFFORT); // 尽力传输

        ...
    }
}
```

4.2.3 C++话题编程示例

创建话题通信功能包

```
# 创建功能包
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_cmake topic_example \
    --dependencies rclcpp std_msgs
```

命令解析：

- ros2 pkg create

- ros2: ROS2的命令行工具
- pkg: 包管理子命令
- **create: 创建新包的操作**
- --build-type ament_cmake
 - --build-type: 指定构建系统类型
 - **ament_cmake: 使用CMake构建系统，这是C++包的标准构建方式**
 - 其他选项还有：ament_python（Python包）
- topic_example
 - **这是要创建的包名**
 - 包名必须符合ROS2命名约定（小写字母、数字、下划线）
- --dependencies rclcpp std_msgs
- --dependencies: 指定包的依赖项
- **rclcpp: ROS2的C++客户端库，C++节点开发的核心库**
- **std_msgs: 标准消息类型库，包含基本的消息类型如String、Int32等**

简单发布者示例

源文件（src/simple_publisher.cpp）：

src/topic_example/src/simple_publisher.cpp

这里话题是：**hello_topic**

```
/**
 * @file topic_example/src/simple_publisher.cpp
 * @brief ROS2话题发布者示例节点
 * @description 演示如何创建发布者节点，定时发布字符串消息到指定话题
 */

// 包含ROS2 C++客户端库的头文件
#include <rclcpp/rclcpp.hpp>
// 包含标准字符串消息类型的头文件
#include <std_msgs/msg/string.hpp>
// 包含时间相关的头文件，用于定时器功能
#include <chrono>

// 使用chrono字面量，方便时间单位的表示（如2s表示2秒）
using namespace std::chrono_literals;

/**
 * @class SimplePublisher
 * @brief 简单发布者节点类
 * @description 继承自rclcpp::Node，实现了定时发布字符串消息的功能
 */
class SimplePublisher : public rclcpp::Node
```

```

{
public:
    /**
     * @brief 构造函数
     * @description 初始化节点，设置节点名称为"simple_publisher"，创建发布者和定时器
     */
    SimplePublisher() : Node("simple_publisher"), count_(0)
    {
        // 创建发布者对象
        // 模板参数指定消息类型为std_msgs::msg::String
        // "hello_topic"是话题名称，10是队列大小（QoS设置）
        publisher_ = this->create_publisher<std_msgs::msg::String>("hello_topic",
10);

        // 创建墙时钟定时器，每2秒执行一次timer_callback函数
        // 使用std::bind将成员函数绑定到定时器回调
        timer_ = this->create_wall_timer(
            2s, std::bind(&SimplePublisher::timer_callback, this));

        // 输出节点启动信息到控制台
        RCLCPP_INFO(this->get_logger(), "简单发布者节点已启动");
    }

private:
    /**
     * @brief 定时器回调函数
     * @description 每当定时器触发时调用此函数，创建并发布字符串消息
     */
    void timer_callback()
    {
        // 创建std_msgs::msg::String类型的消息对象
        auto message = std_msgs::msg::String();

        // 设置消息内容，包含固定文本和递增的消息编号
        message.data = "Hello ROS2! 消息编号: " + std::to_string(count_);

        // 输出即将发布的消息内容到控制台
        // c_str()将std::string转换为C风格字符串用于printf格式化
        RCLCPP_INFO(this->get_logger(), "发布消息: '%s'", message.data.c_str());

        // 通过发布者将消息发布到"hello_topic"话题
        publisher_->publish(message);

        // 消息计数器自增，用于下次消息编号
        count_++;
    }

    // 定时器对象的智能指针，管理定时器的生命周期
    rclcpp::TimerBase::SharedPtr timer_;

    // 发布者对象的智能指针，用于发布std_msgs::msg::String类型的消息
    rclcpp::Publisher<std_msgs::msg::String>::SharedPtr publisher_;

```

```

    // 消息计数器，记录已发布的消息数量
    size_t count_;
};

/**
 * @brief 主函数
 * @description 程序入口点，负责初始化ROS2环境、创建和运行发布者节点
 */
int main(int argc, char * argv[])
{
    // 初始化ROS2通信中间件
    // 必须在使用任何ROS2功能之前调用
    rclcpp::init(argc, argv);

    // 创建SimplePublisher节点并进入事件循环
    // make_shared创建智能指针，spin开始处理回调函数
    rclcpp::spin(std::make_shared<SimplePublisher>());

    // 清理ROS2资源，关闭通信中间件
    rclcpp::shutdown();

    // 程序正常退出
    return 0;
}

```

简单订阅者示例

源文件 (src/simple_subscriber.cpp) :

src/topic_example/src/simple_subscriber.cpp

```

/**
 * @file topic_example/src/simple_subscriber.cpp
 * @brief ROS2话题订阅者示例节点
 * @description 演示如何创建订阅者节点，接收并处理来自指定话题的字符串消息
 */

// 包含ROS2 C++客户端库的头文件
#include <rclcpp/rclcpp.hpp>
// 包含标准字符串消息类型的头文件
#include <std_msgs/msg/string.hpp>

/**
 * @class SimpleSubscriber
 * @brief 简单订阅者节点类
 * @description 继承自rclcpp::Node，实现了订阅指定话题并处理接收消息的功能
 */
class SimpleSubscriber : public rclcpp::Node
{
public:
    /**

```

```

* @brief 构造函数
* @description 初始化节点，设置节点名称为"simple_subscriber"，创建订阅者对象
*/
SimpleSubscriber() : Node("simple_subscriber")
{
    // 创建订阅者对象
    // 模板参数指定消息类型为std_msgs::msg::String
    // "hello_topic"是要订阅的话题名称，10是队列大小（QoS设置）
    // 使用std::bind绑定回调函数，std::placeholders::_1表示消息参数占位符
    subscription_ = this->create_subscription<std_msgs::msg::String>(
        "hello_topic", 10,
        std::bind(&SimpleSubscriber::topic_callback, this,
std::placeholders::_1));

    // 输出节点启动信息到控制台
    RCLCPP_INFO(this->get_logger(), "简单订阅者节点已启动");
}

private:
/**
* @brief 话题回调函数
* @param msg 接收到的消息智能指针，类型为std_msgs::msg::String::SharedPtr
* @description 每当有新消息到达订阅的话题时，此函数会被自动调用
*/
void topic_callback(const std_msgs::msg::String::SharedPtr msg) const
{
    // 输出接收到的消息内容到控制台
    // msg->data访问消息的字符串数据，c_str()转换为C风格字符串
    RCLCPP_INFO(this->get_logger(), "接收到消息: '%s'", msg->data.c_str());
}

// 订阅者对象的智能指针，用于订阅std_msgs::msg::String类型的消息
// 管理订阅者的生命周期，当对象销毁时自动取消订阅
rclcpp::Subscription<std_msgs::msg::String>::SharedPtr subscription_;
};

/**
* @brief 主函数
* @description 程序入口点，负责初始化ROS2环境、创建和运行订阅者节点
*/
int main(int argc, char * argv[])
{
    // 初始化ROS2通信中间件
    // 必须在使用任何ROS2功能之前调用
    rclcpp::init(argc, argv);

    // 创建SimpleSubscriber节点并进入事件循环
    // make_shared创建智能指针，spin进入阻塞状态等待消息到达
    // 当有消息到达时会自动调用回调函数处理
    rclcpp::spin(std::make_shared<SimpleSubscriber>());

    // 清理ROS2资源，关闭通信中间件

```

```

    rclcpp::shutdown();

    // 程序正常退出
    return 0;
}

```

稍微复杂的数值发布订阅示例

源文件 (src/number_publisher.cpp) :

```

/**
 * @file topic_example/src/number_publisher.cpp
 * @brief ROS2数值发布者示例节点
 * @description 演示如何创建发布者节点，定时发布递增的整数消息到指定话题
 */

// 包含ROS2 C++客户端库的头文件
#include <rclcpp/rclcpp.hpp>
// 包含标准32位整数消息类型的头文件
#include <std_msgs/msg/int32.hpp>
// 包含时间相关的头文件，用于定时器功能
#include <chrono>

// 使用chrono字面量，方便时间单位的表示（如1s表示1秒）
using namespace std::chrono_literals;

/**
 * @class NumberPublisher
 * @brief 数值发布者节点类
 * @description 继承自rclcpp::Node，实现了定时发布递增整数消息的功能
 */
class NumberPublisher : public rclcpp::Node
{
public:
    /**
     * @brief 构造函数
     * @description 初始化节点，设置节点名称为"number_publisher"，创建发布者和定时器，初始化数值为0
     */
    NumberPublisher() : Node("number_publisher"), number_(0)
    {
        // 创建发布者对象
        // 模板参数指定消息类型为std_msgs::msg::Int32
        // "numbers"是话题名称，10是队列大小（QoS设置）
        publisher_ = this->create_publisher<std_msgs::msg::Int32>("numbers", 10);

        // 创建墙时钟定时器，每1秒执行一次publish_number函数
        // 使用std::bind将成员函数绑定到定时器回调
        timer_ = this->create_wall_timer(
            1s, std::bind(&NumberPublisher::publish_number, this));

        // 输出节点启动信息到控制台

```



```

        RCLCPP_INFO(this->get_logger(), "数值发布者节点已启动");
    }

private:
    /**
     * @brief 发布数值函数
     * @description 定时器回调函数，每次被调用时创建并发布当前数值，然后递增数值
     */
    void publish_number()
    {
        // 创建std_msgs::msg::Int32类型的消息对象
        auto msg = std_msgs::msg::Int32();

        // 设置消息数据为当前数值
        msg.data = number_;

        // 输出即将发布的数值到控制台
        RCLCPP_INFO(this->get_logger(), "发布数值: %d", msg.data);

        // 通过发布者将消息发布到"numbers"话题
        publisher_>publish(msg);

        // 数值递增，为下次发布准备
        number_++;
    }

    // 定时器对象的智能指针，用于管理定时器的生命周期
    rclcpp::TimerBase::SharedPtr timer_;

    // 发布者对象的智能指针，用于发布std_msgs::msg::Int32类型的消息
    rclcpp::Publisher<std_msgs::msg::Int32>::SharedPtr publisher_;

    // 当前数值变量，记录下一个要发布的整数值
    int32_t number_;
};

/**
 * @brief 主函数
 * @description 程序入口点，负责初始化ROS2环境、创建和运行数值发布者节点
 */
int main(int argc, char * argv[])
{
    // 初始化ROS2通信中间件
    // 必须在使用任何ROS2功能之前调用
    rclcpp::init(argc, argv);

    // 创建NumberPublisher节点并进入事件循环
    // make_shared创建智能指针，spin开始处理回调函数
    // 定时器会定期触发发布数值的操作
    rclcpp::spin(std::make_shared<NumberPublisher>());

    // 清理ROS2资源，关闭通信中间件

```

```

    rclcpp::shutdown();

    // 程序正常退出
    return 0;
}

```

源文件 (src/number_subscriber.cpp) :

```

/**
 * @file topic_example/src/number_subscriber.cpp
 * @brief ROS2数值订阅者示例节点
 * @description 演示如何创建订阅者节点，接收整数消息并计算累计和
 */

// 包含ROS2 C++客户端库的头文件
#include <rclcpp/rclcpp.hpp>
// 包含标准32位整数消息类型的头文件
#include <std_msgs/msg/int32.hpp>

/**
 * @class NumberSubscriber
 * @brief 数值订阅者节点类
 * @description 继承自rclcpp::Node，实现了订阅整数话题并计算累计和的功能
 */
class NumberSubscriber : public rclcpp::Node
{
public:
    /**
     * @brief 构造函数
     * @description 初始化节点，设置节点名称为"number_subscriber"，创建订阅者对象，初始化累计和为0
     */
    NumberSubscriber() : Node("number_subscriber"), sum_(0)
    {
        // 创建订阅者对象
        // 模板参数指定消息类型为std_msgs::msg::Int32
        // "numbers"是要订阅的话题名称，10是队列大小（QoS设置）
        // 使用std::bind绑定回调函数，std::placeholders::_1表示消息参数占位符
        subscription_ = this->create_subscription<std_msgs::msg::Int32>(
            "numbers", 10,
            std::bind(&NumberSubscriber::number_callback, this,
std::placeholders::_1));

        // 输出节点启动信息到控制台
        RCLCPP_INFO(this->get_logger(), "数值订阅者节点已启动");
    }

private:
    /**
     * @brief 数值消息回调函数
     * @param msg 接收到的整数消息智能指针，类型为std_msgs::msg::Int32::SharedPtr

```

```

    * @description 每当有新的整数消息到达订阅的话题时，此函数会被自动调用，累加数值并输出统计信息
    */
void number_callback(const std_msgs::msg::Int32::SharedPtr msg)
{
    // 将接收到的数值累加到总和中
    sum_ += msg->data;

    // 输出接收到的数值和当前累计和
    // msg->data访问消息中的整数数据
    RCLCPP_INFO(this->get_logger(), "接收数值: %d, 累计和: %d", msg->data, sum_);
}

// 订阅者对象的智能指针，用于订阅std_msgs::msg::Int32类型的消息
// 管理订阅者的生命周期，当对象销毁时自动取消订阅
rclcpp::Subscription<std_msgs::msg::Int32>::SharedPtr subscription_;

// 累计和变量，用于统计接收到的所有数值的总和
int32_t sum_;
};

/**
 * @brief 主函数
 * @description 程序入口点，负责初始化ROS2环境、创建和运行数值订阅者节点
 */
int main(int argc, char * argv[])
{
    // 初始化ROS2通信中间件
    // 必须在使用任何ROS2功能之前调用
    rclcpp::init(argc, argv);

    // 创建NumberSubscriber节点并进入事件循环
    // make_shared创建智能指针，spin进入阻塞状态等待消息到达
    // 当有数值消息到达时会自动调用回调函数进行处理和累加
    rclcpp::spin(std::make_shared<NumberSubscriber>());

    // 清理ROS2资源，关闭通信中间件
    rclcpp::shutdown();

    // 程序正常退出
    return 0;
}

```

编译配置

CMakeLists.txt:

```

cmake_minimum_required(VERSION 3.8)
project(topic_example)

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
    add_compile_options(-Wall -Wextra -Wpedantic)

```

```

endif()

# 查找依赖包
find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)

# 创建可执行文件
add_executable(simple_publisher src/simple_publisher.cpp)
add_executable(simple_subscriber src/simple_subscriber.cpp)
add_executable(number_publisher src/number_publisher.cpp)
add_executable(number_subscriber src/number_subscriber.cpp)

# 设置依赖
ament_target_dependencies(simple_publisher rclcpp std_msgs)
ament_target_dependencies(simple_subscriber rclcpp std_msgs)
ament_target_dependencies(number_publisher rclcpp std_msgs)
ament_target_dependencies(number_subscriber rclcpp std_msgs)

# 安装可执行文件
install(TARGETS
  simple_publisher
  simple_subscriber
  number_publisher
  number_subscriber
  DESTINATION lib/${PROJECT_NAME})

ament_package()

```

编译和运行

这里两个范例的节点名和话题名：

- 节点名：simple_publisher, simple_subscriber, 话题：hello_topic
- 节点名：number_publisher, number_subscriber, 话题：numbers

程序运行后，可以使用：`watch -n 1 "ros2 node list"` 实时查看话题

```

# 1. 编译功能包
cd ~/ros2_ws
colcon build --packages-select topic_example

# 2. 设置环境
source install/setup.bash

# 3. 运行发布者（终端1），也可以看多个终端 多个发布
ros2 run topic_example simple_publisher

# 4. 运行订阅者（终端2）
ros2 run topic_example simple_subscriber

# 5. 运行稍微复杂的数值发布者（终端1）

```

```
ros2 run topic_example number_publisher

# 6. 运行稍微复杂的数值订阅者（终端2）
ros2 run topic_example number_subscriber
```

命令行话题操作示例

查看话题信息

```
# 查看所有话题
ros2 topic list

# 查看话题详细信息
ros2 topic info /hello_topic

# 查看话题消息类型
ros2 topic type /hello_topic

# 实时监听话题消息
ros2 topic echo /hello_topic

# 查看话题发布频率
ros2 topic hz /hello_topic

# 查看话题带宽使用
ros2 topic bw /hello_topic
```

命令行发布消息

```
# 发布字符串消息
ros2 topic pub /hello_topic std_msgs/msg/String '{data: "命令行发布的消息"}'

# 持续发布消息（每2秒一次）
ros2 topic pub /hello_topic std_msgs/msg/String '{data: "持续消息"}' --rate 0.5

# 发布数值消息
ros2 topic pub /numbers std_msgs/msg/Int32 '{data: 42}'

# 发布一次后退出
ros2 topic pub --once /hello_topic std_msgs/msg/String '{data: "单次消息"}'
```

运行效果演示

发布者输出：

```
[INFO] [1702123456.789] [simple_publisher]: 简单发布者节点已启动
[INFO] [1702123458.789] [simple_publisher]: 发布消息: 'Hello ROS2! 消息编号: 0'
[INFO] [1702123460.789] [simple_publisher]: 发布消息: 'Hello ROS2! 消息编号: 1'
[INFO] [1702123462.789] [simple_publisher]: 发布消息: 'Hello ROS2! 消息编号: 2'
```

订阅者输出：

```
[INFO] [1702123457.123] [simple_subscriber]: 简单订阅者节点已启动
[INFO] [1702123458.790] [simple_subscriber]: 接收到消息: 'Hello ROS2! 消息编号: 0'
[INFO] [1702123460.791] [simple_subscriber]: 接收到消息: 'Hello ROS2! 消息编号: 1'
[INFO] [1702123462.792] [simple_subscriber]: 接收到消息: 'Hello ROS2! 消息编号: 2'
```

多节点通信测试

```
# 同时运行多个发布者和订阅者
# 终端1: 发布者1
ros2 run topic_example simple_publisher

# 终端2: 发布者2（重映射话题名）
ros2 run topic_example simple_publisher --ros-args --remap hello_topic:=hello_topic

# 终端3: 订阅者1
ros2 run topic_example simple_subscriber

# 终端4: 订阅者2
ros2 run topic_example simple_subscriber

# 终端5: 查看话题连接
ros2 topic info /hello_topic
```

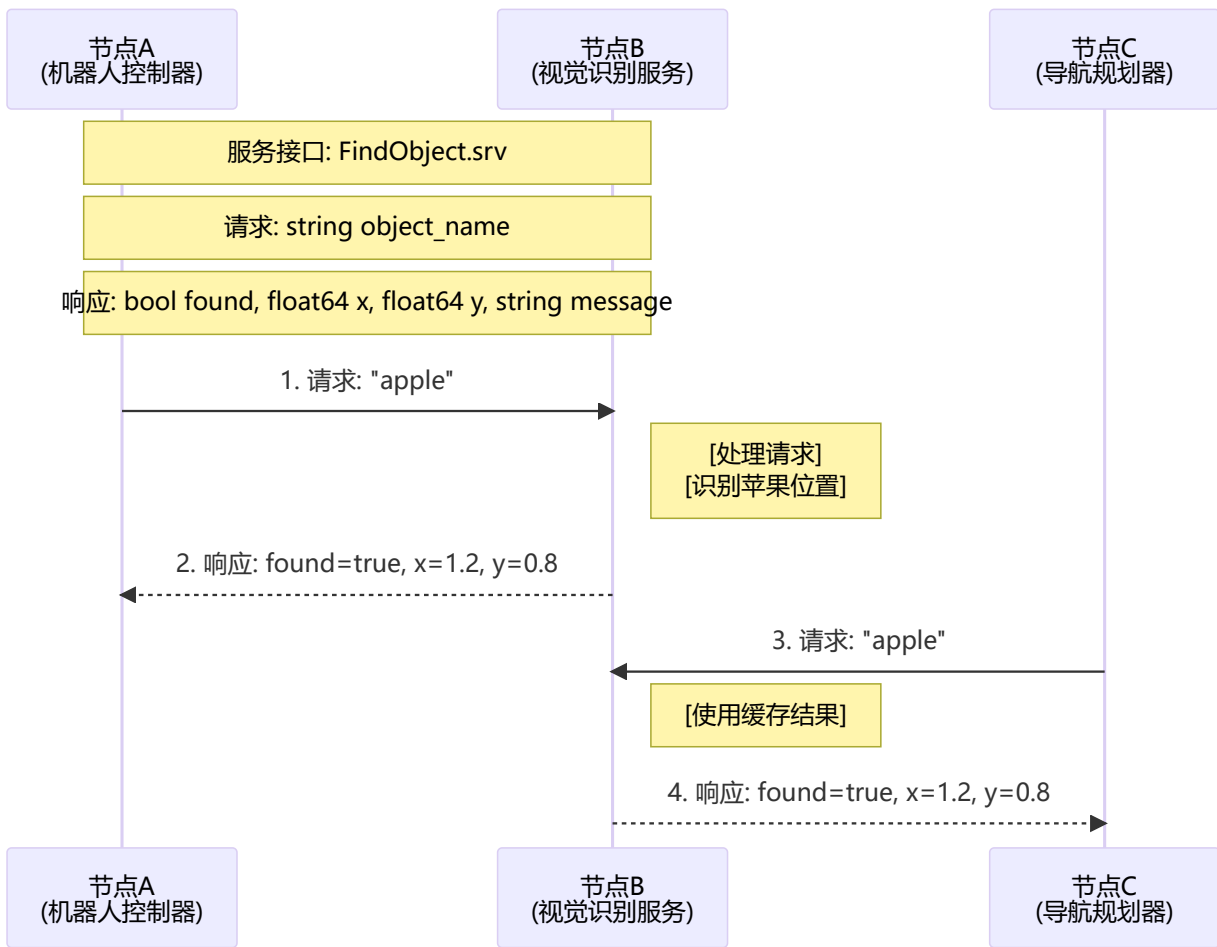
4.3 通信机制-服务

4.3.1 什么是服务

定义和概念

- **服务 (Service)**：ROS2中节点间进行**请求-响应** 通信的机制
- **特点**：客户端发送请求，服务端处理后返回响应
- **应用场景**：执行特定任务、获取状态信息、触发动作等

服务通信图示



4.3.2 服务通信模型

同步通信特点

同步通信流程:

客户端 → 发送请求 → 阻塞等待 → 接收响应 → 继续执行

服务端 → 接收请求 → 处理任务 → 发送响应 → 等待下一个请求

异步编程通信特点

异步通信流程：
客户端 → 发送请求 → 其他任务 → 处理响应（回调触发）

一对多通信模式

- 一个服务端：提供特定功能的节点
- 多个客户端：可以同时向同一服务端发送请求
- 队列处理：服务端按顺序处理客户端请求

服务接口组成

这里的.srv是重要技术点。

服务接口文件（.srv）：
├─ 请求部分（Request）
| ├── 输入参数1
| ├── 输入参数2
| └─ ...
├─ 分隔符（---）
└─ 响应部分（Response）
 ├── 输出参数1
 ├── 输出参数2
 └─ ...

4.3.3 服务通信C++编程示例

创建服务通信功能包

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_cmake service_example \
  --dependencies rclcpp std_srvs
```

命令解析：

- std_srvs: 标准服务类型库

与topic_example的区别：

方面	topic_example	service_example
通信模式	发布-订阅	请求-响应
消息库	std_msgs	std_srvs
通信特点	异步、持续	同步、临时

简单服务端示例

这里的请求响应格式:

```
Request:
  bool data

Response:
  bool success,
  string message
```

源文件 (src/simple_service_server.cpp) :

src/service_example/src/simple_service_server.cpp

```
/**
 * @file service_example/src/simple_service_server.cpp
 * @brief ROS2简单服务服务器示例节点
 * @description 演示如何创建服务服务器，处理客户端请求并返回响应，实现机器人控制服务
 */

// 包含ROS2 C++客户端库的头文件
#include <rclcpp/rclcpp.hpp>
// 包含标准布尔设置服务类型的头文件
#include <std_srvs/srv/set_bool.hpp>
// 包含智能指针相关的头文件
#include <memory>

/**
 * @class SimpleServiceServer
 * @brief 简单服务服务器节点类
 * @description 继承自rclcpp::Node，实现了机器人控制服务，处理启用/禁用机器人的请求
 */
class SimpleServiceServer : public rclcpp::Node
{
public:
  /**
   * @brief 构造函数
   * @description 初始化节点，设置节点名称为"simple_service_server"，创建服务服务器，初始化
   机器人状态为禁用
   */
  SimpleServiceServer() : Node("simple_service_server"), robot_enabled_(false)
  {
    // 创建服务服务器对象
    // 模板参数指定服务类型为std_srvs::srv::SetBool
    // "robot_control"是服务名称，客户端通过此名称调用服务
    // 使用std::bind绑定服务处理函数，std::placeholders用于参数占位
    service_ = this->create_service<std_srvs::srv::SetBool>(
```

```

        "robot_control",
        std::bind(&SimpleServiceServer::handle_service, this,
                  std::placeholders::_1, std::placeholders::_2));

    // 输出服务启动信息到控制台
    RCLCPP_INFO(this->get_logger(), "机器人控制服务已启动");
}

private:
    /**
     * @brief 服务请求处理函数
     * @param request 客户端请求对象，包含布尔值数据（true=启用，false=禁用）
     * @param response 服务器响应对象，用于返回处理结果和状态信息
     * @description 当有客户端调用服务时，此函数会被自动调用来处理请求并生成响应
     */
    void handle_service(
        const std::shared_ptr<std_srvs::srv::SetBool::Request> request,
        std::shared_ptr<std_srvs::srv::SetBool::Response> response)
    {
        // 根据请求数据更新机器人状态
        // request->data 包含客户端发送的布尔值（启用/禁用指令）
        robot_enabled_ = request->data;

        // 设置响应的成功标志
        // 在实际应用中，这里可能需要根据实际操作结果设置
        response->success = true;

        // 根据机器人状态设置不同的响应消息
        if (robot_enabled_) {
            response->message = "机器人已启用";
            RCLCPP_INFO(this->get_logger(), "收到请求：启用机器人");
        } else {
            response->message = "机器人已禁用";
            RCLCPP_INFO(this->get_logger(), "收到请求：禁用机器人");
        }

        // 输出服务处理完成信息
        RCLCPP_INFO(this->get_logger(), "服务响应已发送");
    }

    // 服务服务器对象的智能指针，用于管理服务的生命周期
    // 当对象销毁时，服务会自动注销
    rclcpp::Service<std_srvs::srv::SetBool>::SharedPtr service_;

    // 机器人状态变量，记录当前机器人是否已启用
    // true表示启用，false表示禁用
    bool robot_enabled_;
};

/**
 * @brief 主函数
 * @description 程序入口点，负责初始化ROS2环境、创建和运行服务服务器节点

```

```

*/
int main(int argc, char * argv[])
{
    // 初始化ROS2通信中间件
    // 必须在使用任何ROS2功能之前调用
    rclcpp::init(argc, argv);

    // 创建SimpleServiceServer节点并进入事件循环
    // make_shared创建智能指针，spin进入阻塞状态等待服务请求
    // 当有客户端调用服务时会自动触发服务处理函数
    rclcpp::spin(std::make_shared<SimpleServiceServer>());

    // 清理ROS2资源，关闭通信中间件
    rclcpp::shutdown();

    // 程序正常退出
    return 0;
}

```

简单客户端示例

源文件 (src/simple_service_client.cpp) :

src/service_example/src/simple_service_client.cpp

```

/**
 * @file service_example/src/simple_service_client.cpp
 * @brief ROS2简单服务客户端示例节点
 * @description 演示如何创建服务客户端，调用远程服务并处理响应，实现机器人控制客户端
 */

// 包含ROS2 C++客户端库的头文件
#include <rclcpp/rclcpp.hpp>
// 包含标准布尔设置服务类型的头文件
#include <std_srvs/srv/set_bool.hpp>
// 包含时间相关的头文件，用于延迟和超时控制
#include <chrono>
// 包含智能指针相关的头文件
#include <memory>

// 使用chrono字面量，方便时间单位的表示（如1s表示1秒）
using namespace std::chrono_literals;

/**
 * @class SimpleServiceClient
 * @brief 简单服务客户端节点类
 * @description 继承自rclcpp::Node，实现了机器人控制服务的客户端，发送启用/禁用请求
 */
class SimpleServiceClient : public rclcpp::Node
{
public:
    /**

```

```

    * @brief 构造函数
    * @description 初始化节点，设置节点名称为"simple_service_client"，创建服务客户端，等待服务可用并发送测试请求
    */
SimpleServiceClient() : Node("simple_service_client")
{
    // 创建服务客户端对象
    // 模板参数指定服务类型为std_srvs::srv::SetBool
    // "robot_control"是要调用的服务名称，必须与服务端一致
    client_ = this->create_client<std_srvs::srv::SetBool>("robot_control");

    // 输出客户端启动信息到控制台
    RCLCPP_INFO(this->get_logger(), "服务客户端已启动");

    // 等待服务变为可用状态
    // 这是一个阻塞循环，直到服务服务器上线
    while (!client_->wait_for_service(1s)) {
        // 检查ROS2系统是否正常运行
        if (!rclcpp::ok()) {
            RCLCPP_ERROR(this->get_logger(), "等待服务时被中断");
            return;
        }
        // 每次等待超时后输出提示信息
        RCLCPP_INFO(this->get_logger(), "等待服务可用...");
    }

    // 发送测试请求序列
    // 首先启用机器人
    send_request(true);    // 启用机器人

    // 等待2秒后禁用机器人，演示服务调用的时序
    std::this_thread::sleep_for(2s);

    // 然后禁用机器人
    send_request(false);   // 禁用机器人
}

private:
    /**
    * @brief 发送服务请求函数
    * @param enable 布尔值参数，true表示启用机器人，false表示禁用机器人
    * @description 创建服务请求，异步发送给服务器，并等待响应结果
    */
    void send_request(bool enable)
    {
        // 创建服务请求对象
        auto request = std::make_shared<std_srvs::srv::SetBool::Request>();

        // 设置请求数据
        request->data = enable;

        // 输出即将发送的请求信息

```

```

RCLCPP_INFO(this->get_logger(), "发送请求: %s机器人",
            enable ? "启用" : "禁用");

// 异步发送服务请求
// async_send_request返回一个future对象, 用于获取异步结果
auto future = client->async_send_request(request);

// 等待服务响应
// spin_until_future_complete会阻塞直到future完成或超时
// 这种方式确保在单线程环境中正确处理异步响应
if (rclcpp::spin_until_future_complete(this->get_node_base_interface(),
future) ==
    rclcpp::FutureReturnCode::SUCCESS)
{
    // 服务调用成功, 获取响应结果
    auto response = future.get();

    // 输出服务响应信息
    // 显示操作是否成功以及服务器返回的消息
    RCLCPP_INFO(this->get_logger(), "收到响应: 成功=%s, 消息='%s'",
                response->success ? "true" : "false",
                response->message.c_str());
} else {
    // 服务调用失败(超时、网络错误等)
    RCLCPP_ERROR(this->get_logger(), "服务调用失败");
}
}

// 服务客户端对象的智能指针, 用于调用std_srvs::srv::SetBool类型的服务
// 管理客户端的生命周期, 当对象销毁时自动清理连接
rclcpp::Client<std_srvs::srv::SetBool>::SharedPtr client_;
};

/**
 * @brief 主函数
 * @description 程序入口点, 负责初始化ROS2环境、创建和运行服务客户端节点
 */
int main(int argc, char * argv[])
{
    // 初始化ROS2通信中间件
    // 必须在使用任何ROS2功能之前调用
    rclcpp::init(argc, argv);

    // 创建SimpleServiceClient节点
    // 构造函数中会自动执行服务调用序列
    auto node = std::make_shared<SimpleServiceClient>();

    // 客户端任务完成后直接关闭ROS2系统
    // 与服务器不同, 客户端通常执行完任务就退出
    rclcpp::shutdown();

    // 程序正常退出

```

```
    return 0;
}
```

自定义服务接口示例

首先创建自定义服务接口：

srv/AddTwoInts.srv:

src/service_example/srv/AddTwoInts.srv

```
int64 a
int64 b
---
int64 sum
string message
```

服务端实现 (src/add_service_server.cpp) :

src/service_example/src/add_service_server.cpp

```
/**
 * @file service_example/src/add_service_server.cpp
 * @brief ROS2自定义加法服务服务器示例节点
 * @description 演示如何创建使用自定义服务接口的服务服务器，实现两个整数相加的服务
 */

// 包含ROS2 C++客户端库的头文件
#include <rclcpp/rclcpp.hpp>
// 包含自定义的加法服务接口头文件
// 这个头文件是从srv/AddTwoInts.srv自动生成的
#include <service_example/srv/add_two_ints.hpp>

/**
 * @class AddServiceServer
 * @brief 加法服务服务器节点类
 * @description 继承自rclcpp::Node，实现了两个整数相加的服务，使用自定义服务接口
 */
class AddServiceServer : public rclcpp::Node
{
public:
    /**
     * @brief 构造函数
     * @description 初始化节点，设置节点名称为"add_service_server"，创建加法服务服务器
     */
    AddServiceServer() : Node("add_service_server")
    {
        // 创建服务服务器对象
        // 模板参数指定自定义服务类型为service_example::srv::AddTwoInts
        // "add_two_ints"是服务名称，客户端通过此名称调用服务
        // 使用std::bind绑定回调函数，std::placeholders用于参数占位
        service_ = this->create_service<service_example::srv::AddTwoInts>(<
```

```

        "add_two_ints",
        std::bind(&AddServiceServer::add_callback, this,
                  std::placeholders::_1, std::placeholders::_2));

    // 输出服务启动信息到控制台
    RCLCPP_INFO(this->get_logger(), "加法服务已启动");
}

private:
    /**
     * @brief 加法服务回调函数
     * @param request 客户端请求对象，包含两个待相加的整数(a, b)
     * @param response 服务器响应对象，用于返回计算结果和状态信息
     * @description 当有客户端调用加法服务时，此函数会被自动调用来处理加法运算并返回结果
     */
    void add_callback(
        const std::shared_ptr<service_example::srv::AddTwoInts::Request> request,
        std::shared_ptr<service_example::srv::AddTwoInts::Response> response)
    {
        // 执行加法运算
        // request->a 和 request->b 是客户端发送的两个整数
        // response->sum 用于存储计算结果
        response->sum = request->a + request->b;

        // 设置响应消息，告知客户端计算已完成
        response->message = "计算完成";

        // 输出服务处理信息到控制台
        // 显示接收到的两个数和计算结果
        RCLCPP_INFO(this->get_logger(),
                    "收到请求: %ld + %ld = %ld",
                    request->a, request->b, response->sum);
    }

    // 服务服务器对象的智能指针，用于管理加法服务的生命周期
    // 当对象销毁时，服务会自动注销
    rclcpp::Service<service_example::srv::AddTwoInts>::SharedPtr service_;
};

/**
 * @brief 主函数
 * @description 程序入口点，负责初始化ROS2环境、创建和运行加法服务服务器节点
 */
int main(int argc, char * argv[])
{
    // 初始化ROS2通信中间件
    // 必须在使用任何ROS2功能之前调用
    rclcpp::init(argc, argv);

    // 创建AddServiceServer节点并进入事件循环
    // make_shared创建智能指针，spin进入阻塞状态等待服务请求
    // 当有客户端调用加法服务时会自动触发回调函数

```

```

    rclcpp::spin(std::make_shared<AddServiceServer>());

    // 清理ROS2资源，关闭通信中间件
    rclcpp::shutdown();

    // 程序正常退出
    return 0;
}

```

客户端实现 (src/add_service_client.cpp) :

```

/**
 * @file service_example/src/add_service_client.cpp
 * @brief ROS2自定义加法服务客户端示例节点
 * @description 演示如何创建使用自定义服务接口的服务客户端，调用加法服务并处理响应
 */

// 包含ROS2 C++客户端库的头文件
#include <rclcpp/rclcpp.hpp>
// 包含自定义的加法服务接口头文件
// 这个头文件是从srv/AddTwoInts.srv自动生成的
#include <service_example/srv/add_two_ints.hpp>
// 包含时间相关的头文件，用于服务等待超时控制
#include <chrono>

// 使用chrono字面量，方便时间单位的表示（如1s表示1秒）
using namespace std::chrono_literals;

/**
 * @class AddServiceClient
 * @brief 加法服务客户端节点类
 * @description 继承自rclcpp::Node，实现了调用加法服务的客户端，使用自定义服务接口
 */
class AddServiceClient : public rclcpp::Node
{
public:
    /**
     * @brief 构造函数
     * @description 初始化节点，设置节点名称为"add_service_client"，创建服务客户端，等待服务可用
     */
    AddServiceClient() : Node("add_service_client")
    {
        // 创建服务客户端对象
        // 模板参数指定自定义服务类型为service_example::srv::AddTwoInts
        // "add_two_ints"是要调用的服务名称，必须与服务器端一致
        client_ = this->create_client<service_example::srv::AddTwoInts>
("add_two_ints");

        // 等待服务变为可用状态
        // 这是一个阻塞循环，直到加法服务服务器上线
        while (!client_->wait_for_service(1s)) {

```



```

        // 检查ROS2系统是否正常运行
        if (!rclcpp::ok()) {
            RCLCPP_ERROR(this->get_logger(), "等待服务时被中断");
            return;
        }
        // 每次等待超时后输出提示信息
        RCLCPP_INFO(this->get_logger(), "等待加法服务可用...");
    }

    // 输出客户端启动成功信息
    RCLCPP_INFO(this->get_logger(), "加法客户端已启动");
}

/**
 * @brief 发送加法服务请求函数
 * @param a 第一个整数操作数
 * @param b 第二个整数操作数
 * @description 创建加法服务请求，异步发送给服务器，并等待计算结果
 */
void send_request(int64_t a, int64_t b)
{
    // 创建自定义服务请求对象
    auto request = std::make_shared<service_example::srv::AddTwoInts::Request>();

    // 设置请求数据 - 两个待相加的整数
    request->a = a;
    request->b = b;

    // 输出即将发送的请求信息
    RCLCPP_INFO(this->get_logger(), "发送请求: %ld + %ld", a, b);

    // 异步发送服务请求
    // async_send_request返回一个future对象，用于获取异步结果
    auto future = client_->async_send_request(request);

    // 等待服务响应
    // spin_until_future_complete会阻塞直到future完成或超时
    // 这种方式确保在单线程环境中正确处理异步响应
    if (rclcpp::spin_until_future_complete(this->get_node_base_interface(),
future) ==
        rclcpp::FutureReturnCode::SUCCESS)
    {
        // 服务调用成功，获取响应结果
        auto response = future.get();

        // 输出服务响应信息
        // 显示计算的算式、结果和服务器返回的状态消息
        RCLCPP_INFO(this->get_logger(), "结果: %ld + %ld = %ld, 消息: %s",
            a, b, response->sum, response->message.c_str());
    } else {
        // 服务调用失败（超时、网络错误等）

```

```

        RCLCPP_ERROR(this->get_logger(), "服务调用失败");
    }
}

private:
    // 服务客户端对象的智能指针，用于调用service_example::srv::AddTwoInts类型的服务
    // 管理客户端的生命周期，当对象销毁时自动清理连接
    rclcpp::Client<service_example::srv::AddTwoInts>::SharedPtr client_;
};

/**
 * @brief 主函数
 * @description 程序入口点，负责初始化ROS2环境、创建客户端节点并发送测试请求
 */
int main(int argc, char * argv[])
{
    // 初始化ROS2通信中间件
    // 必须在使用任何ROS2功能之前调用
    rclcpp::init(argc, argv);

    // 创建AddServiceClient节点
    auto node = std::make_shared<AddServiceClient>();

    // 发送多个测试请求，演示不同的加法运算
    node->send_request(5, 3);      // 正数加法: 5 + 3 = 8
    node->send_request(10, 20);    // 较大数加法: 10 + 20 = 30
    node->send_request(-5, 15);    // 负数与正数加法: -5 + 15 = 10

    // 所有请求完成后关闭ROS2系统
    // 与服务器不同，客户端通常执行完任务就退出
    rclcpp::shutdown();

    // 程序正常退出
    return 0;
}

```

编译配置

CMakeLists.txt:

```

cmake_minimum_required(VERSION 3.8)
project(service_example)

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
    add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# 查找依赖包
find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(std_srvs REQUIRED)
find_package(rosidl_default_generators REQUIRED)

```

```

# 生成自定义服务接口
rosidl_generate_interfaces(${PROJECT_NAME}
  "srv/AddTwoInts.srv"
)

# 创建可执行文件
add_executable(simple_service_server src/simple_service_server.cpp)
add_executable(simple_service_client src/simple_service_client.cpp)
add_executable(add_service_server src/add_service_server.cpp)
add_executable(add_service_client src/add_service_client.cpp)

# 设置依赖
ament_target_dependencies(simple_service_server rclcpp std_srvs)
ament_target_dependencies(simple_service_client rclcpp std_srvs)
ament_target_dependencies(add_service_server rclcpp)
ament_target_dependencies(add_service_client rclcpp)

# 链接自定义接口
rosidl_target_interfaces(add_service_server ${PROJECT_NAME}
  "rosidl_typesupport_cpp")
rosidl_target_interfaces(add_service_client ${PROJECT_NAME}
  "rosidl_typesupport_cpp")

# 安装可执行文件
install(TARGETS
  simple_service_server
  simple_service_client
  add_service_server
  add_service_client
  DESTINATION lib/${PROJECT_NAME})

# 导出依赖
ament_export_dependencies(rosidl_default_runtime)

ament_package()

```

package.xml:

```

<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>service_example</name>
  <version>0.0.0</version>
  <description>ROS2 服务通信示例</description>
  <maintainer email="you@example.com">Your Name</maintainer>
  <license>Apache License 2.0</license>

  <buildtool_depend>ament_cmake</buildtool_depend>
  <depend>rclcpp</depend>

```

```

<depend>std_srvs</depend>

<!-- 自定义接口依赖 -->
<build_depend>roscpp</build_depend>
<exec_depend>roscpp</exec_depend>
<member_of_group>roscpp</member_of_group>

<test_depend>ament_lint_auto</test_depend>
<test_depend>ament_lint_common</test_depend>

<export>
  <build_type>ament_cmake</build_type>
</export>
</package>

```

编译和运行

```

# 1. 编译功能包
cd ~/ros2_ws
colcon build --packages-select service_example

# 2. 设置环境
source install/setup.bash

# 3. 运行简单服务端（终端1）
ros2 run service_example simple_service_server

# 4. 运行简单客户端（终端2）
ros2 run service_example simple_service_client

# 5. 运行加法服务端（终端3）
ros2 run service_example add_service_server

# 6. 运行加法客户端（终端4）
ros2 run service_example add_service_client

```

4.3.4 服务的命令行操作

查看服务信息

```
# 查看所有可用服务
ros2 service list

# 查看服务类型，这里 /robot_control对应simple_service_server的服务名
ros2 service type /robot_control

# 查看服务接口定义
ros2 interface show std_srvs/srv/SetBool

# 查看自定义服务接口
ros2 interface show service_example/srv/AddTwoInts
```

调用服务

```
# 调用简单服务（启用机器人）
ros2 service call /robot_control std_srvs/srv/SetBool '{data: true}'

# 调用简单服务（禁用机器人）
ros2 service call /robot_control std_srvs/srv/SetBool '{data: false}'

# 调用加法服务
ros2 service call /add_two_ints service_example/srv/AddTwoInts '{a: 5, b: 3}'
```

服务调试和监控

```
# 查找提供特定服务的节点
ros2 service find std_srvs/srv/SetBool

# 实时监控服务调用
ros2 service echo /robot_control

# 查看服务统计信息
ros2 service info /robot_control
```

运行效果演示

服务端输出：

```
[INFO] [1702123456.789] [simple_service_server]: 机器人控制服务已启动
[INFO] [1702123460.123] [simple_service_server]: 收到请求：启用机器人
[INFO] [1702123460.124] [simple_service_server]: 服务响应已发送
[INFO] [1702123462.456] [simple_service_server]: 收到请求：禁用机器人
[INFO] [1702123462.457] [simple_service_server]: 服务响应已发送
```

客户端输出：

```
[INFO] [1702123458.789] [simple_service_client]: 服务客户端已启动
[INFO] [1702123459.123] [simple_service_client]: 等待服务可用...
[INFO] [1702123460.123] [simple_service_client]: 发送请求: 启用机器人
[INFO] [1702123460.125] [simple_service_client]: 收到响应: 成功=true, 消息='机器人已启用'
[INFO] [1702123462.456] [simple_service_client]: 发送请求: 禁用机器人
[INFO] [1702123462.458] [simple_service_client]: 收到响应: 成功=true, 消息='机器人已禁用'
```

4.3.5 话题和服务对比

发布-订阅 (Topic)

- 通信模式：发布者-订阅者模式
- 数据流向：单向数据流
- 通信方式：异步通信
- 连接关系：多对多

服务 (Service)

- 通信模式：请求-响应模式
- 数据流向：双向数据交换
- 通信方式：同步通信
- 连接关系：多对一（多个客户端，一个服务端）

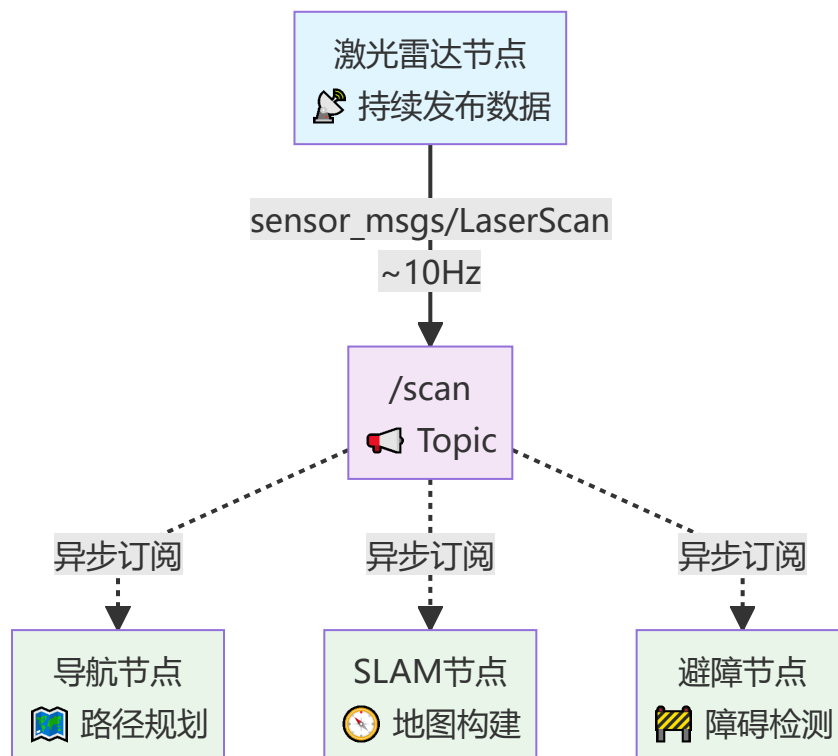
特别注意：这里讲的异步是 发送端只管发送数据；这里讲的同步是 发送请求后需要处理响应(这个响应可以是同步也可以是异步。)

详细特性对比表

特性	话题 发布-订阅 (Topic)	服务 (Service)
通信方式	异步、非阻塞	同步、阻塞
数据流向	单向（发布者→订阅者）	双向（客户端↔服务端）
连接模式	多对多	多对一
数据持续性	持续数据流	一次性请求-响应
实时性	高实时性	相对较低
可靠性	发布即忘记	有响应确认
适用数据	传感器数据、状态信息	命令执行、计算请求

通信模式图示对比

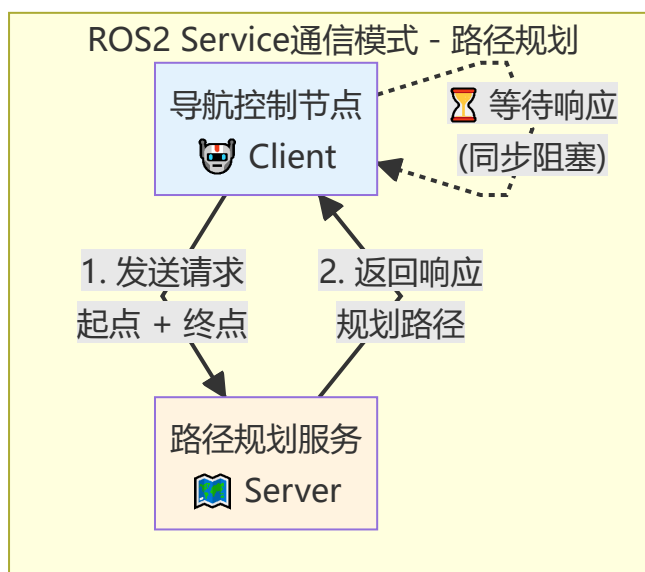
话题Topic通信模式



Topic通信的核心特征：

- 一对多：一个发布者，多个订阅者
- 异步：发布者不等待订阅者处理
- 解耦：发布者不需要知道订阅者的存在
- 广播：所有订阅者都能接收到相同的数据

服务Service通信模式



Service通信的核心特征：

- 客户端发送请求后等待响应
- 服务端处理完成后返回结果
- 一对一的请求-响应关系
- 同步处理，意思是需要处理返回结果

选择合适的通信方式是ROS2系统设计的关键，应该根据数据特性、频率要求和应用场景来决定。

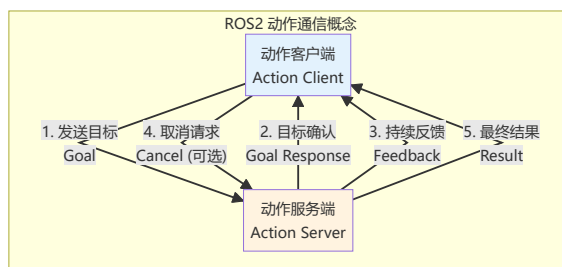
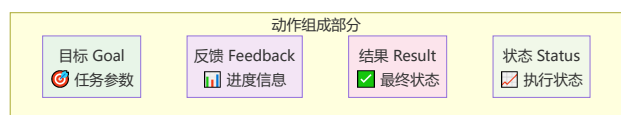
4.4 通信机制-动作

4.4.1 什么是动作（What is Action）

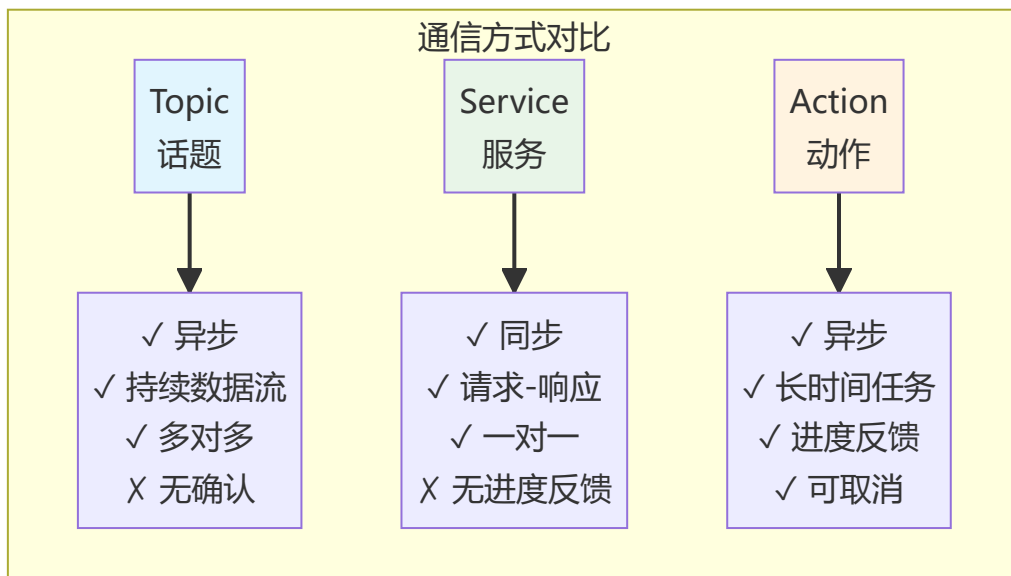
定义和概念

- **动作（Action）**：ROS2中用于管理长时间运行任务的通信机制
- **特点**：支持目标设置、实时反馈、结果返回和任务取消
- **应用场景**：导航、抓取、充电等需要较长时间完成的任务

ROS2动作概念图

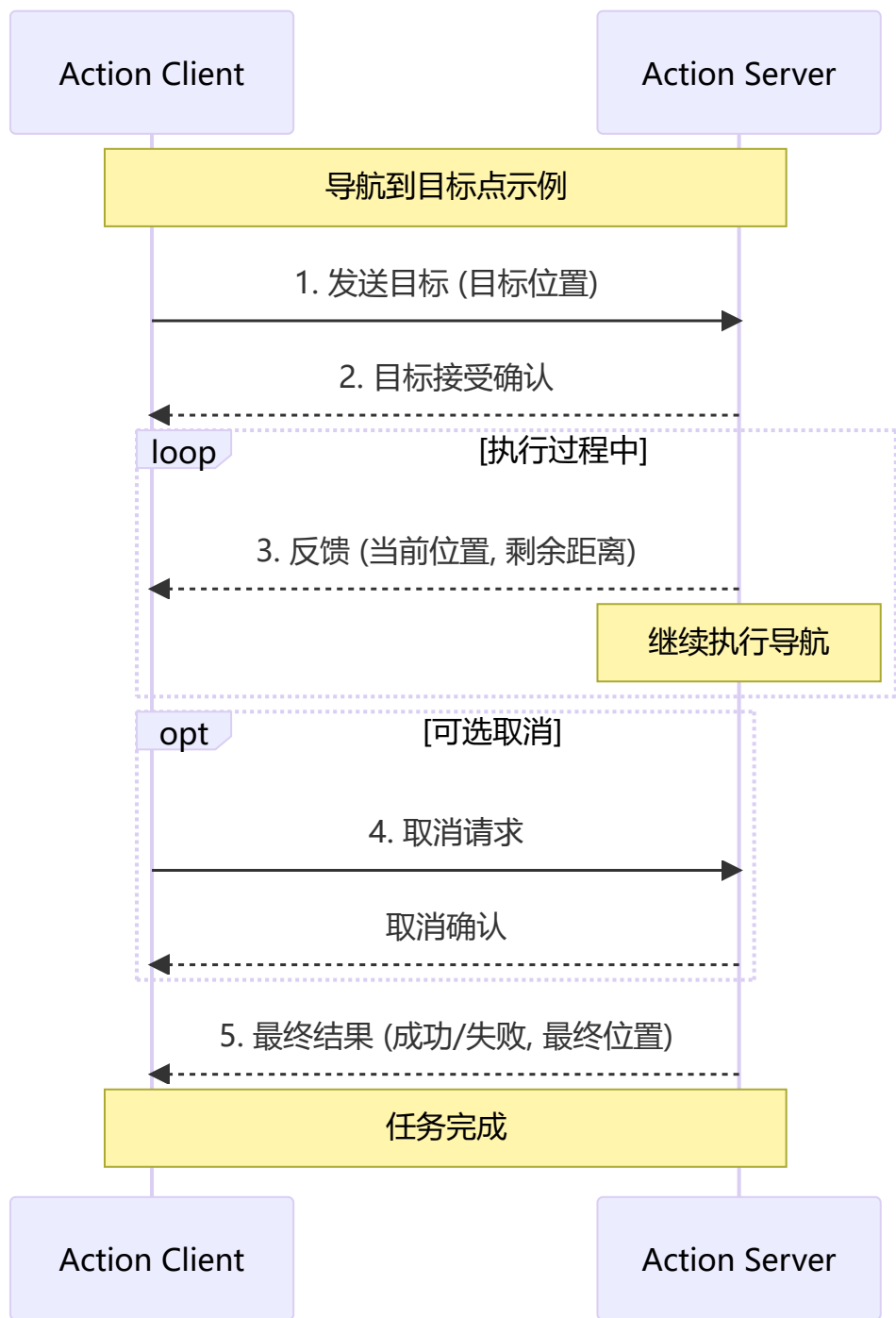


动作vs其他通信方式对比



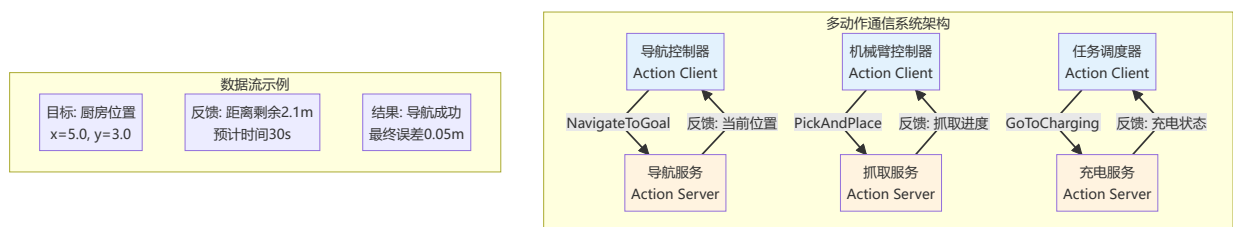
4.4.2 动作通信模型

动作通信时序图



4.4.3 C++动作通信接口示例

多动作通信机制数据交互示意图



动作接口定义的方法

创建动作接口功能包

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_cmake action_interfaces \
  --dependencies rosidl_default_generators rosidl_default_runtime \
  geometry_msgs builtin_interfaces
```

参数说明：

- 包名: action_interfaces
 - 命名约定: **以_interfaces结尾，表明这是接口定义包**
 - 用途: 专门用于定义动作（Action）接口
 - 动作: ROS2中用于长时间运行任务的通信模式
- 核心依赖: 接口生成工具
 - rosidl_default_generators
 - 作用: 编译时代码生成器
 - 功能: **将.action文件转换为C++/Python代码**
 - 类型: build_depend（构建依赖）
 - rosidl_default_runtime
 - 作用: 运行时支持库
 - 功能: **提供生成接口的运行时基础设施**
 - 类型: exec_depend（执行依赖）
- 消息类型依赖
 - **geometry_msgs**
 - 作用: 几何相关的标准消息类型，包含:
 - **Point, Pose, Twist（位置、姿态、速度）**
 - Vector3, Quaternion（向量、四元数）
 - 用途: 机器人导航、定位、运动控制
 - **builtin_interfaces**
 - 作用: ROS2内置的基础接口类型，包含:
 - Time, Duration（时间相关）
 - 基础数据类型支持
 - 用途: 时间戳、持续时间等

基础动作接口定义

使用 --- 区分 目标定义 和 结果定义

action/NavigateToGoal.action:

```
# @file action_interfaces/action/NavigateToGoal.action
# @brief ROS2导航动作接口定义
# @description 定义了机器人导航任务的完整动作接口，包括目标参数、结果信息和实时反馈数据

# =====
# 目标定义 (Goal) - 客户端发送给服务器的任务参数
# =====

# 目标位置和姿态
# 包含坐标系信息、3D位置坐标和四元数姿态
geometry_msgs/PoseStamped target_pose

# 最大移动速度限制 (单位: 米/秒)
# 用于控制机器人的移动速度，确保安全性
float64 max_speed

# 是否使用路径规划算法
# true: 使用高级路径规划避障算法
# false: 使用简单的直线导航
bool use_path_planning

# 路径规划算法名称
# 可选值: "A_star", "RRT", "Dijkstra", "direct"
# 当use_path_planning为true时生效
string planning_algorithm

---

# =====
# 结果定义 (Result) - 服务器返回给客户端的最终结果
# =====

# 导航任务是否成功完成
# true: 成功到达目标位置
# false: 任务失败或被取消
bool success

# 详细的结果描述消息
# 例如: "导航成功完成", "目标不可达", "导航被用户取消"
string result_message

# 最终到目标位置的距离误差 (单位: 米)
# 用于评估导航精度，越小越精确
float64 final_distance_to_goal

# 导航任务总耗时
```

```

# 包含秒和纳秒两个字段的時間duration
builtin_interfaces/Duration total_time

# 机器人的最终位置和姿态
# 任务结束时机器人的实际位置
geometry_msgs/PoseStamped final_pose

---

# =====
# 反馈定义 (Feedback) - 服务器定期发送给客户端的进度信息
# =====

# 机器人当前位置和姿态
# 实时更新的机器人位置信息
geometry_msgs/PoseStamped current_pose

# 到目标位置的剩余距离 (单位: 米)
# 动态计算的直线距离, 用于显示进度
float64 distance_remaining

# 预计到达目标的剩余时间 (单位: 秒)
# 基于当前速度和剩余距离的估算
float64 estimated_time_remaining

# 机器人当前移动速度 (单位: 米/秒)
# 实时监测的实际移动速度
float64 current_speed

# 任务完成百分比 (0-100)
# 基于已完成距离与总距离的比值计算
float64 completion_percentage

# 当前状态描述消息
# 例如: "导航中...", "避障中...", "重新规划路径..."
string status_message

```

复杂动作接口定义

action/PickAndPlace.action:

```

# @file action_interfaces/action/PickAndPlace.action
# @brief ROS2机械臂抓取放置动作接口定义
# @description 定义了机械臂执行抓取和放置操作的完整动作接口, 包括目标参数、执行结果和实时反馈

# =====
# 目标定义 (Goal) - 客户端发送给服务器的抓取任务参数
# =====

# 抓取位置和姿态
# 包含目标物体的空间位置坐标和抓取姿态 (四元数表示)
geometry_msgs/PoseStamped pick_pose

```

```
# 放置位置和姿态
# 机械臂完成抓取后将物体放置到的目标位置和姿态
geometry_msgs/PoseStamped place_pose

# 目标物体标识符
# 用于识别和跟踪特定物体的唯一ID字符串
string object_id

# 夹爪抓取力度（单位：牛顿）
# 控制夹爪的抓取力度，避免损坏脆弱物体或抓取力不足
float64 gripper_force

# 接近距离（单位：米）
# 机械臂在实际抓取前的预接近距离，用于精确定位
float64 approach_distance

# 是否使用视觉引导
# true：启用视觉系统进行实时位置修正
# false：仅使用预设的位置坐标
bool use_vision_guidance

---

# =====
# 结果定义 (Result) - 服务器返回给客户端的最终执行结果
# =====

# 抓取放置任务是否成功完成
# true：完整完成抓取和放置操作
# false：任务失败或被中断
bool success

# 详细的结果描述消息
# 例如："抓取放置成功完成", "物体检测失败", "抓取力度不足"
string result_message

# 失败原因详细说明
# 当success为false时，提供具体的失败原因分析
string failure_reason

# 实际使用的夹爪力度（单位：牛顿）
# 机械臂实际执行时的夹爪力度反馈值
float64 actual_gripper_force

# 实际抓取位置和姿态
# 机械臂实际执行抓取时的末端执行器位置（可能与目标位置有偏差）
geometry_msgs/PoseStamped actual_pick_pose

# 实际放置位置和姿态
# 机械臂实际放置物体时的末端执行器位置
geometry_msgs/PoseStamped actual_place_pose
```

```

# 整个任务的执行时间
# 从开始接近物体到完成放置的总耗时
builtin_interfaces/Duration execution_time

---

# =====
# 反馈定义 (Feedback) - 服务器定期发送给客户端的执行进度信息
# =====

# 当前执行阶段标识符
# 可能的值: "approaching"(接近中), "grasping"(抓取中), "lifting"(提升中),
#          "moving"(移动中), "placing"(放置中)
string current_phase

# 当前阶段完成百分比 (0-100)
# 表示当前执行阶段的完成程度
float64 phase_completion

# 机械臂末端执行器当前位置和姿态
# 实时更新的机械臂末端坐标和姿态信息
geometry_msgs/PoseStamped current_end_effector_pose

# 夹爪当前开合位置 (单位: 米或度)
# 夹爪的实时开合状态, 用于监控抓取过程
float64 current_gripper_position

# 是否检测到目标物体
# true: 传感器成功检测到目标物体
# false: 未检测到物体或物体丢失
bool object_detected

# 夹爪力度反馈 (单位: 牛顿)
# 夹爪传感器实时反馈的抓取力度值
float64 grip_force_feedback

# 当前状态描述消息
# 例如: "正在接近物体...", "调整抓取姿态...", "物体抓取成功..."
string status_message

```

action/ChargeBattery.action:

```

# @file action_interfaces/action/ChargeBattery.action
# @brief ROS2机器人电池充电动作接口定义
# @description 定义了机器人自主充电任务的完整动作接口, 包括充电目标、执行结果和实时状态反馈

# =====
# 目标定义 (Goal) - 客户端发送给服务器的充电任务参数
# =====

```

```

# 目标充电百分比 (0-100)
# 设定机器人电池需要充电到的目标电量百分比
float64 target_charge_percentage

# 最大充电时间限制 (单位: 分钟)
# 防止充电时间过长, 设置充电任务的超时时间
float64 max_charging_time

# 是否需要返回充电底座
# true: 机器人需要导航到充电底座进行充电
# false: 机器人已在充电位置或使用便携充电器
bool return_to_dock

# 充电底座位置和姿态
# 充电底座的空间坐标和机器人对接时的目标姿态
geometry_msgs/PoseStamped charging_dock_pose

---

# =====
# 结果定义 (Result) - 服务器返回给客户端的充电任务结果
# =====

# 充电任务是否成功完成
# true: 成功完成充电或达到目标电量
# false: 充电失败、超时或被中断
bool success

# 详细的结果描述消息
# 例如: "充电完成", "充电超时", "充电底座连接失败", "电池故障"
string result_message

# 最终充电百分比 (0-100)
# 充电任务结束时的实际电池电量百分比
float64 final_charge_percentage

# 实际充电时间 (单位: 分钟)
# 从开始充电到任务结束的实际耗时
float64 charging_time

# 充电底座连接是否成功
# true: 成功连接到充电底座并建立充电连接
# false: 连接失败或未使用充电底座
bool dock_connection_success

---

# =====
# 反馈定义 (Feedback) - 服务器定期发送给客户端的充电进度信息
# =====

# 当前电池电量百分比 (0-100)

```



```

# 实时更新的电池电量状态
float64 current_charge_percentage

# 充电速率（单位：%/分钟）
# 当前的充电速度，表示每分钟电量增加的百分比
float64 charging_rate

# 预计剩余充电时间（单位：分钟）
# 基于当前充电速率和目标电量计算的剩余时间
float64 estimated_time_remaining

# 当前充电状态标识符
# 可能的值："approaching_dock"(接近充电底座)，"docking"(对接中)，
#           "charging"(充电中)，"undocking"(脱离充电底座)
string charging_status

# 电池温度（单位：摄氏度）
# 充电过程中的电池温度监控，用于安全保护
float64 temperature

# 是否已连接到充电器
# true：充电器连接正常，正在进行充电
# false：充电器未连接或连接异常
bool is_connected_to_charger

```

编译配置

CMakeLists.txt（完整的见源码）：

```

.....

# 查找依赖包
find_package(ament_cmake REQUIRED)
find_package(rosidl_default_generators REQUIRED)
find_package(geometry_msgs REQUIRED)
find_package(builtin_interfaces REQUIRED)
find_package(std_msgs REQUIRED)

# 生成动作接口
rosidl_generate_interfaces(${PROJECT_NAME}
  "action/NavigateToGoal.action"
  "action/PickAndPlace.action"
  "action/ChargeBattery.action"
  DEPENDENCIES
    geometry_msgs
    builtin_interfaces
    std_msgs
)
.....

```

src/action_demo/package.xml

```

<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>action_interfaces</name>
  <version>0.0.0</version>
  <description>自定义ROS2接口包</description>
  <maintainer email="you@example.com">Your Name</maintainer>
  <license>Apache License 2.0</license>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <!-- 接口生成依赖 -->
  <build_depend>rosidl_default_generators</build_depend>
  <exec_depend>rosidl_default_runtime</exec_depend>

  <!-- 接口依赖 -->
  <depend>geometry_msgs</depend>
  <depend>builtin_interfaces</depend>

  <member_of_group>rosidl_interface_packages</member_of_group>

  <test_depend>ament_lint_auto</test_depend>
  <test_depend>ament_lint_common</test_depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>

```

编译和验证接口

```

# 编译接口包
cd ~/ros2_ws
colcon build --packages-select action_interfaces

# 设置环境
source install/setup.bash

# 验证接口生成
ros2 interface list | grep action_interfaces

```

执行后提示：

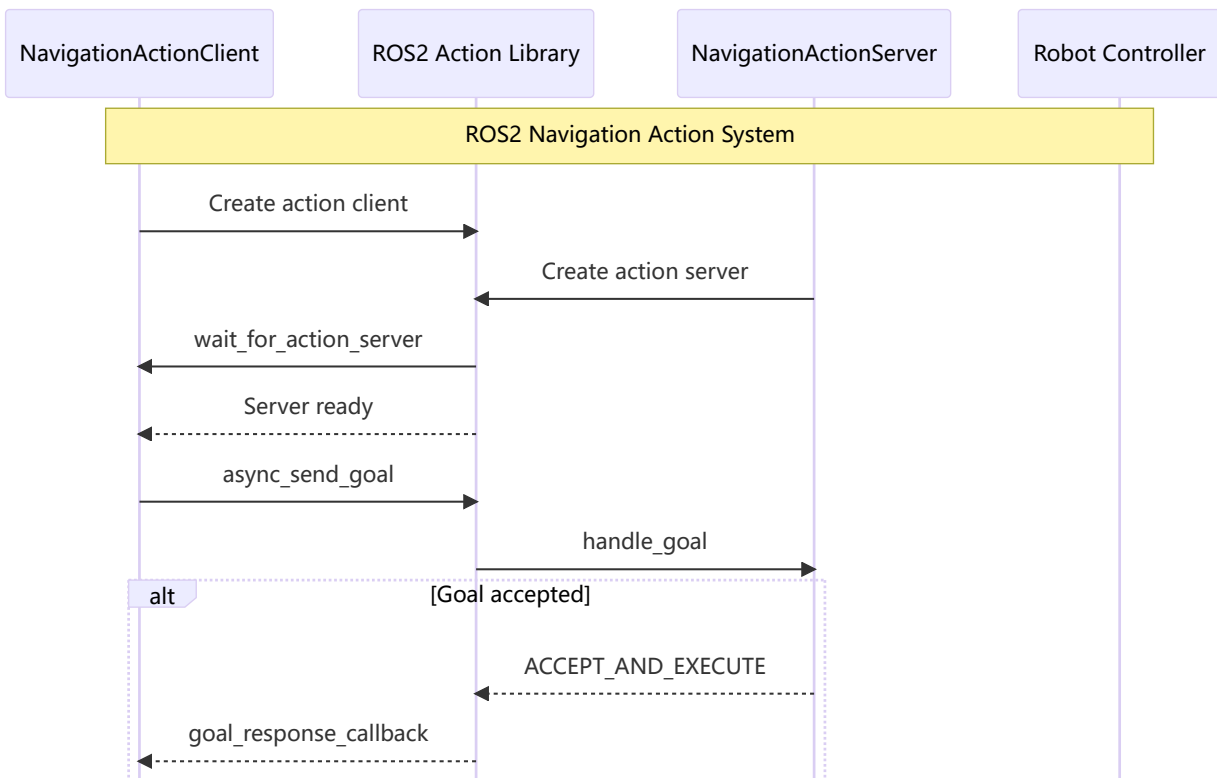
```
lqf@lqf-ros2:~/ros2_ws$ ros2 interface list | grep action_interfaces
action_interfaces/action/ChargeBattery
action_interfaces/action/NavigateToGoal
action_interfaces/action/PickAndPlace
```

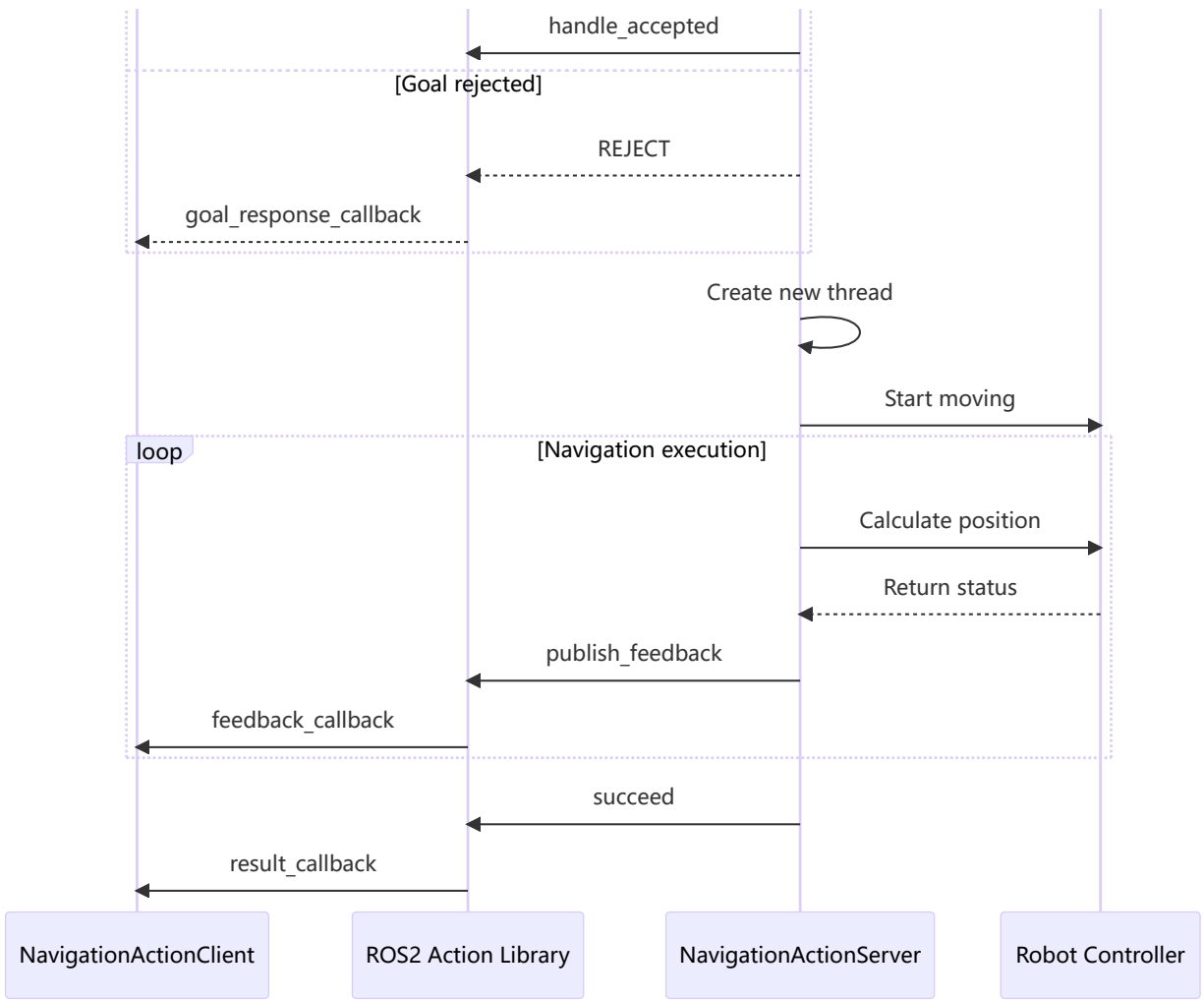
```
lqf@lqf-ros2:~/ros2_ws$ ros2 interface list | grep action_interfaces
action_interfaces/action/ChargeBattery
action_interfaces/action/NavigateToGoal
action_interfaces/action/PickAndPlace
```

后续需要使用：action_interfaces/action/NavigateToGoal。

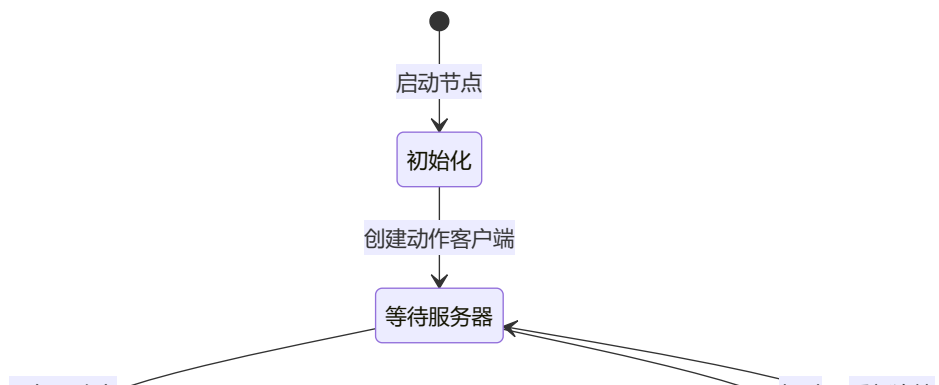
4.4.4 C++编程方法（使用NavigateToGoal）

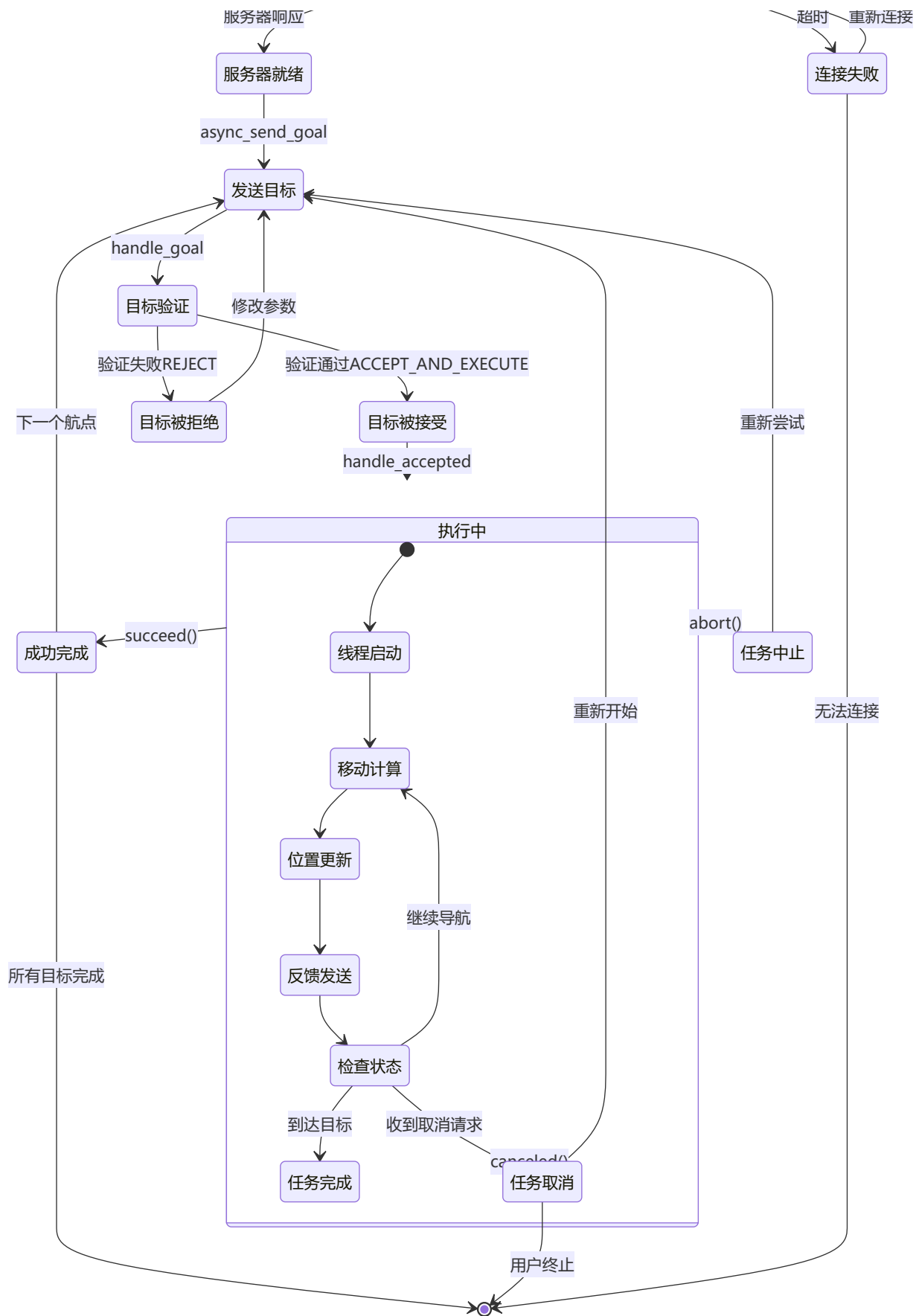
交互逻辑





动作生命周期





创建动作演示功能包

```
# 回到src目录
cd ~/ros2_ws/src

# 创建action_demo功能包
ros2 pkg create --build-type ament_cmake action_demo \
  --dependencies rclcpp rclcpp_action action_interfaces geometry_msgs tf2
tf2_geometry_msgs
```

命令解析：

- action_demo
 - 作用: 新创建的ROS2包名称
 - 生成: 会创建名为action_demo的目录
- --dependencies 依赖:
 - rclcpp_action
 - 用途: ROS2 C++动作系统库
 - 提供: 动作客户端和服务器的实现
 - 功能: 处理长时间运行的任务, 支持取消和进度反馈
 - action_interfaces (**即是我们自己创建的动作接口**)
 - 用途: 自定义动作接口包
 - 包含: 项目特定的.action文件定义
 - 作用: 提供NavigateToGoal、PickAndPlace等动作接口
 - geometry_msgs
 - 用途: 几何消息类型标准库
 - 提供: PoseStamped、Twist、Point、Quaternion等
 - 用途: 处理位置、姿态、速度等几何数据
 - tf2
 - 用途: 坐标变换库核心
 - 功能: 坐标系变换计算和管理
 - 作用: 处理机器人各部件间的空间关系
 - tf2_geometry_msgs
 - 用途: TF2与geometry_msgs的集成
 - 功能: 为geometry_msgs提供坐标变换功能
 - 例如: 将PoseStamped在不同坐标系间转换

动作服务端编程方法

action_demo/src/navigation_action_server.cpp:

```
/**
 * @file action_demo/src/navigation_action_server.cpp
 * @brief ROS2导航动作服务器示例节点
 * @description 演示如何创建动作服务器，实现长时间运行的导航任务，支持目标处理、取消、进度反馈和结果返回
 */

// 包含ROS2 C++客户端库的头文件
#include <rclcpp/rclcpp.hpp>
// 包含ROS2动作系统的头文件
#include <rclcpp_action/rclcpp_action.hpp>
// 包含自定义导航动作接口的头文件
#include <action_interfaces/action/navigate_to_goal.hpp>
// 包含几何消息类型，用于位置和姿态表示
#include <geometry_msgs/msg/pose_stamped.hpp>
// 包含TF2四元数类，用于姿态计算
#include <tf2/LinearMath/Quaternion.h>
// 包含TF2几何消息转换工具
#include <tf2_geometry_msgs/tf2_geometry_msgs.hpp>
// 包含数学计算函数
#include <cmath>
// 包含多线程支持
#include <thread>

// 使用类型别名简化代码
using NavigateToGoal = action_interfaces::action::NavigateToGoal;
using GoalHandleNavigate = rclcpp_action::ServerGoalHandle<NavigateToGoal>;

/**
 * @class NavigationActionServer
 * @brief 导航动作服务器节点类
 * @description 继承自rclcpp::Node，实现了导航动作服务器，处理导航目标请求，执行路径规划和移动控制
 */
class NavigationActionServer : public rclcpp::Node
{
public:
    /**
     * @brief 构造函数
     * @description 初始化节点，设置节点名称为"navigation_action_server"，创建动作服务器，初始化机器人位置
     */
    NavigationActionServer() : Node("navigation_action_server")
    {
        // 创建动作服务器对象
        // 模板参数指定动作类型为NavigateToGoal
    }
};
```

```

// "navigate_to_goal"是动作名称，客户端通过此名称调用动作
// 绑定三个回调函数：目标处理、取消处理、接受处理
action_server_ = rclcpp_action::create_server<NavigateToGoal>(
    this,
    "navigate_to_goal",
    // 处理目标请求回调函数
    std::bind(&NavigationActionServer::handle_goal, this,
std::placeholders::_1, std::placeholders::_2),
    // 处理取消请求回调函数
    std::bind(&NavigationActionServer::handle_cancel, this,
std::placeholders::_1),
    // 处理被接受的目标回调函数
    std::bind(&NavigationActionServer::handle_accepted, this,
std::placeholders::_1)
);

// 初始化模拟的当前机器人位置
// 设置坐标系为"map"，这是ROS导航中的标准全局坐标系
current_pose_.header.frame_id = "map";
current_pose_.pose.position.x = 0.0;    // 初始X坐标
current_pose_.pose.position.y = 0.0;    // 初始Y坐标
current_pose_.pose.position.z = 0.0;    // 初始Z坐标（地面机器人通常为0）
current_pose_.pose.orientation.w = 1.0; // 四元数w分量，表示无旋转

// 输出服务器启动信息到控制台
RCLCPP_INFO(this->get_logger(), "导航动作服务端已启动");
}

```

private:

```

// 动作服务器对象的智能指针，管理导航动作的生命周期
rclcpp_action::Server<NavigateToGoal>::SharedPtr action_server_;

// 当前机器人位置，包含位置和姿态信息
geometry_msgs::msg::PoseStamped current_pose_;

/**
 * @brief 处理目标请求回调函数
 * @param uuid 目标的唯一标识符
 * @param goal 客户端发送的导航目标，包含目标位置和最大速度
 * @return GoalResponse 表示是否接受或拒绝该目标
 * @description 当客户端发送导航目标时，此函数会被调用来验证目标的有效性
 */
rclcpp_action::GoalResponse handle_goal(
    const rclcpp_action::GoalUUID & uuid,
    std::shared_ptr<const NavigateToGoal::Goal> goal)
{
    // 避免未使用变量警告
    (void)uuid;

    // 输出接收到的目标信息
    RCLCPP_INFO(this->get_logger(),
        "收到导航目标：(%.2f, %.2f)，最大速度：%.2f m/s",

```



```

        goal->target_pose.pose.position.x,
        goal->target_pose.pose.position.y,
        goal->max_speed);

// 验证最大速度的有效性
// 速度必须为正数且不超过安全限制
if (goal->max_speed <= 0 || goal->max_speed > 5.0) {
    RCLCPP_WARN(this->get_logger(), "无效的最大速度: %.2f", goal->max_speed);
    return rclcpp_action::GoalResponse::REJECT;
}

// 验证目标距离是否在合理范围内
double distance = calculate_distance(current_pose_, goal->target_pose);
if (distance > 100.0) { // 距离太远
    RCLCPP_WARN(this->get_logger(), "目标距离太远: %.2f 米", distance);
    return rclcpp_action::GoalResponse::REJECT;
}

// 目标验证通过, 接受并开始执行
RCLCPP_INFO(this->get_logger(), "目标已接受, 开始导航");
return rclcpp_action::GoalResponse::ACCEPT_AND_EXECUTE; // -> 触发
handle_accepted的调用
}

/**
 * @brief 处理取消请求回调函数
 * @param goal_handle 要取消的目标句柄
 * @return CancelResponse 表示是否接受取消请求
 * @description 当客户端请求取消正在进行的导航任务时, 此函数会被调用
 */
rclcpp_action::CancelResponse handle_cancel(
    const std::shared_ptr<GoalHandleNavigate> goal_handle)
{
    // 避免未使用变量警告
    (void)goal_handle;

    // 输出取消请求信息
    RCLCPP_INFO(this->get_logger(), "收到取消请求");

    // 接受取消请求 (在实际应用中可能需要更复杂的逻辑)
    return rclcpp_action::CancelResponse::ACCEPT;
}

/**
 * @brief 处理被接受的目标回调函数
 * @param goal_handle 被接受的目标句柄
 * @description 当目标被接受后, 此函数会被调用来开始执行导航任务
 */
void handle_accepted(const std::shared_ptr<GoalHandleNavigate> goal_handle)
{
    RCLCPP_INFO(this->get_logger(), "收到被接受的目标");
    // 在新线程中执行导航任务, 避免阻塞主线程

```

```

        // 使用detach()让线程独立运行
        std::thread(std::bind(&NavigationActionServer::execute_navigation, this,
goal_handle)).detach();
    }

    /**
     * @brief 执行导航任务函数
     * @param goal_handle 目标句柄，用于发布反馈和设置结果
     * @description 在独立线程中运行的导航执行逻辑，包含路径规划、移动控制、进度反馈等
     */
    void execute_navigation(const std::shared_ptr<GoalHandleNavigate> goal_handle)
    {
        RCLCPP_INFO(this->get_logger(), "开始执行导航任务");

        // 获取目标信息
        const auto goal = goal_handle->get_goal();

        // 创建反馈和结果对象
        auto feedback = std::make_shared<NavigateToGoal::Feedback>();
        auto result = std::make_shared<NavigateToGoal::Result>();

        // 记录任务开始时间
        auto start_time = this->now();

        // 计算到目标的总距离
        double total_distance = calculate_distance(current_pose_, goal-
>target_pose);

        // 设置控制循环参数
        const double update_rate = 10.0; // 10Hz更新频率
        const double time_step = 1.0 / update_rate; // 时间步长
        rclcpp::Rate rate(update_rate);

        // 主要导航执行循环
        while (rclcpp::ok()) {
            // 检查是否收到取消请求
            if (goal_handle->is_canceling()) {
                // 设置取消结果
                result->success = false;
                result->result_message = "导航被用户取消";
                result->final_distance_to_goal = calculate_distance(current_pose_,
goal->target_pose);
                result->total_time = this->now() - start_time;
                result->final_pose = current_pose_;

                // 标记目标为已取消
                goal_handle->canceled(result);
                RCLCPP_INFO(this->get_logger(), "导航任务已取消");
                return;
            }

            // 模拟机器人向目标移动

```

```

        move_towards_goal(goal->target_pose, goal->max_speed, time_step);

        // 计算到目标的剩余距离
        double remaining_distance = calculate_distance(current_pose_, goal->target_pose);

        // 更新反馈信息
        feedback->current_pose = current_pose_; // 当前位置
        feedback->distance_remaining = remaining_distance; // 剩余距离
        feedback->completion_percentage = ((total_distance - remaining_distance) / total_distance) * 100.0; // 完成百分比
        feedback->current_speed = goal->max_speed; // 当前速度
        feedback->estimated_time_remaining = remaining_distance / goal->max_speed; // 预计剩余时间
        feedback->status_message = "导航中..."; // 状态消息

        // 向客户端发布反馈信息
        goal_handle->publish_feedback(feedback);

        // 输出调试信息 (DEBUG级别, 正常情况下不显示)
        RCLCPP_DEBUG(this->get_logger(),
            "当前位置: (%.2f, %.2f), 剩余距离: %.2f 米, 完成度: %.1f%%",
            current_pose_.pose.position.x,
            current_pose_.pose.position.y,
            remaining_distance,
            feedback->completion_percentage);

        // 检查是否到达目标 (10cm精度)
        if (remaining_distance < 0.1) {
            // 设置成功结果
            result->success = true;
            result->result_message = "导航成功完成";
            result->final_distance_to_goal = remaining_distance;
            result->total_time = this->now() - start_time;
            result->final_pose = current_pose_;

            // 标记目标为成功完成
            goal_handle->succeed(result);
            RCLCPP_INFO(this->get_logger(),
                "导航成功! 最终误差: %.3f 米, 总耗时: %.2f 秒",
                remaining_distance,
                result->total_time.sec + result->total_time.nanosec * 1e-9);

            return;
        }

        // 等待下一个控制周期
        rate.sleep();
    }
}

/**

```

```

* @brief 计算两点间距离函数
* @param pose1 第一个位置点
* @param pose2 第二个位置点
* @return double 两点间的欧几里得距离（米）
* @description 计算二维平面上两个位置之间的直线距离
*/
double calculate_distance(
    const geometry_msgs::msg::PoseStamped& pose1,
    const geometry_msgs::msg::PoseStamped& pose2)
{
    // 计算X和Y方向的距离差
    double dx = pose2.pose.position.x - pose1.pose.position.x;
    double dy = pose2.pose.position.y - pose1.pose.position.y;

    // 返回欧几里得距离
    return std::sqrt(dx * dx + dy * dy);
}

/**
* @brief 模拟机器人向目标移动函数
* @param target 目标位置
* @param max_speed 最大移动速度（米/秒）
* @param time_step 时间步长（秒）
* @description 根据当前位置和目标位置，计算并更新机器人的新位置和朝向
*/
void move_towards_goal(
    const geometry_msgs::msg::PoseStamped& target,
    double max_speed,
    double time_step)
{
    // 计算到目标的方向向量
    double dx = target.pose.position.x - current_pose_.pose.position.x;
    double dy = target.pose.position.y - current_pose_.pose.position.y;
    double distance = std::sqrt(dx * dx + dy * dy);

    // 如果还有距离需要移动
    if (distance > 0) {
        // 计算这个时间步内应该移动的距离
        // 取速度限制和剩余距离的较小值
        double move_distance = std::min(max_speed * time_step, distance);
        double ratio = move_distance / distance;

        // 更新机器人位置
        current_pose_.pose.position.x += dx * ratio;
        current_pose_.pose.position.y += dy * ratio;
        current_pose_.header.stamp = this->now();

        // 更新机器人朝向（面向目标方向）
        double yaw = std::atan2(dy, dx); // 计算偏航角
        tf2::Quaternion q;
        q.setRPY(0, 0, yaw); // 设置Roll-Pitch-Yaw角度
        current_pose_.pose.orientation = tf2::toMsg(q); // 转换为消息格式
    }
}

```

```

    }
}
};

/**
 * @brief 主函数
 * @description 程序入口点，负责初始化ROS2环境、创建和运行导航动作服务器节点
 */
int main(int argc, char ** argv)
{
    // 初始化ROS2通信中间件
    rclcpp::init(argc, argv);

    // 创建NavigationActionServer节点
    auto node = std::make_shared<NavigationActionServer>();

    // 进入ROS2事件循环，等待动作请求
    // spin会持续运行直到节点被关闭
    rclcpp::spin(node);

    // 清理ROS2资源，关闭通信中间件
    rclcpp::shutdown();

    // 程序正常退出
    return 0;
}

```

动作客户端编程方法

action_demo/src/navigation_action_client.cpp:

```

/**
 * @file action_demo/src/navigation_action_client.cpp
 * @brief ROS2导航动作客户端实现
 * @description 实现了一个完整的导航动作客户端，能够发送多个导航目标、
 *             接收实时反馈、处理结果回调，并支持目标取消功能
 */

#include <rclcpp/rclcpp.hpp> // ROS2 C++核心库
#include <rclcpp_action/rclcpp_action.hpp> // ROS2动作系统库
#include <action_interfaces/action/navigate_to_goal.hpp> // 自定义导航动作接口
#include <geometry_msgs/msg/pose_stamped.hpp> // 几何消息：带时间戳的位
置姿态
#include <chrono> // C++时间库

// 类型别名定义，简化代码书写
using NavigateToGoal = action_interfaces::action::NavigateToGoal; // 导航动
作类型
using GoalHandleNavigate = rclcpp_action::ClientGoalHandle<NavigateToGoal>; // 目标句
柄类型
using namespace std::chrono_literals; // 时间字
面量命名空间

```

```

/**
 * @class NavigationActionClient
 * @brief 导航动作客户端类
 * @description 继承自rclcpp::Node，实现了完整的导航动作客户端功能，
 *              包括目标发送、反馈接收、结果处理和目标取消
 */
class NavigationActionClient : public rclcpp::Node
{
public:
    /**
     * @brief 构造函数
     * @description 初始化动作客户端，等待服务可用，然后开始发送导航目标
     */
    NavigationActionClient() : Node("navigation_action_client")
    {
        // 创建动作客户端，连接到名为"navigate_to_goal"的动作服务
        action_client_ = rclcpp_action::create_client<NavigateToGoal>(
            this, "navigate_to_goal");

        // 等待动作服务可用，超时时间10秒
        // 这是一个阻塞调用，确保服务器已启动
        if (!action_client_>wait_for_action_server(10s)) {
            RCLCPP_ERROR(this->get_logger(), "动作服务不可用");
            return;
        }

        RCLCPP_INFO(this->get_logger(), "导航动作客户端已启动");

        // 开始测试导航功能
        send_navigation_goals();
    }

private:
    // 动作客户端智能指针，用于与动作服务器通信
    rclcpp_action::Client<NavigateToGoal>::SharedPtr action_client_;

    /**
     * @brief 发送多个导航目标进行测试
     * @description 定义一系列航点，按顺序发送给导航服务器，
     *              模拟机器人按预定路径移动的场景
     */
    void send_navigation_goals()
    {
        // 定义测试航点列表：(x, y)坐标对
        // 这些点构成一个简单的导航路径
        std::vector<std::pair<double, double>> waypoints = {
            {3.0, 2.0}, // 第一个目标点：向右前方移动
            {5.0, 5.0}, // 第二个目标点：继续向右上移动
            {1.0, 4.0}, // 第三个目标点：向左移动
            {0.0, 0.0}  // 第四个目标点：回到起点
        };
    }
};

```

```

// 遍历所有航点，依次发送导航目标
for (const auto& waypoint : waypoints) {
    send_goal(waypoint.first, waypoint.second, 1.5);

    // 等待当前目标完成再发送下一个
    // 这里简单使用睡眠，实际应用中应该等待结果回调
    std::this_thread::sleep_for(2s);
}
}

/**
 * @brief 发送单个导航目标
 * @param x 目标位置X坐标（米）
 * @param y 目标位置Y坐标（米）
 * @param max_speed 最大移动速度（米/秒）
 * @description 构造导航目标消息，设置各种参数，并通过异步方式发送给服务器
 */
void send_goal(double x, double y, double max_speed)
{
    // 创建导航目标消息
    auto goal_msg = NavigateToGoal::Goal();

    // 设置目标位置信息
    goal_msg.target_pose.header.stamp = this->now(); // 设置时间戳为当前时
                                                    // 间
    goal_msg.target_pose.header.frame_id = "map"; // 设置坐标系为地图坐
                                                    // 标系
    goal_msg.target_pose.pose.position.x = x; // 目标X坐标
    goal_msg.target_pose.pose.position.y = y; // 目标Y坐标
    goal_msg.target_pose.pose.position.z = 0.0; // Z坐标（2D导航设为
0)
    goal_msg.target_pose.pose.orientation.w = 1.0; // 四元数w分量（无旋
转)

    // 设置导航参数
    goal_msg.max_speed = max_speed; // 最大移动速度
    goal_msg.use_path_planning = true; // 启用路径规划
    goal_msg.planning_algorithm = "A_star"; // 使用A*算法

    RCLCPP_INFO(this->get_logger(),
        "发送导航目标：(%.2f, %.2f)，最大速度：%.2f m/s",
        x, y, max_speed);

    // 设置发送目标的选项，包括各种回调函数
    auto send_goal_options =
rclcpp_action::Client<NavigateToGoal>::SendGoalOptions();

    // 目标响应回调：服务器接受或拒绝目标时调用
    send_goal_options.goal_response_callback =
        std::bind(&NavigationActionClient::goal_response_callback, this,
std::placeholders::_1);

```

```

// 反馈回调：服务器定期发送执行进度时调用
send_goal_options.feedback_callback =
    std::bind(&NavigationActionClient::feedback_callback, this,
std::placeholders::_1, std::placeholders::_2);

// 结果回调：目标执行完成（成功/失败/取消）时调用
send_goal_options.result_callback =
    std::bind(&NavigationActionClient::result_callback, this,
std::placeholders::_1);

// 异步发送目标，不阻塞当前线程
auto goal_handle_future = action_client->async_send_goal(goal_msg,
send_goal_options);
}

/**
 * @brief 目标响应回调函数
 * @param goal_handle 目标句柄智能指针
 * @description 当服务器响应目标请求时调用，判断目标是否被接受
 */
void goal_response_callback(const GoalHandleNavigate::SharedPtr & goal_handle)
{
    if (!goal_handle) {
        // 目标被拒绝的情况
        RCLCPP_ERROR(this->get_logger(), "目标被拒绝");
    } else {
        // 目标被接受，服务器开始执行
        RCLCPP_INFO(this->get_logger(), "目标被接受，开始执行");
    }
}

/**
 * @brief 反馈回调函数
 * @param goal_handle 目标句柄（未使用但必需的参数）
 * @param feedback 反馈消息智能指针
 * @description 服务器定期发送执行进度时调用，显示当前位置、剩余距离等信息
 */
void feedback_callback(
    GoalHandleNavigate::SharedPtr,
    const std::shared_ptr<const NavigateToGoal::Feedback> feedback)
{
    // 显示详细的导航进度信息
    RCLCPP_INFO(this->get_logger(),
        "导航反馈：位置(%.2f, %.2f)，剩余距离：%.2f m，完成度：%.1f%%，状态：
%s",

        feedback->current_pose.pose.position.x, // 当前X坐标
        feedback->current_pose.pose.position.y, // 当前Y坐标
        feedback->distance_remaining, // 到目标的剩余距离
        feedback->completion_percentage, // 任务完成百分比
        feedback->status_message.c_str()); // 状态描述消息
}

```



```

/**
 * @brief 结果回调函数
 * @param result 包装的结果对象
 * @description 当目标执行完成时调用，处理成功、中止、取消等不同结果状态
 */
void result_callback(const GoalHandleNavigate::WrappedResult & result)
{
    // 根据结果代码进行不同处理
    switch(result.code) {
        case rclcpp_action::ResultCode::SUCCEEDED:
            // 导航成功完成
            RCLCPP_INFO(this->get_logger(),
                "✅ 导航成功: %s, 最终误差: %.3f m, 总时间: %.2f s",
                result.result->result_message.c_str(),

            // 结果描述
                result.result->final_distance_to_goal,

            // 最终到目标的距离误差
                result.result->total_time.sec + result.result->
>total_time.nanosec * 1e-9); // 总耗时 (秒)
            break;
        case rclcpp_action::ResultCode::ABORTED:
            // 导航被中止 (通常因为遇到障碍或错误)
            RCLCPP_ERROR(this->get_logger(), "❌ 导航中止: %s", result.result->
>result_message.c_str());
            break;
        case rclcpp_action::ResultCode::CANCELED:
            // 导航被取消 (用户主动取消)
            RCLCPP_WARN(this->get_logger(), "⚠️ 导航取消: %s", result.result->
>result_message.c_str());
            break;
        default:
            // 未知的结果代码
            RCLCPP_ERROR(this->get_logger(), "❓ 未知结果代码");
            break;
    }
}

public:
/**
 * @brief 取消当前目标的公共方法
 * @description 提供给外部调用的接口，用于取消当前正在执行的导航目标
 *
 * 这在紧急情况或用户干预时非常有用
 */
void cancel_current_goal()
{
    RCLCPP_INFO(this->get_logger(), "取消当前导航目标");
    // 异步取消所有正在执行的目标
    auto cancel_future = action_client->async_cancel_all_goals();
}
};

```

```

/**
 * @brief 主函数
 * @param argc 命令行参数个数
 * @param argv 命令行参数数组
 * @return 程序退出状态码
 * @description 初始化ROS2系统，创建导航客户端节点，启动事件循环
 */
int main(int argc, char ** argv)
{
    // 初始化ROS2系统
    rclcpp::init(argc, argv);

    // 创建导航动作客户端节点
    auto node = std::make_shared<NavigationActionClient>();

    // 启动ROS2事件循环，处理回调函数
    rclcpp::spin(node);

    // 关闭ROS2系统
    rclcpp::shutdown();
    return 0;
}

```

编译配置

CMakeLists.txt（部分节选）：

```

# 查找依赖包
...
find_package(rclcpp_action REQUIRED)
find_package(action_interfaces REQUIRED)
find_package(geometry_msgs REQUIRED)
find_package(tf2 REQUIRED)
find_package(tf2_geometry_msgs REQUIRED)

# 创建可执行文件
add_executable(navigation_action_server src/navigation_action_server.cpp)
add_executable(navigation_action_client src/navigation_action_client.cpp)

# 设置依赖
ament_target_dependencies(navigation_action_server
    rclcpp rclcpp_action action_interfaces geometry_msgs tf2 tf2_geometry_msgs)
ament_target_dependencies(navigation_action_client
    rclcpp rclcpp_action action_interfaces geometry_msgs)
...

```

编译和运行

```
# 1. 编译接口和应用包
cd ~/ros2_ws
colcon build --packages-select action_interfaces action_demo

# 2. 设置环境
source install/setup.bash

# 3.1 运行动作服务端（终端1）
ros2 run action_demo navigation_action_server

# 3.2 运行动作客户端（终端2）
ros2 run action_demo navigation_action_client
```

4.4.5 动作的命令行操作

动作命令行操作

```
# 查看可用的动作
ros2 action list

# 查看动作接口定义
ros2 interface show action_interfaces/action/NavigateToGoal

# 发送动作目标
ros2 action send_goal /navigate_to_goal action_interfaces/action/NavigateToGoal \
  '{target_pose: {header: {frame_id: "map"}, pose: {position: {x: 3.0, y: 2.0}}},
  max_speed: 1.5}'

# 带反馈的动作调用
ros2 action send_goal /navigate_to_goal action_interfaces/action/NavigateToGoal \
  '{target_pose: {header: {frame_id: "map"}, pose: {position: {x: 5.0, y: 3.0}}},
  max_speed: 2.0}' \
  --feedback

# 查看动作信息
ros2 action info /navigate_to_goal

# 监听动作反馈
ros2 topic echo /navigate_to_goal/_action/feedback

# 监听动作状态
ros2 topic echo /navigate_to_goal/_action/status

# 使用rqt图形界面
rqt_action # 动作监控器
```

动作调试和监控

```
# 查看动作服务端
ros2 node info /navigation_action_server

# 查看动作相关话题
ros2 topic list | grep navigate_to_goal

# 查看动作相关服务
ros2 service list | grep navigate_to_goal

# 实时监控动作执行
ros2 topic echo /navigate_to_goal/_action/feedback --once
```

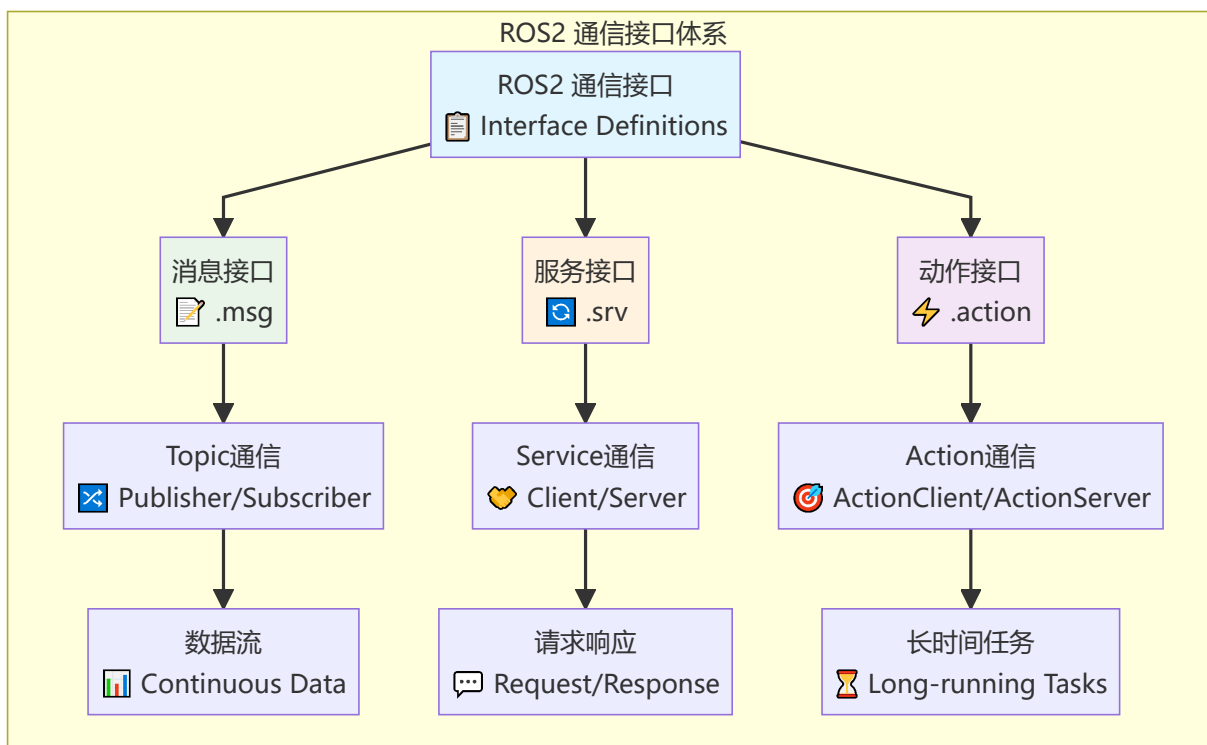
4.5 通信接口

4.5.1 什么是通信接口

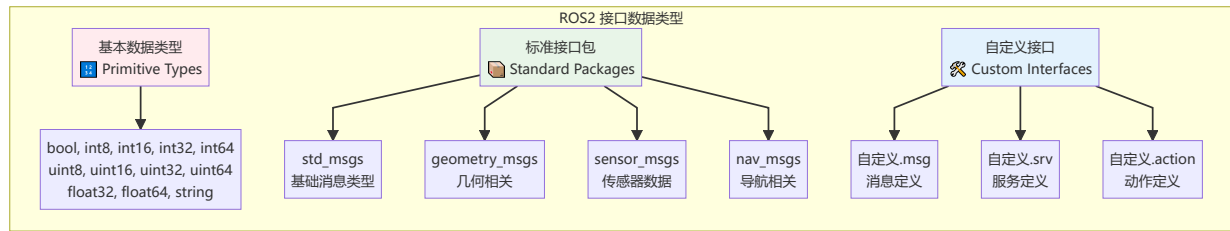
定义和概念

- **通信接口**：定义ROS2节点间数据传输格式的标准结构
- **作用**：确保不同节点间数据交换的一致性和兼容性
- **类型**：消息（.msg）、服务（.srv）、动作（.action）

ROS2通信接口体系结构图



接口数据类型层次结构



4.5.2 通信接口的定义方法

4.5.2.1 创建接口功能包

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_cmake my_interfaces \
  --dependencies rosidl_default_generators rosidl_default_runtime
```

生成的功能包路径：~/ros2_ws/src/my_interfaces

参数解析：

- my_interfaces
 - 包名：自定义接口包名称
 - 命名约定：通常以 **interfaces** 或 **msgs** 结尾
 - 用于存放自定义消息、服务、动作定义
- --dependencies rosidl_default_generators rosidl_default_runtime
 - **rosidl_default_generators** (构建时依赖)
 - rosidl: ROS Interface Definition Language (ROS接口定义语言)
 - **generators**: 代码生成器
 - 功能：将 .msg、.srv、.action 文件转换为 C++/Python 代码
 - 仅在编译时需要
 - rosidl_default_runtime (运行时依赖)
 - runtime: 运行时支持库
 - 功能：提供生成的接口在运行时所需的基础设施
 - 在运行时需要

4.5.2.2 消息接口 (.msg) 定义

基础消息定义

msg/RobotStatus.msg:

```
# 机器人状态消息
# 基本数据类型
bool is_enabled
int32 battery_level
float64 speed
string current_mode

# 时间戳
builtin_interfaces/Time timestamp

# 数组类型
float64[] joint_positions
string[5] sensor_names

# 嵌套消息类型
geometry_msgs/Point position
geometry_msgs/Quaternion orientation
```

复杂消息定义

msg/SensorData.msg:

```
# 传感器数据集合
std_msgs/Header header

# 传感器信息
string sensor_id
string sensor_type
bool is_active

# 测量数据
float64 temperature
float64 humidity
float64 pressure

# 位置信息
geometry_msgs/Point sensor_position

# 数据数组
float64[] raw_data
uint8[] status_flags

# 可选字段（使用常量）
uint8 STATUS_OK = 0
uint8 STATUS_WARNING = 1
uint8 STATUS_ERROR = 2
```

```
uint8 current_status
```

4.5.2.3 服务接口 (.srv) 定义

基础服务定义

srv/SetRobotMode.srv:

```
# 请求部分
string mode_name
bool enable_safety
float64 max_speed
---
# 响应部分
bool success
string message
string previous_mode
float64 actual_speed
```

复杂服务定义

srv/NavigationRequest.srv:

```
# 导航请求服务
# 请求部分
geometry_msgs/PoseStamped start_pose
geometry_msgs/PoseStamped goal_pose
string planning_algorithm
float64 max_planning_time
bool use_obstacles
string[] obstacle_types
---
# 响应部分
bool success
string result_message
nav_msgs/Path planned_path
float64 path_length
float64 estimated_time
geometry_msgs/PoseStamped[] waypoints
```

文件操作服务

srv/FileOperation.srv:

```

# 文件操作服务
# 请求部分
string operation_type    # "read", "write", "delete"
string file_path
string file_content      # 用于写操作
---
# 响应部分
bool success
string message
string file_content      # 用于读操作
int64 file_size
builtin_interfaces/Time last_modified

```

4.5.2.4 动作接口 (.action) 定义

基础动作定义

action/NavigateToGoal.action:

```

# 目标定义
geometry_msgs/PoseStamped target_pose
float64 max_speed
bool use_path_planning
---
# 结果定义
bool success
string result_message
float64 final_distance_to_goal
builtin_interfaces/Duration total_time
---
# 反馈定义
geometry_msgs/PoseStamped current_pose
float64 distance_remaining
float64 estimated_time_remaining
float64 current_speed

```

复杂动作定义

action/ProcessData.action:

```

# 数据处理动作
# 目标定义
string[] input_files
string processing_type
int32 batch_size
bool save_intermediate_results
---
# 结果定义
bool success
string result_message
string[] output_files

```



```

int32 processed_items
float64 processing_time
string[] error_logs
---
# 反馈定义
int32 current_item
int32 total_items
float64 completion_percentage
string current_file
builtin_interfaces/Duration elapsed_time
string status_message

```

编译配置

CMakeLists.txt:

```

cmake_minimum_required(VERSION 3.8)
project(my_interfaces)

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "clang")
    add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# 查找依赖包
find_package(ament_cmake REQUIRED)
find_package(rosidl_default_generators REQUIRED)
find_package(std_msgs REQUIRED)
find_package(geometry_msgs REQUIRED)
find_package(sensor_msgs REQUIRED)
find_package(nav_msgs REQUIRED)
find_package(builtin_interfaces REQUIRED)

# 生成接口
rosidl_generate_interfaces(${PROJECT_NAME}
    # 消息文件
    "msg/RobotStatus.msg"
    "msg/SensorData.msg"

    # 服务文件
    "srv/SetRobotMode.srv"
    "srv/NavigationRequest.srv"
    "srv/FileOperation.srv"

    # 动作文件
    "action/NavigateToGoal.action"
    "action/ProcessData.action"

    # 依赖的接口包
    DEPENDENCIES
        std_msgs
        geometry_msgs
        sensor_msgs

```

```

    nav_msgs
    builtin_interfaces
)

ament_export_dependencies(rosidl_default_runtime)
ament_package()

```

package.xml:

```

<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>my_interfaces</name>
  <version>0.0.0</version>
  <description>自定义ROS2接口包</description>
  <maintainer email="you@example.com">Your Name</maintainer>
  <license>Apache License 2.0</license>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <!-- 接口生成依赖 -->
  <build_depend>rosidl_default_generators</build_depend>
  <exec_depend>rosidl_default_runtime</exec_depend>

  <!-- 接口依赖 -->
  <depend>std_msgs</depend>
  <depend>geometry_msgs</depend>
  <depend>sensor_msgs</depend>
  <depend>nav_msgs</depend>
  <depend>builtin_interfaces</depend>

  <member_of_group>rosidl_interface_packages</member_of_group>

  <test_depend>ament_lint_auto</test_depend>
  <test_depend>ament_lint_common</test_depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>

```

4.5.3 通信接口的命令行操作

编译和验证接口

```
# 编译接口包
cd ~/ros2_ws
colcon build --packages-select my_interfaces

# 设置环境
source install/setup.bash

# 验证接口生成
ros2 interface list | grep my_interfaces
```

查看接口信息

```
# 查看所有接口
ros2 interface list

# 查看特定包的接口
ros2 interface package my_interfaces

# 查看消息接口详情
ros2 interface show my_interfaces/msg/RobotStatus
ros2 interface show my_interfaces/msg/SensorData

# 查看服务接口详情
ros2 interface show my_interfaces/srv/SetRobotMode
ros2 interface show my_interfaces/srv/NavigationRequest

# 查看动作接口详情
ros2 interface show my_interfaces/action/NavigateToGoal
ros2 interface show my_interfaces/action/ProcessData
```

接口类型查询

```
# 查看标准消息类型
ros2 interface show std_msgs/msg/String
ros2 interface show geometry_msgs/msg/Twist
ros2 interface show sensor_msgs/msg/LaserScan

# 查看标准服务类型
ros2 interface show std_srvs/srv/SetBool
ros2 interface show std_srvs/srv/Trigger

# 搜索包含特定关键字的接口
ros2 interface list | grep -i navigation
ros2 interface list | grep -i sensor
```

使用接口进行通信测试

```
source install/setup.bash
# 使用自定义消息发布话题
ros2 topic pub /robot_status my_interfaces/msg/RobotStatus \
    '{is_enabled: true, battery_level: 85, speed: 1.5, current_mode: "navigation"}'

# 调用自定义服务
ros2 service call /set_robot_mode my_interfaces/srv/SetRobotMode \
    '{mode_name: "autonomous", enable_safety: true, max_speed: 2.0}'

# 使用rqt图形界面
rqt_msg      # 消息发布者
rqt_srv      # 服务调用器
```

如果没有相关工具：

```
sudo apt update
```

4.5.4 C++服务接口应用示例

创建使用自定义接口的功能包

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_cmake interface_demo \
    --dependencies rclcpp my_interfaces
```

服务端实现示例

src/robot_mode_server.cpp:

```
#include <rclcpp/rclcpp.hpp>
#include <my_interfaces/srv/set_robot_mode.hpp>
#include <memory>
#include <string>

class RobotModeServer : public rclcpp::Node
{
public:
    RobotModeServer() : Node("robot_mode_server"), current_mode_("manual"),
max_speed_(1.0)
    {
        // 创建服务端
        service_ = this->create_service<my_interfaces::srv::SetRobotMode>(
            "set_robot_mode",
```

```

        std::bind(&RobotModeServer::handle_set_mode, this,
                  std::placeholders::_1, std::placeholders::_2));

RCLCPP_INFO(this->get_logger(), "机器人模式设置服务已启动");
RCLCPP_INFO(this->get_logger(), "当前模式: %s, 最大速度: %.2f m/s",
            current_mode_.c_str(), max_speed_);
}

private:
void handle_set_mode(
    const std::shared_ptr<my_interfaces::srv::SetRobotMode::Request> request,
    std::shared_ptr<my_interfaces::srv::SetRobotMode::Response> response)
{
    RCLCPP_INFO(this->get_logger(),
        "收到模式设置请求: %s, 安全模式: %s, 最大速度: %.2f",
        request->mode_name.c_str(),
        request->enable_safety ? "启用" : "禁用",
        request->max_speed);

    // 保存之前的模式
    response->previous_mode = current_mode_;

    // 验证请求参数
    if (request->mode_name.empty()) {
        response->success = false;
        response->message = "模式名称不能为空";
        response->actual_speed = max_speed_;
        return;
    }

    if (request->max_speed <= 0 || request->max_speed > 5.0) {
        response->success = false;
        response->message = "无效的最大速度, 应在 0-5.0 m/s 范围内";
        response->actual_speed = max_speed_;
        return;
    }

    // 检查支持的模式
    std::vector<std::string> supported_modes = {
        "manual", "autonomous", "navigation", "teleoperation"
    };

    if (std::find(supported_modes.begin(), supported_modes.end(),
        request->mode_name) == supported_modes.end()) {
        response->success = false;
        response->message = "不支持的模式: " + request->mode_name;
        response->actual_speed = max_speed_;
        return;
    }

    // 应用新设置
    current_mode_ = request->mode_name;

```

```

// 根据安全模式调整速度
if (request->enable_safety) {
    max_speed_ = std::min(request->max_speed, 2.0); // 安全模式限速
} else {
    max_speed_ = request->max_speed;
}

// 设置响应
response->success = true;
response->message = "模式设置成功: " + current_mode_;
response->actual_speed = max_speed_;

RCLCPP_INFO(this->get_logger(),
            "模式已切换: %s -> %s, 速度限制: %.2f m/s",
            response->previous_mode.c_str(),
            current_mode_.c_str(),
            max_speed_);
}

rclcpp::Service<my_interfaces::srv::SetRobotMode>::SharedPtr service_;
std::string current_mode_;
double max_speed_;
};

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<RobotModeServer>());
    rclcpp::shutdown();
    return 0;
}

```

客户端实现示例

src/robot_mode_client.cpp:

```

#include <rclcpp/rclcpp.hpp>
#include <my_interfaces/srv/set_robot_mode.hpp>
#include <chrono>
#include <memory>

using namespace std::chrono_literals;

class RobotModeClient : public rclcpp::Node
{
public:
    RobotModeClient() : Node("robot_mode_client")
    {
        // 创建客户端
        client_ = this->create_client<my_interfaces::srv::SetRobotMode>
("set_robot_mode");
    }
}

```

```

// 等待服务可用
while (!client_>wait_for_service(1s)) {
    if (!rclcpp::ok()) {
        RCLCPP_ERROR(this->get_logger(), "等待服务时被中断");
        return;
    }
    RCLCPP_INFO(this->get_logger(), "等待机器人模式设置服务可用...");
}

RCLCPP_INFO(this->get_logger(), "机器人模式客户端已启动");

// 演示不同的模式设置
test_mode_settings();
}

private:
void test_mode_settings()
{
    // 测试1: 设置为自主导航模式
    set_robot_mode("autonomous", true, 1.5);
    std::this_thread::sleep_for(2s);

    // 测试2: 设置为手动模式
    set_robot_mode("manual", false, 3.0);
    std::this_thread::sleep_for(2s);

    // 测试3: 设置为导航模式 (启用安全)
    set_robot_mode("navigation", true, 4.0);
    std::this_thread::sleep_for(2s);

    // 测试4: 无效模式测试
    set_robot_mode("invalid_mode", true, 2.0);
    std::this_thread::sleep_for(2s);

    // 测试5: 无效速度测试
    set_robot_mode("manual", true, 10.0);
}

void set_robot_mode(const std::string& mode, bool safety, double speed)
{
    auto request = std::make_shared<my_interfaces::srv::SetRobotMode::Request>();

    request->mode_name = mode;
    request->enable_safety = safety;
    request->max_speed = speed;

    RCLCPP_INFO(this->get_logger(),
        "请求设置模式: %s, 安全模式: %s, 最大速度: %.2f m/s",
        mode.c_str(), safety ? "启用" : "禁用", speed);

    // 异步调用服务

```

```

        auto future = client_>async_send_request(request);

        // 等待响应
        if (rclcpp::spin_until_future_complete(this->get_node_base_interface(),
future) ==
            rclcpp::FutureReturnCode::SUCCESS)
        {
            auto response = future.get();

            if (response->success) {
                RCLCPP_INFO(this->get_logger(),
                    "✅ 设置成功: %s (之前: %s), 实际速度: %.2f m/s",
                    response->message.c_str(),
                    response->previous_mode.c_str(),
                    response->actual_speed);
            } else {
                RCLCPP_WARN(this->get_logger(),
                    "❌ 设置失败: %s, 当前速度: %.2f m/s",
                    response->message.c_str(),
                    response->actual_speed);
            }
        } else {
            RCLCPP_ERROR(this->get_logger(), "服务调用失败");
        }
    }

    rclcpp::Client<my_interfaces::srv::SetRobotMode>::SharedPtr client_;
};

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    auto node = std::make_shared<RobotModeClient>();
    rclcpp::shutdown();
    return 0;
}

```

导航请求服务示例

src/navigation_server.cpp:

```

#include <rclcpp/rclcpp.hpp>
#include <my_interfaces/srv/navigation_request.hpp>
#include <nav_msgs/msg/path.hpp>
#include <geometry_msgs/msg/pose_stamped.hpp>
#include <cmath>

class NavigationServer : public rclcpp::Node
{
public:
    NavigationServer() : Node("navigation_server")
    {

```



```

        service_ = this->create_service<my_interfaces::srv::NavigationRequest>(
            "navigation_request",
            std::bind(&NavigationServer::handle_navigation_request, this,
                std::placeholders::_1, std::placeholders::_2));

        RCLCPP_INFO(this->get_logger(), "导航请求服务已启动");
    }

private:
    void handle_navigation_request(
        const std::shared_ptr<my_interfaces::srv::NavigationRequest::Request>
request,
        std::shared_ptr<my_interfaces::srv::NavigationRequest::Response> response)
    {
        RCLCPP_INFO(this->get_logger(), "收到导航请求, 算法: %s",
            request->planning_algorithm.c_str());

        // 计算路径长度
        double dx = request->goal_pose.pose.position.x - request-
>start_pose.pose.position.x;
        double dy = request->goal_pose.pose.position.y - request-
>start_pose.pose.position.y;
        double distance = std::sqrt(dx * dx + dy * dy);

        // 模拟路径规划
        nav_msgs::msg::Path path;
        path.header = request->start_pose.header;

        // 生成简单的直线路径
        int waypoint_count = static_cast<int>(distance * 2); // 每0.5米一个路径点
        waypoint_count = std::max(2, std::min(waypoint_count, 20));

        for (int i = 0; i <= waypoint_count; ++i) {
            geometry_msgs::msg::PoseStamped waypoint;
            waypoint.header = path.header;

            double ratio = static_cast<double>(i) / waypoint_count;
            waypoint.pose.position.x = request->start_pose.pose.position.x + dx *
ratio;
            waypoint.pose.position.y = request->start_pose.pose.position.y + dy *
ratio;
            waypoint.pose.position.z = request->start_pose.pose.position.z;

            waypoint.pose.orientation = request->goal_pose.pose.orientation;

            path.poses.push_back(waypoint);
            response->waypoints.push_back(waypoint);
        }

        // 设置响应
        response->success = true;
        response->result_message = "路径规划成功";
    }

```

```

        response->planned_path = path;
        response->path_length = distance;
        response->estimated_time = distance / 1.0; // 假设1m/s的速度

        RCLCPP_INFO(this->get_logger(),
                    "路径规划完成：长度=%.2fm，预计时间=%.2fs，路径点数=%zu",
                    distance, response->estimated_time, response->waypoints.size());
    }

    rclcpp::Service<my_interfaces::srv::NavigationRequest>::SharedPtr service_;
};

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<NavigationServer>());
    rclcpp::shutdown();
    return 0;
}

```

导航请求客户端示例

src/navigation_client.cpp:

```

#include <rclcpp/rclcpp.hpp>
#include <my_interfaces/srv/navigation_request.hpp>
#include <geometry_msgs/msg/pose_stamped.hpp>
#include <chrono>
#include <memory>

using namespace std::chrono_literals;

class NavigationClient : public rclcpp::Node
{
public:
    NavigationClient() : Node("navigation_client")
    {
        // 创建客户端
        client_ = this->create_client<my_interfaces::srv::NavigationRequest>
("navigation_request");

        // 等待服务可用
        while (!client_->wait_for_service(1s)) {
            if (!rclcpp::ok()) {
                RCLCPP_ERROR(this->get_logger(), "等待服务时被中断");
                return;
            }
            RCLCPP_INFO(this->get_logger(), "等待导航请求服务可用...");
        }
    }
};

```

```

}

RCLCPP_INFO(this->get_logger(), "导航客户端已启动");

// 演示不同的导航请求
test_navigation_requests();
}

private:
void test_navigation_requests()
{
    // 测试1: 简单直线导航
    request_navigation(
        create_pose(0.0, 0.0, 0.0), // 起点
        create_pose(5.0, 3.0, 0.0), // 终点
        "A_star", // 算法
        10.0 // 最大规划时间
    );
    std::this_thread::sleep_for(3s);

    // 测试2: 复杂路径规划
    request_navigation(
        create_pose(1.0, 1.0, 0.0), // 起点
        create_pose(-2.0, 4.0, 1.57), // 终点 (包含旋转)
        "RRT", // 算法
        15.0 // 最大规划时间
    );
    std::this_thread::sleep_for(3s);

    // 测试3: 长距离导航
    request_navigation(
        create_pose(0.0, 0.0, 0.0), // 起点
        create_pose(10.0, 8.0, 3.14), // 终点
        "PRM", // 算法
        20.0 // 最大规划时间
    );
}

geometry_msgs::msg::PoseStamped create_pose(double x, double y, double yaw)
{
    geometry_msgs::msg::PoseStamped pose;
    pose.header.stamp = this->now();
    pose.header.frame_id = "map";

    pose.pose.position.x = x;
    pose.pose.position.y = y;
    pose.pose.position.z = 0.0;

    // 将yaw角转换为四元数
    pose.pose.orientation.x = 0.0;
    pose.pose.orientation.y = 0.0;
    pose.pose.orientation.z = sin(yaw / 2.0);
}

```

```

        pose.pose.orientation.w = cos(yaw / 2.0);

        return pose;
    }

    void request_navigation(
        const geometry_msgs::msg::PoseStamped& start,
        const geometry_msgs::msg::PoseStamped& goal,
        const std::string& algorithm,
        double max_time)
    {
        auto request =
std::make_shared<my_interfaces::srv::NavigationRequest::Request>();

        // 设置请求参数
        request->start_pose = start;
        request->goal_pose = goal;
        request->planning_algorithm = algorithm;
        request->max_planning_time = max_time;
        request->use_obstacles = true;
        request->obstacle_types = {"wall", "furniture", "person"};

        RCLCPP_INFO(this->get_logger(),
            "请求导航规划:");
        RCLCPP_INFO(this->get_logger(),
            "  起点: (%.2f, %.2f, %.2f°)",
            start.pose.position.x, start.pose.position.y,
            atan2(start.pose.orientation.z, start.pose.orientation.w) * 2 *
180 / M_PI);
        RCLCPP_INFO(this->get_logger(),
            "  终点: (%.2f, %.2f, %.2f°)",
            goal.pose.position.x, goal.pose.position.y,
            atan2(goal.pose.orientation.z, goal.pose.orientation.w) * 2 * 180
/ M_PI);
        RCLCPP_INFO(this->get_logger(),
            "  算法: %s, 最大时间: %.1fs",
            algorithm.c_str(), max_time);

        // 异步调用服务
        auto future = client_->async_send_request(request);

        // 等待响应
        if (rclcpp::spin_until_future_complete(this->get_node_base_interface(),
future) ==
            rclcpp::FutureReturnCode::SUCCESS)
        {
            auto response = future.get();

            if (response->success) {
                RCLCPP_INFO(this->get_logger(),
                    "✅ 路径规划成功:");
                RCLCPP_INFO(this->get_logger(),

```

```

        " 消息: %s", response->result_message.c_str());
RCLCPP_INFO(this->get_logger(),
        " 路径长度: %.2f 米", response->path_length);
RCLCPP_INFO(this->get_logger(),
        " 预计时间: %.2f 秒", response->estimated_time);
RCLCPP_INFO(this->get_logger(),
        " 路径点数量: %zu", response->waypoints.size());

// 显示部分路径点
if (!response->waypoints.empty()) {
    RCLCPP_INFO(this->get_logger(), " 关键路径点:");

    // 显示起点
    auto& first = response->waypoints.front();
    RCLCPP_INFO(this->get_logger(),
        " 起点: (%.2f, %.2f)",
        first.pose.position.x, first.pose.position.y);

    // 显示中点（如果有的话）
    if (response->waypoints.size() > 2) {
        auto& middle = response->waypoints[response-
>waypoints.size() / 2];
        RCLCPP_INFO(this->get_logger(),
            " 中点: (%.2f, %.2f)",
            middle.pose.position.x, middle.pose.position.y);
    }

    // 显示终点
    auto& last = response->waypoints.back();
    RCLCPP_INFO(this->get_logger(),
        " 终点: (%.2f, %.2f)",
        last.pose.position.x, last.pose.position.y);
}

// 分析路径
analyze_path(response->planned_path);

} else {
    RCLCPP_WARN(this->get_logger(),
        "✗ 路径规划失败: %s", response->result_message.c_str());
}
} else {
    RCLCPP_ERROR(this->get_logger(), "导航服务调用失败");
}
}

void analyze_path(const nav_msgs::msg::Path& path)
{
    if (path.poses.size() < 2) {
        RCLCPP_WARN(this->get_logger(), "路径点太少, 无法分析");
        return;
    }
}

```

```

// 计算路径统计信息
double total_distance = 0.0;
double max_segment = 0.0;
double min_segment = std::numeric_limits<double>::max();

for (size_t i = 1; i < path.poses.size(); ++i) {
    const auto& p1 = path.poses[i-1].pose.position;
    const auto& p2 = path.poses[i].pose.position;

    double dx = p2.x - p1.x;
    double dy = p2.y - p1.y;
    double segment_length = sqrt(dx * dx + dy * dy);

    total_distance += segment_length;
    max_segment = std::max(max_segment, segment_length);
    min_segment = std::min(min_segment, segment_length);
}

RCLCPP_INFO(this->get_logger(),
            "  路径分析: 总长度=%.2fm, 最大段=%.2fm, 最小段=%.2fm",
            total_distance, max_segment, min_segment);
}

rclcpp::Client<my_interfaces::srv::NavigationRequest>::SharedPtr client_;
};

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    auto node = std::make_shared<NavigationClient>();
    rclcpp::shutdown();
    return 0;
}

```

交互式导航客户端示例

src/interactive_navigation_client.cpp:

```

#include <rclcpp/rclcpp.hpp>
#include <my_interfaces/srv/navigation_request.hpp>
#include <geometry_msgs/msg/pose_stamped.hpp>
#include <iostream>
#include <string>

class InteractiveNavigationClient : public rclcpp::Node
{
public:
    InteractiveNavigationClient() : Node("interactive_navigation_client")
    {
        client_ = this->create_client<my_interfaces::srv::NavigationRequest>
("navigation_request");
    }
}

```

```

while (!client_>wait_for_service(std::chrono::seconds(1))) {
    if (!rclcpp::ok()) {
        RCLCPP_ERROR(this->get_logger(), "等待服务时被中断");
        return;
    }
    RCLCPP_INFO(this->get_logger(), "等待导航请求服务可用...");
}

RCLCPP_INFO(this->get_logger(), "交互式导航客户端已启动");
run_interactive_mode();
}

private:
void run_interactive_mode()
{
    std::string input;

    while (rclcpp::ok()) {
        std::cout << "\n=== 导航请求客户端 ===" << std::endl;
        std::cout << "请选择操作:" << std::endl;
        std::cout << "1. 发送导航请求" << std::endl;
        std::cout << "2. 预设导航点" << std::endl;
        std::cout << "3. 退出" << std::endl;
        std::cout << "请输入选择 (1-3): ";

        std::getline(std::cin, input);

        if (input == "1") {
            handle_custom_navigation();
        } else if (input == "2") {
            handle_preset_navigation();
        } else if (input == "3") {
            break;
        } else {
            std::cout << "无效选择, 请重试." << std::endl;
        }
    }
}

void handle_custom_navigation()
{
    double start_x, start_y, start_yaw;
    double goal_x, goal_y, goal_yaw;
    std::string algorithm;
    double max_time;

    std::cout << "\n--- 自定义导航请求 ---" << std::endl;

    // 获取起点
    std::cout << "起点 X 坐标: ";
    std::cin >> start_x;

```

```

std::cout << "起点 Y 坐标: ";
std::cin >> start_y;
std::cout << "起点角度 (度): ";
std::cin >> start_yaw;

// 获取终点
std::cout << "终点 X 坐标: ";
std::cin >> goal_x;
std::cout << "终点 Y 坐标: ";
std::cin >> goal_y;
std::cout << "终点角度 (度): ";
std::cin >> goal_yaw;

// 获取算法
std::cout << "规划算法 (A_star/RRT/PRM): ";
std::cin >> algorithm;

// 获取最大时间
std::cout << "最大规划时间 (秒): ";
std::cin >> max_time;

std::cin.ignore(); // 清除输入缓冲区

// 发送请求
auto start_pose = create_pose(start_x, start_y, start_yaw * M_PI / 180.0);
auto goal_pose = create_pose(goal_x, goal_y, goal_yaw * M_PI / 180.0);

send_navigation_request(start_pose, goal_pose, algorithm, max_time);
}

void handle_preset_navigation()
{
    std::cout << "\n--- 预设导航点 ---" << std::endl;
    std::cout << "1. 客厅到厨房" << std::endl;
    std::cout << "2. 门口到充电桩" << std::endl;
    std::cout << "3. 书房到阳台" << std::endl;
    std::cout << "请选择预设路线 (1-3): ";

    std::string choice;
    std::getline(std::cin, choice);

    if (choice == "1") {
        // 客厅到厨房
        auto start = create_pose(1.0, 2.0, 0.0);
        auto goal = create_pose(5.0, 1.0, -1.57);
        send_navigation_request(start, goal, "A_star", 10.0);
    } else if (choice == "2") {
        // 门口到充电桩
        auto start = create_pose(0.0, 0.0, 0.0);
        auto goal = create_pose(-2.0, 3.0, 3.14);
        send_navigation_request(start, goal, "RRT", 15.0);
    } else if (choice == "3") {

```



```

        // 书房到阳台
        auto start = create_pose(3.0, 4.0, 1.57);
        auto goal = create_pose(6.0, 6.0, 0.0);
        send_navigation_request(start, goal, "PRM", 12.0);
    } else {
        std::cout << "无效选择。" << std::endl;
    }
}

geometry_msgs::msg::PoseStamped create_pose(double x, double y, double yaw)
{
    geometry_msgs::msg::PoseStamped pose;
    pose.header.stamp = this->now();
    pose.header.frame_id = "map";

    pose.pose.position.x = x;
    pose.pose.position.y = y;
    pose.pose.position.z = 0.0;

    pose.pose.orientation.x = 0.0;
    pose.pose.orientation.y = 0.0;
    pose.pose.orientation.z = sin(yaw / 2.0);
    pose.pose.orientation.w = cos(yaw / 2.0);

    return pose;
}

void send_navigation_request(
    const geometry_msgs::msg::PoseStamped& start,
    const geometry_msgs::msg::PoseStamped& goal,
    const std::string& algorithm,
    double max_time)
{
    auto request =
std::make_shared<my_interfaces::srv::NavigationRequest::Request>();
    request->start_pose = start;
    request->goal_pose = goal;
    request->planning_algorithm = algorithm;
    request->max_planning_time = max_time;
    request->use_obstacles = true;
    request->obstacle_types = {"wall", "furniture", "person"};

    std::cout << "\n发送导航请求..." << std::endl;

    auto future = client_->async_send_request(request);

    if (rclcpp::spin_until_future_complete(this->get_node_base_interface(),
future) ==
        rclcpp::FutureReturnCode::SUCCESS)
    {
        auto response = future.get();
    }
}

```

```

        if (response->success) {
            std::cout << "✅ 导航规划成功!" << std::endl;
            std::cout << "    路径长度: " << response->path_length << " 米" <<
std::endl;
            std::cout << "    预计时间: " << response->estimated_time << " 秒" <<
std::endl;
            std::cout << "    路径点数: " << response->waypoints.size() <<
std::endl;
        } else {
            std::cout << "❌ 导航规划失败: " << response->result_message <<
std::endl;
        }
    } else {
        std::cout << "❌ 服务调用失败" << std::endl;
    }

    std::cout << "\n按Enter键继续...";
    std::cin.get();
}

rclcpp::Client<my_interfaces::srv::NavigationRequest>::SharedPtr client_;
};

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    auto node = std::make_shared<InteractiveNavigationClient>();
    rclcpp::shutdown();
    return 0;
}

```

编译配置

CMakeLists.txt

```

cmake_minimum_required(VERSION 3.8)
project(interface_demo)

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
    add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# 查找依赖包
find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(my_interfaces REQUIRED)
find_package(nav_msgs REQUIRED)
find_package(geometry_msgs REQUIRED)

# 创建可执行文件
add_executable(robot_mode_server src/robot_mode_server.cpp)
add_executable(robot_mode_client src/robot_mode_client.cpp)

```

```

add_executable(navigation_server src/navigation_server.cpp)
add_executable(navigation_client src/navigation_client.cpp) # 新增
add_executable(interactive_navigation_client src/interactive_navigation_client.cpp)
# 新增

# 设置依赖
ament_target_dependencies(robot_mode_server rclcpp my_interfaces)
ament_target_dependencies(robot_mode_client rclcpp my_interfaces)
ament_target_dependencies(navigation_server rclcpp my_interfaces nav_msgs
geometry_msgs)
ament_target_dependencies(navigation_client rclcpp my_interfaces nav_msgs
geometry_msgs) # 新增
ament_target_dependencies(interactive_navigation_client rclcpp my_interfaces
nav_msgs geometry_msgs) # 新增

# 安装可执行文件
install(TARGETS
  robot_mode_server
  robot_mode_client
  navigation_server
  navigation_client # 新增
  interactive_navigation_client # 新增
  DESTINATION lib/${PROJECT_NAME})

ament_package()

```

编译和运行

```

# 1. 编译
cd ~/ros2_ws
colcon build --packages-select interface_demo

# 2. 设置环境
source install/setup.bash

# 3.1 运行服务端（终端1）
ros2 run interface_demo robot_mode_server

# 3.2 运行客户端（终端2）
ros2 run interface_demo robot_mode_client

# 3.3 手动调用服务
ros2 service call /set_robot_mode my_interfaces/srv/SetRobotMode \
  '{mode_name: "autonomous", enable_safety: true, max_speed: 2.5}'

# 4.1 运行导航服务端（终端1）
ros2 run interface_demo navigation_server

# 4.2 运行自动化导航客户端（终端2）
ros2 run interface_demo navigation_client

```

4.3 或运行交互式导航客户端（终端3）

```
ros2 run interface_demo interactive_navigation_client
```

4.4 手动调用导航服务

```
ros2 service call /navigation_request my_interfaces/srv/NavigationRequest \
  '{start_pose: {header: {frame_id: "map"}, pose: {position: {x: 0, y: 0, z: 0}}},
  goal_pose: {header: {frame_id: "map"}, pose: {position: {x: 5, y: 3, z: 0}}},
  planning_algorithm: "A_star", max_planning_time: 10.0, use_obstacles: true}'
```

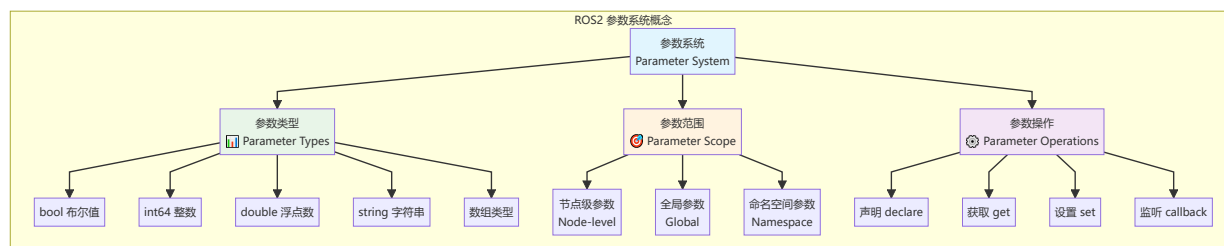
4.6 参数

4.6.1 参数是什么

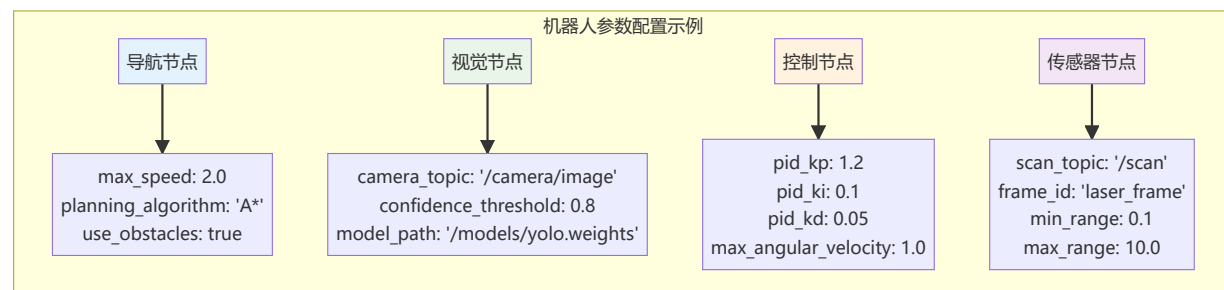
定义和概念

- **参数 (Parameters)**：ROS2中用于配置节点行为的键值对系统
- **特点**：动态可配置、类型安全、支持运行时修改
- **作用**：作为机器人系统的全局字典，存储配置信息

ROS2参数系统概念图

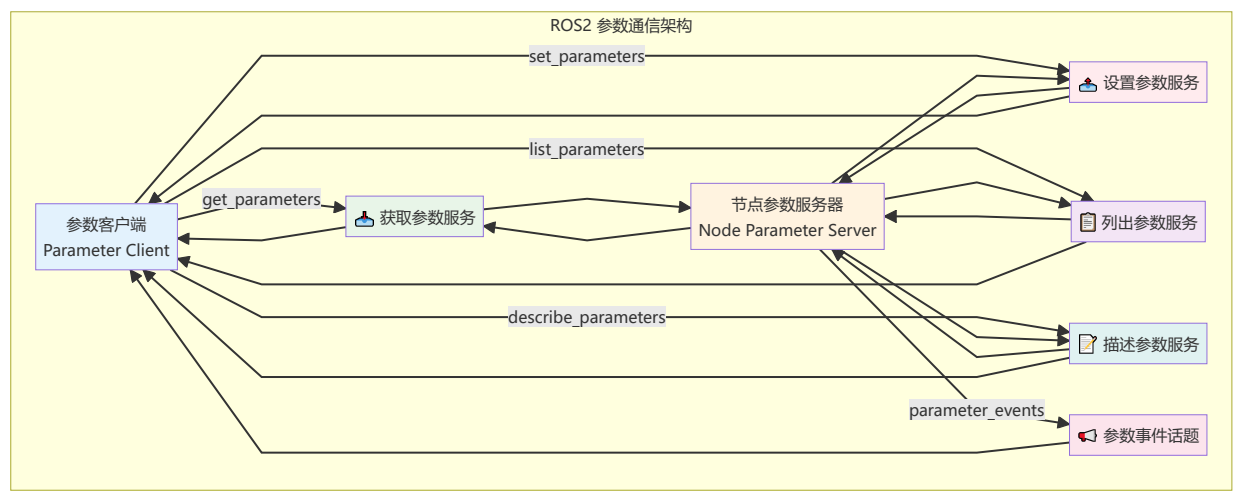


参数在机器人系统中的作用

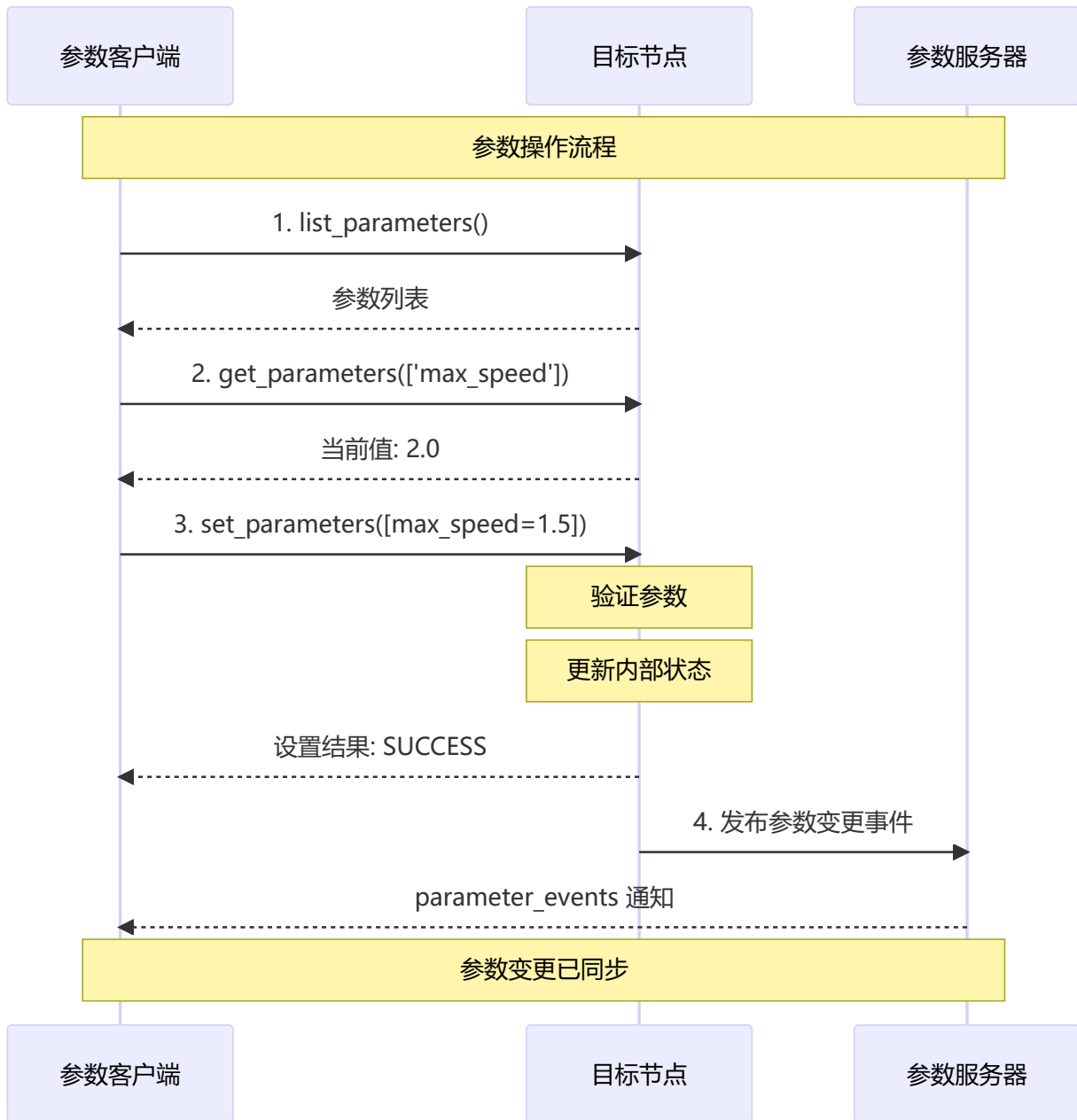


4.6.2 参数通信模型

参数服务架构图



参数操作时序图

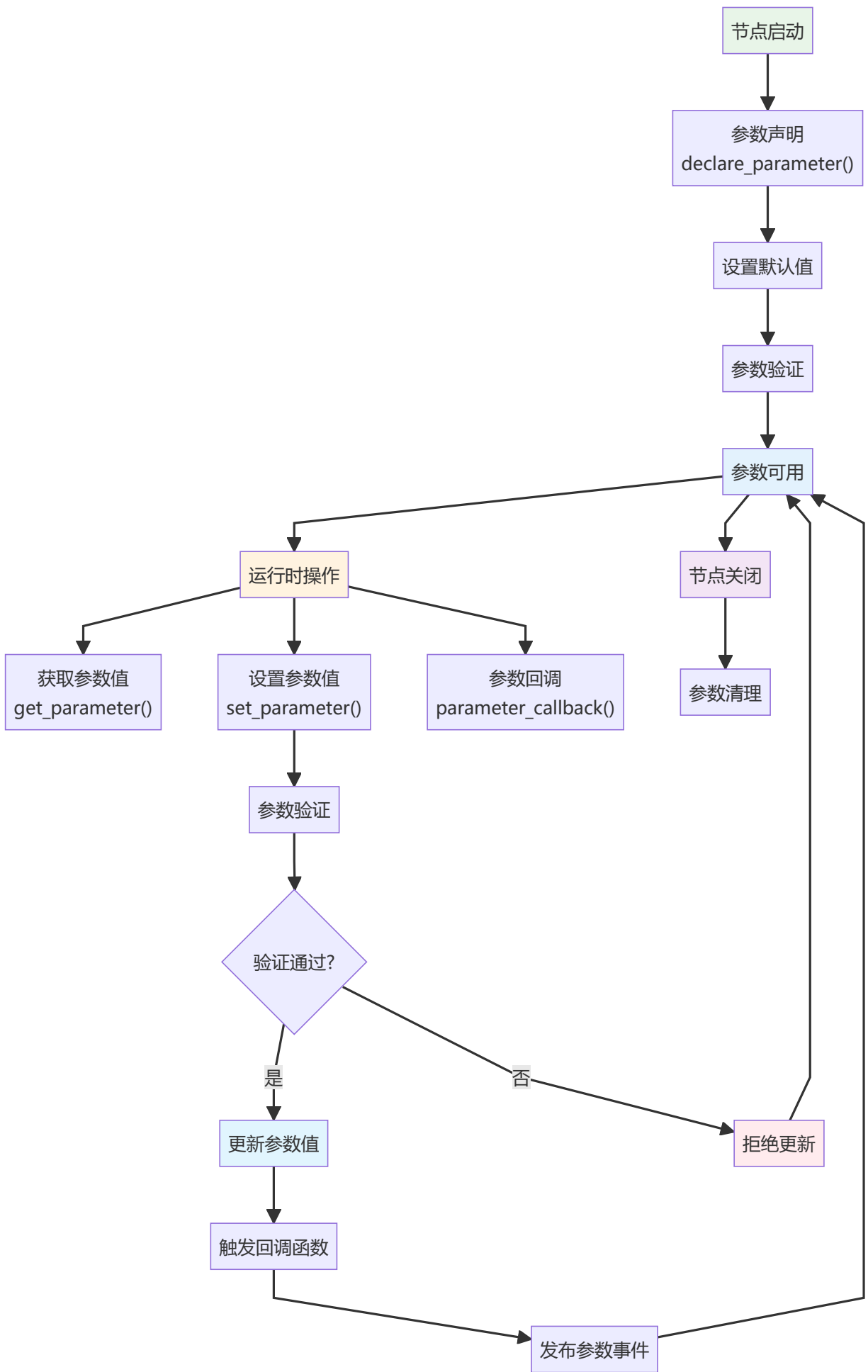


详细关系说明

1. 目标节点 (Target Node) - 参数的拥有者。
 - 目标节点是参数的实际拥有者，每个节点都内置了参数管理功能：
2. 参数客户端 (Parameter Client) - 参数的访问者
 - 参数客户端是想要访问其他节点参数的组件
3. 参数服务端 (Parameter Server) - 内置在每个节点中

- 重要概念：ROS2中没有独立的参数服务器，每个节点都内置参数服务端功能！

参数生命周期图



4.6.3 参数C++编程方法

创建参数演示功能包

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_cmake parameter_demo \
  --dependencies rclcpp rcl_interfaces
```

命令解析：

- --dependencies: 指定包的依赖项
 - rclcpp: ROS2: 的C++客户端库
 - rcl_interfaces: ROS2的核心接口包（包含参数相关的接口）

基础参数使用示例

src/basic_parameter_node.cpp:

```
/**
 * @file parameter_demo/src/basic_parameter_node.cpp
 * @brief ROS2 基础参数演示节点 - 展示参数的声明、获取、设置和回调机制
 *
 * 本文件演示了ROS2中参数系统的基本使用方法，包括：
 * 1. 声明不同类型的参数（布尔、整数、浮点、字符串、数组）
 * 2. 设置参数描述符和约束条件
 * 3. 获取和更新参数值
 * 4. 监听参数变更事件
 * 5. 参数验证和错误处理
 */

#include <rclcpp/rclcpp.hpp>                                     // ROS2 C++ 客户端库
#include <rcl_interfaces/msg/parameter_descriptor.hpp>          // 参数描述符消息类型
#include <rcl_interfaces/msg/parameter_event.hpp>               // 参数事件消息类型
#include <string>                                                  // 标准字符串库
#include <vector>                                                  // 标准向量容器库

/**
 * @class BasicParameterNode
 * @brief 基础参数节点类 - 继承自rclcpp::Node
 *
 * 该类演示了ROS2参数系统的完整使用流程：
 * - 参数的声明和初始化
 * - 参数值的获取和显示
 * - 参数变更的监听和处理
 */
```

```

* - 定时器的使用
*/
class BasicParameterNode : public rclcpp::Node
{
public:
    /**
     * @brief 构造函数 - 初始化基础参数节点
     *
     * 在构造函数中完成以下操作：
     * 1. 调用父类构造函数，设置节点名称为"basic_parameter_node"
     * 2. 声明所有需要的参数并设置默认值
     * 3. 获取当前参数值并显示
     * 4. 设置参数变更回调函数
     * 5. 创建定时器定期执行任务
     */
    BasicParameterNode() : Node("basic_parameter_node")
    {
        // 声明参数并设置默认值
        // 这一步是必须的，所有要使用的参数都需要先声明
        declare_basic_parameters();

        // 获取参数值
        // 从参数服务器获取当前参数值并存储到成员变量中
        get_parameters();

        // 设置参数变更回调
        // 当参数被外部修改时（如通过ros2 param set命令），会触发回调函数
        setup_parameter_callback();

        // 创建定时器定期显示参数
        // 每5秒执行一次timer_callback函数，用于定期显示状态信息
        timer_ = this->create_wall_timer(
            std::chrono::seconds(5),
            std::bind(&BasicParameterNode::timer_callback, this));

        RCLCPP_INFO(this->get_logger(), "基础参数节点已启动");
    }

private:
    /**
     * @brief 声明基础参数 - 定义节点使用的所有参数
     *
     * 本函数演示了如何声明不同类型的参数：
     * - 布尔参数：用于开关控制
     * - 整数参数：用于频率、计数等
     * - 浮点参数：用于速度、距离等连续值
     * - 字符串参数：用于名称、路径等文本信息
     * - 数组参数：用于多维数据，如关节位置、坐标等
     *
     * 每个参数都包含：
     * - 参数名称：用于标识参数
     * - 默认值：节点启动时的初始值

```

```

* - 描述符: 包含参数说明和约束条件
*/
void declare_basic_parameters()
{
    // 声明不同类型的参数

    // 布尔参数 - 控制是否启用调试模式
    auto bool_desc = rcl_interfaces::msg::ParameterDescriptor();
    bool_desc.description = "是否启用调试模式";
    this->declare_parameter("debug_mode", false, bool_desc);

    // 整数参数 - 设置循环频率, 带有范围约束
    auto int_desc = rcl_interfaces::msg::ParameterDescriptor();
    int_desc.description = "循环频率 (Hz)";
    int_desc.integer_range.resize(1); // 设置范围约束数组大小

    int_desc.integer_range[0].from_value = 1; // 最小值: 1 Hz
    int_desc.integer_range[0].to_value = 100; // 最大值: 100 Hz
    this->declare_parameter("loop_rate", 10, int_desc);

    // 浮点参数 - 设置最大速度, 带有范围约束
    auto double_desc = rcl_interfaces::msg::ParameterDescriptor();
    double_desc.description = "最大速度 (m/s)";
    double_desc.floating_point_range.resize(1); // 设置浮点范围约束
    double_desc.floating_point_range[0].from_value = 0.0; // 最小速度: 0.0 m/s
    double_desc.floating_point_range[0].to_value = 5.0; // 最大速度: 5.0 m/s
    this->declare_parameter("max_speed", 1.0, double_desc);

    // 字符串参数 - 设置机器人名称
    auto string_desc = rcl_interfaces::msg::ParameterDescriptor();
    string_desc.description = "机器人名称";
    this->declare_parameter("robot_name", std::string("robot1"), string_desc);

    // 数组参数 - 设置关节位置数组 (6自由度机械臂)
    auto array_desc = rcl_interfaces::msg::ParameterDescriptor();
    array_desc.description = "关节位置数组";
    std::vector<double> default_joints = {0.0, 0.0, 0.0, 0.0, 0.0, 0.0}; // 6个关节的默认位置
    this->declare_parameter("joint_positions", default_joints, array_desc);

    // 字符串数组参数 - 设置传感器名称列表
    auto string_array_desc = rcl_interfaces::msg::ParameterDescriptor();
    string_array_desc.description = "传感器名称列表";
    std::vector<std::string> default_sensors = {"camera", "lidar", "imu"}; // 默认传感器列表
    this->declare_parameter("sensor_names", default_sensors, string_array_desc);
}

/**
 * @brief 获取参数值 - 从参数服务器读取当前参数值

```

```

*
* 本函数演示了如何获取不同类型的参数值：
* - 使用as_bool()获取布尔值
* - 使用as_int()获取整数值
* - 使用as_double()获取浮点值
* - 使用as_string()获取字符串值
* - 使用as_double_array()获取浮点数组
* - 使用as_string_array()获取字符串数组
*
* 获取的参数值会存储到对应的成员变量中，供节点其他部分使用
*/
void get_parameters()
{
    // 获取参数值并存储到成员变量
    debug_mode_ = this->get_parameter("debug_mode").as_bool(); //
    获取调试模式状态
    loop_rate_ = this->get_parameter("loop_rate").as_int(); //
    获取循环频率
    max_speed_ = this->get_parameter("max_speed").as_double(); //
    获取最大速度
    robot_name_ = this->get_parameter("robot_name").as_string(); //
    获取机器人名称
    joint_positions_ = this->get_parameter("joint_positions").as_double_array();
    // 获取关节位置数组
    sensor_names_ = this->get_parameter("sensor_names").as_string_array(); //
    获取传感器名称数组

    // 显示当前参数值
    display_parameters();
}

/**
 * @brief 设置参数变更回调 - 监听参数的外部修改
 *
 * 当参数通过外部方式被修改时（例如：ros2 param set命令、rqt参数工具等），
 * 注册的回调函数会被自动调用，允许节点响应参数变更
 */
void setup_parameter_callback()
{
    // 设置参数变更回调
    // 使用std::bind绑定成员函数和占位符
    parameter_callback_handle_ = this->add_on_set_parameters_callback(
        std::bind(&BasicParameterNode::on_parameter_change, this,
std::placeholders::_1));
}

/**
 * @brief 参数变更回调函数 - 处理参数修改事件
 *
 * @param parameters 被修改的参数列表
 * @return SetParametersResult 参数设置结果，包含成功状态和错误信息
 */

```

```

* 该函数负责：
* 1. 验证新参数值的合法性
* 2. 更新内部成员变量
* 3. 记录参数变更日志
* 4. 返回操作结果
*
* 如果参数值不合法，会拒绝修改并返回错误信息
*/
rc1_interfaces::msg::SetParametersResult on_parameter_change(
    const std::vector<rc1cpp::Parameter> & parameters)
{
    rc1_interfaces::msg::SetParametersResult result;
    result.successful = true; // 默认设置为成功

    // 遍历所有被修改的参数
    for (const auto & param : parameters) {
        RCLCPP_INFO(this->get_logger(), "参数变更: %s",
param.get_name().c_str());

        // 根据参数名称进行不同的处理
        if (param.get_name() == "debug_mode") {
            // 处理调试模式参数变更
            debug_mode_ = param.as_bool();
            RCLCPP_INFO(this->get_logger(), "调试模式: %s", debug_mode_ ? "启用" :
"禁用");
        }
        else if (param.get_name() == "loop_rate") {
            // 处理循环频率参数变更，需要验证范围
            int new_rate = param.as_int();
            if (new_rate >= 1 && new_rate <= 100) {
                loop_rate_ = new_rate;
                RCLCPP_INFO(this->get_logger(), "循环频率更新为: %d Hz",
loop_rate_);
            }
            else {
                // 参数值超出范围，拒绝修改
                result.successful = false;
                result.reason = "循环频率必须在1-100之间";
                return result;
            }
        }
        else if (param.get_name() == "max_speed") {
            // 处理最大速度参数变更，需要验证范围
            double new_speed = param.as_double();
            if (new_speed >= 0.0 && new_speed <= 5.0) {
                max_speed_ = new_speed;
                RCLCPP_INFO(this->get_logger(), "最大速度更新为: %.2f m/s",
max_speed_);
            }
            else {
                // 参数值超出范围，拒绝修改
                result.successful = false;
                result.reason = "最大速度必须在0.0-5.0之间";
                return result;
            }
        }
    }
}

```

```

    }
}
else if (param.get_name() == "robot_name") {
    // 处理机器人名称参数变更
    robot_name_ = param.as_string();
    RCLCPP_INFO(this->get_logger(), "机器人名称更新为: %s",
robot_name_.c_str());
}
else if (param.get_name() == "joint_positions") {
    // 处理关节位置数组参数变更
    joint_positions_ = param.as_double_array();
    RCLCPP_INFO(this->get_logger(), "关节位置已更新, 关节数量: %zu",
joint_positions_.size());
}
else if (param.get_name() == "sensor_names") {
    // 处理传感器名称数组参数变更
    sensor_names_ = param.as_string_array();
    RCLCPP_INFO(this->get_logger(), "传感器列表已更新, 传感器数量: %zu",
sensor_names_.size());
}
}

return result; // 返回操作结果
}

/**
 * @brief 定时器回调函数 - 定期执行的任务
 *
 * 该函数每5秒被调用一次, 用于:
 * 1. 在调试模式下显示当前参数值
 * 2. 模拟使用参数进行实际工作
 * 3. 输出节点运行状态信息
 */
void timer_callback()
{
    // 如果启用了调试模式, 则显示详细的参数信息
    if (debug_mode_) {
        display_parameters();
    }

    // 模拟使用参数进行工作
    // 在实际应用中, 这里会根据参数值执行具体的机器人控制逻辑
    RCLCPP_INFO(this->get_logger(),
        "机器人 %s 正在以 %.2f m/s 的最大速度运行 (频率: %d Hz)",
        robot_name_.c_str(), max_speed_, loop_rate_);
}

/**
 * @brief 显示参数信息 - 格式化输出所有当前参数值
 *
 * 该函数以易读的格式显示所有参数的当前值,
 * 包括简单类型 (布尔、整数、浮点、字符串) 和数组类型参数

```

```

*/
void display_parameters()
{
    RCLCPP_INFO(this->get_logger(), "=== 当前参数值 ===");

    // 显示简单类型参数
    RCLCPP_INFO(this->get_logger(), "调试模式: %s", debug_mode_ ? "启用" : "禁用");
    RCLCPP_INFO(this->get_logger(), "循环频率: %d Hz", loop_rate_);
    RCLCPP_INFO(this->get_logger(), "最大速度: %.2f m/s", max_speed_);
    RCLCPP_INFO(this->get_logger(), "机器人名称: %s", robot_name_.c_str());

    // 显示关节位置数组（格式化为 [x1, x2, x3, ...] 形式）
    std::string joints_str = "关节位置: [";
    for (size_t i = 0; i < joint_positions_.size(); ++i) {
        joints_str += std::to_string(joint_positions_[i]);
        if (i < joint_positions_.size() - 1) joints_str += ", ";
    }
    joints_str += "];";
    RCLCPP_INFO(this->get_logger(), "%s", joints_str.c_str());

    // 显示传感器名称数组（格式化为 [name1, name2, name3, ...] 形式）
    std::string sensors_str = "传感器: [";
    for (size_t i = 0; i < sensor_names_.size(); ++i) {
        sensors_str += sensor_names_[i];
        if (i < sensor_names_.size() - 1) sensors_str += ", ";
    }
    sensors_str += "];";
    RCLCPP_INFO(this->get_logger(), "%s", sensors_str.c_str());
}

// ===== 成员变量定义 =====

// ROS2 相关对象
rclcpp::TimerBase::SharedPtr timer_; //
定时器对象，用于定期执行任务
rclcpp::node_interfaces::OnSetParametersCallbackHandle::SharedPtr
parameter_callback_handle_; // 参数回调句柄

// 参数值存储变量（与声明的参数一一对应）
bool debug_mode_; // 调试模式开关
int loop_rate_; // 循环频率 (Hz)
double max_speed_; // 最大速度 (m/s)
std::string robot_name_; // 机器人名称
std::vector<double> joint_positions_; // 关节位置数组
std::vector<std::string> sensor_names_; // 传感器名称数组
};

/**
 * @brief 主函数 - 程序入口点
 *
 * @param argc 命令行参数个数
 * @param argv 命令行参数数组

```

```

* @return int 程序退出码
*
* 主函数执行流程：
* 1. 初始化ROS2系统
* 2. 创建BasicParameterNode节点实例
* 3. 进入ROS2事件循环（spin）
* 4. 节点关闭时清理资源
*/
int main(int argc, char ** argv)
{
    // 初始化ROS2系统
    rclcpp::init(argc, argv);

    // 创建节点实例
    auto node = std::make_shared<BasicParameterNode>();

    // 进入ROS2事件循环，处理回调函数和消息
    // 此函数会阻塞直到节点被关闭（如Ctrl+C）
    rclcpp::spin(node);

    // 清理ROS2系统资源
    rclcpp::shutdown();

    return 0; // 正常退出
}

```

高级参数管理示例

src/advanced_parameter_node.cpp:

```

/**
 * @file parameter_demo/src/advanced_parameter_node.cpp
 * @brief ROS2 高级参数演示节点 - 展示参数系统的高级功能
 *
 * 本文件演示了ROS2中参数系统的高级使用方法，包括：
 * 1. 参数数组的组织和管理
 * 2. 从YAML配置文件加载参数
 * 3. 参数验证和约束检查
 * 4. 参数事件监听和响应
 * 5. 参数的保存和导出功能
 * 6. 嵌套参数结构的处理
 * 7. 复杂参数系统的架构设计
 */

#include <rclcpp/rclcpp.hpp> // ROS2 C++ 客户端库
#include <rcl_interfaces/msg/parameter_descriptor.hpp> // 参数描述符消息类型
#include <rcl_interfaces/msg/parameter_event.hpp> // 参数事件消息类型
#include <filesystem> // 文件系统操作库
#include <fstream> // 文件流操作库
#include <yaml-cpp/yaml.h> // YAML文件解析库

/**

```



```

* @class AdvancedParameterNode
* @brief 高级参数节点类 - 继承自rclcpp::Node
*
* 该类演示了ROS2参数系统的高级功能：
* - 分组参数管理：按功能模块组织参数
* - 配置文件支持：从YAML文件加载和保存参数
* - 参数验证：实现复杂的参数约束检查
* - 事件监听：监控参数变更事件
* - 动态配置：支持运行时参数修改
*/
class AdvancedParameterNode : public rclcpp::Node
{
public:
    /**
     * @brief 构造函数 - 初始化高级参数节点
     *
     * 在构造函数中完成以下高级功能的初始化：
     * 1. 声明分组参数（导航、传感器、控制）
     * 2. 从YAML配置文件加载参数值
     * 3. 设置参数验证规则和回调
     * 4. 建立参数事件监听机制
     */
    AdvancedParameterNode() : Node("advanced_parameter_node")
    {
        // 声明参数组
        // 按功能模块组织参数，便于管理和维护
        declare_parameter_groups();

        // 加载参数文件
        // 从YAML配置文件读取预设的参数值
        load_parameters_from_file();

        // 设置参数验证
        // 建立参数合法性检查机制
        setup_parameter_validation();

        // 设置参数监听
        // 监控系统中的参数变更事件
        setup_parameter_monitoring();

        RCLCPP_INFO(this->get_logger(), "高级参数节点已启动");
    }

private:
    /**
     * @brief 声明参数组 - 按功能模块组织参数
     *
     * 将参数按功能划分为不同的组：
     * - 导航参数组：路径规划、速度控制等
     * - 传感器参数组：激光雷达、摄像头等设备配置
     * - 控制参数组：PID控制器、控制频率等
     */

```

```

* 这种组织方式便于：
* - 参数的逻辑分类和管理
* - 配置文件的结构化组织
* - 不同模块的独立配置
*/
void declare_parameter_groups()
{
    // 导航参数组
    // 包含路径规划、速度限制等导航相关参数
    declare_navigation_parameters();

    // 传感器参数组
    // 包含各种传感器的配置参数
    declare_sensor_parameters();

    // 控制参数组
    // 包含控制算法和控制器参数
    declare_control_parameters();
}

/**
 * @brief 声明导航参数 - 路径规划和导航控制相关参数
 *
 * 导航参数组包含：
 * - 路径规划算法选择和配置
 * - 规划时间和分辨率限制
 * - 速度和运动约束参数
 * - 障碍物处理选项
 */
void declare_navigation_parameters()
{
    auto desc = rcl_interfaces::msg::ParameterDescriptor();

    // 路径规划参数
    // 选择使用的路径规划算法（A*、RRT、Dijkstra等）
    desc.description = "路径规划算法";
    this->declare_parameter("navigation.planning_algorithm",
std::string("A_star"), desc);

    // 最大规划时间限制，防止规划算法运行过久
    desc.description = "最大规划时间（秒）";
    this->declare_parameter("navigation.max_planning_time", 10.0, desc);

    // 路径规划的空间分辨率，影响规划精度和计算量
    desc.description = "规划分辨率（米）";
    this->declare_parameter("navigation.planning_resolution", 0.05, desc);

    // 是否在规划中考虑障碍物
    desc.description = "是否使用障碍物";
    this->declare_parameter("navigation.use_obstacles", true, desc);

    // 导航速度参数

```

```

// 机器人线速度上限，确保安全运行
desc.description = "最大线速度 (m/s)";
this->declare_parameter("navigation.max_linear_velocity", 1.0, desc);

// 机器人角速度上限，防止过快转向
desc.description = "最大角速度 (rad/s)";
this->declare_parameter("navigation.max_angular_velocity", 1.0, desc);
}

/**
 * @brief 声明传感器参数 - 各类传感器的配置参数
 *
 * 传感器参数组包含：
 * - 激光雷达的话题、坐标系、检测范围
 * - 摄像头的分辨率、帧率、话题配置
 * - 各传感器的校准和处理参数
 */
void declare_sensor_parameters()
{
    auto desc = rcl_interfaces::msg::ParameterDescriptor();

    // 激光雷达参数配置
    // 激光雷达数据发布的ROS话题名称
    desc.description = "激光雷达话题名称";
    this->declare_parameter("sensors.lidar.topic", std::string("/scan"), desc);

    // 激光雷达在机器人上的坐标系名称
    desc.description = "激光雷达坐标系";
    this->declare_parameter("sensors.lidar.frame_id",
std::string("laser_frame"), desc);

    // 激光雷达的最小有效检测距离
    desc.description = "最小检测距离 (米)";
    this->declare_parameter("sensors.lidar.min_range", 0.1, desc);

    // 激光雷达的最大有效检测距离
    desc.description = "最大检测距离 (米)";
    this->declare_parameter("sensors.lidar.max_range", 10.0, desc);

    // 摄像头参数配置
    // 摄像头图像数据发布的ROS话题名称
    desc.description = "摄像头话题名称";
    this->declare_parameter("sensors.camera.topic",
std::string("/camera/image_raw"), desc);

    // 摄像头图像的像素宽度
    desc.description = "图像宽度";
    this->declare_parameter("sensors.camera.width", 640, desc);

    // 摄像头图像的像素高度
    desc.description = "图像高度";
    this->declare_parameter("sensors.camera.height", 480, desc);
}

```

```

// 摄像头的采集帧率
desc.description = "帧率 (fps)";
this->declare_parameter("sensors.camera.fps", 30.0, desc);
}

/**
 * @brief 声明控制参数 - 机器人控制算法相关参数
 *
 * 控制参数组包含:
 * - PID控制器的三个增益参数
 * - 控制输出的限制和约束
 * - 控制循环的执行频率
 */
void declare_control_parameters()
{
    auto desc = rcl_interfaces::msg::ParameterDescriptor();

    // PID控制器参数
    // 比例增益: 控制当前误差的响应强度
    desc.description = "比例增益";
    this->declare_parameter("control.pid.kp", 1.0, desc);

    // 积分增益: 控制累积误差的消除能力
    desc.description = "积分增益";
    this->declare_parameter("control.pid.ki", 0.1, desc);

    // 微分增益: 控制误差变化率的响应
    desc.description = "微分增益";
    this->declare_parameter("control.pid.kd", 0.05, desc);

    // 控制限制参数
    // 控制器输出的最大值, 防止执行器过载
    desc.description = "最大控制输出";
    this->declare_parameter("control.max_output", 1.0, desc);

    // 控制循环的执行频率
    desc.description = "控制频率 (Hz)";
    this->declare_parameter("control.frequency", 50, desc);
}

/**
 * @brief 从文件加载参数 - 读取YAML配置文件中的参数值
 *
 * 该函数实现了从外部配置文件加载参数的功能:
 * 1. 检查配置文件是否存在
 * 2. 使用YAML解析器读取文件内容
 * 3. 递归处理嵌套的参数结构
 * 4. 将配置值应用到相应的ROS参数
 *
 * 这种方式的优势:
 * - 参数配置与代码分离

```

```

* - 支持复杂的参数结构
* - 便于不同环境的配置管理
*/
void load_parameters_from_file()
{
    // 尝试从配置文件加载参数
    // 配置文件路径: 相对于工作空间的config目录
    std::string config_file = "config/robot_params.yaml";

    // 检查配置文件是否存在
    if (std::filesystem::exists(config_file)) {
        try {
            // 使用YAML-CPP库解析配置文件
            YAML::Node config = YAML::LoadFile(config_file);
            load_yaml_parameters(config);
            RCLCPP_INFO(this->get_logger(), "从文件加载参数: %s",
config_file.c_str());
        } catch (const std::exception& e) {
            // 处理文件解析错误
            RCLCPP_WARN(this->get_logger(), "加载配置文件失败: %s", e.what());
        }
    }
}

/**
 * @brief 递归加载YAML参数 - 处理嵌套的参数结构
 *
 * @param node YAML节点对象
 * @param prefix 参数名称前缀, 用于构建完整的参数路径
 *
 * 该函数递归遍历YAML结构:
 * 1. 对于每个键值对, 构建完整的参数名称
 * 2. 如果值是映射类型, 则递归处理子节点
 * 3. 如果值是标量类型, 则设置对应的ROS参数
 *
 * 支持的YAML结构示例:
 * navigation:
 *   max_speed: 1.0
 *   planning:
 *     algorithm: "A_star"
 */
void load_yaml_parameters(const YAML::Node& node, const std::string& prefix =
"")
{
    // 遍历YAML节点中的所有键值对
    for (auto it = node.begin(); it != node.end(); ++it) {
        // 构建完整的参数名称 (带命名空间前缀)
        std::string key = prefix.empty() ? it->first.as<std::string>() :
            prefix + "." + it->first.as<std::string>();

        if (it->second.IsMap()) {
            // 递归处理嵌套参数

```

```

        // 如果当前值是一个映射（子对象），则递归进入
        load_yaml_parameters(it->second, key);
    } else {
        // 设置参数值
        // 如果当前值是标量，则尝试设置对应的ROS参数
        try {
            if (it->second.IsScalar()) {
                // 检查参数是否已在ROS中声明
                if (this->has_parameter(key)) {
                    // 获取参数类型并设置相应的值
                    auto param_type = this->get_parameter(key).get_type();
                    set_parameter_from_yaml(key, it->second, param_type);
                }
            }
        } catch (const std::exception& e) {
            RCLCPP_WARN(this->get_logger(), "设置参数 %s 失败: %s",
key.c_str(), e.what());
        }
    }
}

/**
 * @brief 从YAML设置参数值 - 根据参数类型转换并设置值
 *
 * @param name 参数名称
 * @param value YAML值节点
 * @param type ROS参数类型
 *
 * 该函数根据ROS参数的声明类型，将YAML中的值转换为正确的类型：
 * - 布尔类型: true/false
 * - 整数类型: int64_t
 * - 浮点类型: double
 * - 字符串类型: string
 */
void set_parameter_from_yaml(const std::string& name, const YAML::Node& value,
                             rclcpp::ParameterType type)
{
    // 根据参数类型进行相应的类型转换和设置
    switch (type) {
        case rclcpp::ParameterType::PARAMETER_BOOL:
            // 布尔参数: 将YAML布尔值转换为ROS布尔参数
            this->set_parameter(rclcpp::Parameter(name, value.as<bool>()));
            break;
        case rclcpp::ParameterType::PARAMETER_INTEGER:
            // 整数参数: 将YAML数值转换为ROS整数参数
            this->set_parameter(rclcpp::Parameter(name, value.as<int64_t>()));
            break;
        case rclcpp::ParameterType::PARAMETER_DOUBLE:
            // 浮点参数: 将YAML数值转换为ROS浮点参数
            this->set_parameter(rclcpp::Parameter(name, value.as<double>()));
            break;
    }
}

```

```

        case rclcpp::ParameterType::PARAMETER_STRING:
            // 字符串参数: 将YAML字符串转换为ROS字符串参数
            this->set_parameter(rclcpp::Parameter(name, value.as<std::string>
    ));

            break;
        default:
            // 不支持的参数类型
            RCLCPP_WARN(this->get_logger(), "不支持的参数类型: %s", name.c_str());
            break;
    }
}

/**
 * @brief 设置参数验证 - 建立参数合法性检查机制
 *
 * 参数验证是高级参数管理的重要功能:
 * - 确保参数值在合理范围内
 * - 防止错误配置导致系统故障
 * - 提供详细的错误信息
 * - 支持复杂的业务逻辑验证
 */
void setup_parameter_validation()
{
    // 设置参数验证回调
    // 当有参数被修改时, 会先调用验证函数
    validation_callback_handle_ = this->add_on_set_parameters_callback(
        std::bind(&AdvancedParameterNode::validate_parameters, this,
std::placeholders::_1));
}

/**
 * @brief 参数验证回调函数 - 验证参数变更的合法性
 *
 * @param parameters 待验证的参数列表
 * @return SetParametersResult 验证结果, 包含成功状态和错误信息
 *
 * 验证流程:
 * 1. 遍历所有待修改的参数
 * 2. 根据参数组调用相应的验证函数
 * 3. 如果任何参数验证失败, 拒绝整个修改操作
 * 4. 返回详细的验证结果和错误信息
 */
rcl_interfaces::msg::SetParametersResult validate_parameters(
    const std::vector<rclcpp::Parameter> & parameters)
{
    rcl_interfaces::msg::SetParametersResult result;
    result.successful = true; // 默认验证成功

    // 遍历所有待验证的参数
    for (const auto & param : parameters) {
        std::string name = param.get_name();

```

```

// 验证导航参数
// 检查导航相关参数的值是否在合理范围内
if (name.find("navigation.") == 0) {
    if (!validate_navigation_parameter(param)) {
        result.successful = false;
        result.reason = "导航参数验证失败: " + name;
        return result;
    }
}
// 验证传感器参数
// 检查传感器配置的合理性
else if (name.find("sensors.") == 0) {
    if (!validate_sensor_parameter(param)) {
        result.successful = false;
        result.reason = "传感器参数验证失败: " + name;
        return result;
    }
}
// 验证控制参数
// 检查控制器参数的稳定性和安全性
else if (name.find("control.") == 0) {
    if (!validate_control_parameter(param)) {
        result.successful = false;
        result.reason = "控制参数验证失败: " + name;
        return result;
    }
}

RCLCPP_INFO(this->get_logger(), "参数验证通过: %s", name.c_str());
}

return result;
}

/**
 * @brief 验证导航参数 - 检查导航相关参数的合理性
 *
 * @param param 待验证的导航参数
 * @return bool 验证结果: true表示合法, false表示非法
 *
 * 导航参数验证规则:
 * - 速度参数必须在物理可行范围内
 * - 分辨率参数必须为正值且不能过大
 * - 时间参数必须为正值
 */
bool validate_navigation_parameter(const rclcpp::Parameter & param)
{
    std::string name = param.get_name();

    if (name == "navigation.max_linear_velocity") {
        // 验证最大线速度: 0-5 m/s 是合理的机器人速度范围
        double value = param.as_double();
    }
}

```



```

        return value >= 0.0 && value <= 5.0;
    }
    else if (name == "navigation.max_angular_velocity") {
        // 验证最大角速度: 0- $\pi$  rad/s 是合理的转向速度范围
        double value = param.as_double();
        return value >= 0.0 && value <= 3.14;
    }
    else if (name == "navigation.planning_resolution") {
        // 验证规划分辨率: 必须为正值且不能太大 (影响性能)
        double value = param.as_double();
        return value > 0.0 && value <= 1.0;
    }
}

return true; // 其他导航参数默认通过验证
}

/**
 * @brief 验证传感器参数 - 检查传感器配置的合理性
 *
 * @param param 待验证的传感器参数
 * @return bool 验证结果: true表示合法, false表示非法
 *
 * 传感器参数验证规则:
 * - 距离范围必须为正值
 * - 图像分辨率必须在合理范围内
 * - 帧率必须在设备支持的范围内
 */
bool validate_sensor_parameter(const rclcpp::Parameter & param)
{
    std::string name = param.get_name();

    if (name == "sensors.lidar.min_range" || name == "sensors.lidar.max_range")
    {
        // 验证激光雷达检测距离: 必须为正值
        double value = param.as_double();
        return value > 0.0;
    }
    else if (name == "sensors.camera.width" || name == "sensors.camera.height")
    {
        // 验证摄像头分辨率: 1-4096像素是常见的分辨率范围
        int value = param.as_int();
        return value > 0 && value <= 4096;
    }
    else if (name == "sensors.camera.fps") {
        // 验证摄像头帧率: 1-120 fps是合理的帧率范围
        double value = param.as_double();
        return value > 0.0 && value <= 120.0;
    }
}

return true; // 其他传感器参数默认通过验证
}

```

```

/**
 * @brief 验证控制参数 - 检查控制器参数的稳定性
 *
 * @param param 待验证的控制参数
 * @return bool 验证结果: true表示合法, false表示非法
 *
 * 控制参数验证规则:
 * - PID参数必须非负 (负值可能导致系统不稳定)
 * - 控制频率必须在合理范围内
 * - 输出限制必须为正值
 */
bool validate_control_parameter(const rclcpp::Parameter & param)
{
    std::string name = param.get_name();

    if (name.find("control.pid.") == 0) {
        // 验证PID参数: 不能为负值 (可能导致控制系统不稳定)
        double value = param.as_double();
        return value >= 0.0; // PID参数不能为负
    }
    else if (name == "control.frequency") {
        // 验证控制频率: 1-1000 Hz是合理的控制频率范围
        int value = param.as_int();
        return value > 0 && value <= 1000;
    }

    return true; // 其他控制参数默认通过验证
}

/**
 * @brief 设置参数监听 - 建立参数事件监控机制
 *
 * 参数监听功能允许节点:
 * - 监控系统中所有节点的参数变更
 * - 响应其他节点的参数修改
 * - 实现节点间的参数同步
 * - 记录参数变更历史
 */
void setup_parameter_monitoring()
{
    // 订阅参数事件话题
    // ROS2系统会在 /parameter_events 话题上发布所有参数变更事件
    parameter_event_subscriber_ = this->create_subscription<rcl_interfaces::msg::ParameterEvent>(
        "/parameter_events", 10,
        std::bind(&AdvancedParameterNode::on_parameter_event, this,
std::placeholders::_1));
}

/**
 * @brief 参数事件回调函数 - 处理系统中的参数变更事件
 *

```

```

* @param event 参数事件消息，包含变更的参数信息
*
* 事件处理功能：
* 1. 过滤本节点相关的参数事件
* 2. 记录参数变更日志
* 3. 触发相应的响应动作
* 4. 维护参数变更历史
*/
void on_parameter_event(const rcl_interfaces::msg::ParameterEvent::SharedPtr
event)
{
    // 监听系统中的参数变更事件
    // 只处理本节点的参数变更事件
    if (event->node == this->get_fully_qualified_name()) {
        // 遍历所有发生变更的参数
        for (const auto & changed_param : event->changed_parameters) {
            RCLCPP_INFO(this->get_logger(),
                "参数已变更：%s (类型：%d)",
                changed_param.name.c_str(),
                changed_param.value.type);
        }
    }
}

public:
/**
* @brief 保存参数到文件 - 将当前参数导出为YAML配置文件
*
* @param filename 保存的文件名
*
* 该功能允许：
* 1. 将运行时调整的参数保存为配置文件
* 2. 创建不同场景的参数配置
* 3. 备份当前的参数设置
* 4. 在不同系统间迁移配置
*/
void save_parameters_to_file(const std::string& filename)
{
    YAML::Node config;

    // 获取所有参数
    // 获取节点中声明的所有参数名称列表
    auto param_names = this->list_parameters({}, 0).names;

    // 遍历所有参数并添加到YAML配置中
    for (const auto& name : param_names) {
        auto param = this->get_parameter(name);
        set_yaml_parameter(config, name, param);
    }

    // 保存到文件
    std::ofstream fout(filename);

```

```

    fout << config;
    fout.close();

    RCLCPP_INFO(this->get_logger(), "参数已保存到: %s", filename.c_str());
}

```

private:

```

/**
 * @brief 设置YAML参数值 - 将ROS参数转换为YAML格式
 *
 * @param node YAML节点对象
 * @param name 参数名称（可能包含点号分隔的路径）
 * @param param ROS参数对象
 *
 * 该函数将带有命名空间的参数名称转换为嵌套的YAML结构：
 * 例如: "navigation.max_speed" -> navigation: { max_speed: 1.0 }
 */
void set_yaml_parameter(YAML::Node& node, const std::string& name,
                        const rclcpp::Parameter& param)
{
    // 将参数路径分解为嵌套结构
    // 按点号分割参数名称，构建嵌套的YAML结构
    std::vector<std::string> keys;
    std::istringstream iss(name);
    std::string key;

    // 分割参数名称: "navigation.max_speed" -> ["navigation", "max_speed"]
    while (std::getline(iss, key, '.')) {
        keys.push_back(key);
    }

    // 导航到正确的YAML节点位置
    YAML::Node current = node;
    for (size_t i = 0; i < keys.size() - 1; ++i) {
        current = current[keys[i]];
    }

    // 设置参数值
    // 根据ROS参数类型设置相应的YAML值
    switch (param.get_type()) {
        case rclcpp::ParameterType::PARAMETER_BOOL:
            current[keys.back()] = param.as_bool();
            break;
        case rclcpp::ParameterType::PARAMETER_INTEGER:
            current[keys.back()] = param.as_int();
            break;
        case rclcpp::ParameterType::PARAMETER_DOUBLE:
            current[keys.back()] = param.as_double();
            break;
        case rclcpp::ParameterType::PARAMETER_STRING:
            current[keys.back()] = param.as_string();
            break;
    }
}

```

```

        default:
            // 不支持的参数类型跳过
            break;
    }
}

// ===== 成员变量定义 =====

// 参数验证回调句柄
rclcpp::node_interfaces::OnSetParametersCallbackHandle::SharedPtr
validation_callback_handle_;

// 参数事件订阅者，用于监听系统中的参数变更事件
rclcpp::Subscription<rcl_interfaces::msg::ParameterEvent>::SharedPtr
parameter_event_subscriber_;
};

/**
 * @brief 主函数 - 高级参数节点程序入口点
 *
 * @param argc 命令行参数个数
 * @param argv 命令行参数数组
 * @return int 程序退出码
 *
 * 主函数执行流程：
 * 1. 初始化ROS2系统
 * 2. 创建AdvancedParameterNode节点实例
 * 3. 进入ROS2事件循环，处理参数事件和回调
 * 4. 节点关闭时清理资源
 */
int main(int argc, char ** argv)
{
    // 初始化ROS2系统
    rclcpp::init(argc, argv);

    // 创建高级参数节点实例
    auto node = std::make_shared<AdvancedParameterNode>();

    // 进入ROS2事件循环
    // 处理参数变更回调、事件监听和定时器等
    rclcpp::spin(node);

    // 清理ROS2系统资源
    rclcpp::shutdown();

    return 0; // 正常退出
}

```

基础参数客户端示例

在原有值上加0.1

src/parameter_client.cpp:

```
/**
 * @file parameter_demo/src/parameter_client.cpp
 * @brief ROS2 参数客户端演示节点 - 展示如何从外部访问和操作其他节点的参数
 *
 * 本文件演示了ROS2中参数客户端的使用方法，包括：
 * 1. 创建参数客户端连接到目标节点
 * 2. 列出目标节点的所有参数
 * 3. 获取特定参数的值
 * 4. 设置和修改参数
 * 5. 批量参数操作
 * 6. 参数值的格式化显示
 * 7. 错误处理和服务可用性检查
 *
 * 参数客户端是ROS2分布式系统中重要的组件，允许：
 * - 跨节点的参数访问和控制
 * - 运行时动态配置系统
 * - 集中化的参数管理
 * - 系统监控和调试
 */

#include <rcppcpp/rcppcpp.hpp> // ROS2 C++ 客户端库
#include <rcpp_interfaces/srv/get_parameters.hpp> // 获取参数服务接口
#include <rcpp_interfaces/srv/set_parameters.hpp> // 设置参数服务接口
#include <rcpp_interfaces/srv/list_parameters.hpp> // 列出参数服务接口

/**
 * @class ParameterClient
 * @brief 参数客户端类 - 继承自rcppcpp::Node
 *
 * 该类演示了如何创建参数客户端来访问和操作其他节点的参数：
 * - 同步参数客户端的创建和使用
 * - 参数服务的调用和错误处理
 * - 不同类型参数的读取和设置
 * - 参数值的类型转换和格式化
 * - 批量参数操作的实现
 *
 * 适用场景：
 * - 配置管理工具
 * - 系统监控程序
 * - 调试和测试工具
 * - 参数同步服务
 */
class ParameterClient : public rcppcpp::Node
{
public:
    /**
```

```

* @brief 构造函数 - 初始化参数客户端节点
*
* 在构造函数中完成以下操作：
* 1. 创建同步参数客户端，连接到目标节点
* 2. 等待目标节点的服务可用
* 3. 执行一系列参数操作演示
*
* 目标节点: "basic_parameter_node" - 之前创建的基础参数节点
*/
ParameterClient() : Node("parameter_client")
{
    // 创建参数客户端
    // SyncParametersClient提供同步的参数操作接口
    // 第二个参数指定目标节点名称
    param_client_ = std::make_shared<rclcpp::SyncParametersClient>(this,
"basic_parameter_node");

    // 等待目标节点可用
    // 在分布式系统中，目标节点可能尚未启动，需要等待其服务可用
    if (!param_client_>wait_for_service(std::chrono::seconds(5))) {
        RCLCPP_ERROR(this->get_logger(), "目标节点服务不可用");
        return;
    }

    RCLCPP_INFO(this->get_logger(), "参数客户端已启动");

    // 演示参数操作
    // 依次执行各种参数操作，展示客户端的完整功能
    demonstrate_parameter_operations();
}

private:
/**
* @brief 演示参数操作 - 执行一系列参数操作示例
*
* 该函数按逻辑顺序执行不同的参数操作：
* 1. 首先列出所有可用参数，了解目标节点的结构
* 2. 获取特定参数的当前值
* 3. 修改参数值，演示参数设置功能
* 4. 执行批量操作，展示复杂参数处理
*
* 这种顺序设计便于理解参数客户端的完整工作流程
*/
void demonstrate_parameter_operations()
{
    // 1. 列出所有参数
    // 首先了解目标节点有哪些参数可用
    list_all_parameters();

    // 2. 获取特定参数
    // 读取我们感兴趣的参数的当前值
    get_specific_parameters();
}

```

```

        // 3. 设置参数
        // 修改参数值，展示写操作
        set_parameters();

        // 4. 批量操作
        // 演示复杂的参数处理场景
        batch_operations();
    }

/**
 * @brief 列出所有参数 - 获取目标节点的所有参数列表
 *
 * 该函数演示如何：
 * 1. 调用list_parameters服务
 * 2. 处理返回的参数名称列表
 * 3. 格式化显示参数信息
 *
 * 这是参数探索的第一步，帮助了解节点的参数结构
 */
void list_all_parameters()
{
    RCLCPP_INFO(this->get_logger(), "\n=== 列出所有参数 ===");

    try {
        // 调用列出参数服务
        // 参数：前缀过滤器（空表示所有参数），深度限制（0表示无限制）
        auto result = param_client->list_parameters({}, 0);
        const auto& parameter_names = result.names;

        // 显示发现的参数数量和名称
        RCLCPP_INFO(this->get_logger(), "发现 %zu 个参数:",
parameter_names.size());
        for (const auto& name : parameter_names) {
            RCLCPP_INFO(this->get_logger(), " - %s", name.c_str());
        }
    } catch (const std::exception& e) {
        // 处理服务调用异常
        RCLCPP_ERROR(this->get_logger(), "列出参数失败: %s", e.what());
    }
}

/**
 * @brief 获取特定参数 - 读取指定参数的当前值
 *
 * 该函数演示如何：
 * 1. 指定要获取的参数名称列表
 * 2. 批量获取多个参数的值
 * 3. 处理不同类型的参数值
 * 4. 格式化显示参数信息
 *
 * 支持的参数类型包括布尔、整数、浮点、字符串和数组类型

```



```

*/
void get_specific_parameters()
{
    RCLCPP_INFO(this->get_logger(), "\n=== 获取特定参数 ===");

    // 定义要获取的参数名称列表
    // 选择不同类型的参数进行演示
    std::vector<std::string> param_names = {
        "debug_mode",      // 布尔类型
        "max_speed",       // 浮点类型
        "robot_name",      // 字符串类型
        "joint_positions"  // 浮点数组类型
    };

    try {
        // 批量获取参数值
        // 一次服务调用可以获取多个参数，提高效率
        auto parameters = param_client->get_parameters(param_names);

        // 遍历返回的参数，显示名称、值和类型信息
        for (const auto& param : parameters) {
            std::string value_str = parameter_value_to_string(param);
            RCLCPP_INFO(this->get_logger(),
                "%s = %s (类型: %s)",
                param.get_name().c_str(),
                value_str.c_str(),
                param.get_type_name().c_str());
        }
    } catch (const std::exception& e) {
        // 处理参数获取异常
        RCLCPP_ERROR(this->get_logger(), "获取参数失败: %s", e.what());
    }
}

/**
 * @brief 设置参数 - 修改目标节点的参数值
 *
 * 该函数演示如何：
 * 1. 创建要设置的参数对象
 * 2. 批量设置多个参数
 * 3. 检查设置操作的结果
 * 4. 处理设置失败的情况
 *
 * 参数设置可能失败的原因：
 * - 参数验证失败（值超出范围）
 * - 参数不存在
 * - 目标节点拒绝修改
 */
void set_parameters()
{
    RCLCPP_INFO(this->get_logger(), "\n=== 设置参数 ===");

```

```

// 创建要设置的参数列表
// 包含不同类型的参数，演示各种数据类型的设置
std::vector<rclcpp::Parameter> parameters_to_set = {
    rclcpp::Parameter("debug_mode", true), // 布尔参数:
启用调试模式
    rclcpp::Parameter("max_speed", 2.5), // 浮点参数:
设置新的最大速度
    rclcpp::Parameter("robot_name", std::string("super_robot")), // 字符串参
数: 更改机器人名称
    rclcpp::Parameter("loop_rate", 20) // 整数参数:
调整循环频率
};

try {
    // 批量设置参数
    // 返回每个参数的设置结果
    auto results = param_client->set_parameters(parameters_to_set);

    // 检查每个参数的设置结果
    for (size_t i = 0; i < results.size(); ++i) {
        const auto& param = parameters_to_set[i];
        const auto& result = results[i];

        if (result.successful) {
            // 设置成功
            RCLCPP_INFO(this->get_logger(),
                "✅ 成功设置 %s = %s",
                param.get_name().c_str(),
                parameter_value_to_string(param).c_str());
        } else {
            // 设置失败，显示失败原因
            RCLCPP_WARN(this->get_logger(),
                "❌ 设置失败 %s: %s",
                param.get_name().c_str(),
                result.reason.c_str());
        }
    }
} catch (const std::exception& e) {
    // 处理设置参数异常
    RCLCPP_ERROR(this->get_logger(), "设置参数失败: %s", e.what());
}

}

/**
 * @brief 批量操作 - 演示复杂的参数处理场景
 *
 * 该函数展示了实际应用中常见的参数操作模式:
 * 1. 获取数组类型参数的当前值
 * 2. 对参数值进行计算处理
 * 3. 将修改后的值写回目标节点
 * 4. 验证操作结果
 *
 */

```

```

* 以关节位置参数为例，模拟了机器人关节控制的典型场景：
* - 读取当前关节位置
* - 计算新的目标位置
* - 更新关节位置参数
*/
void batch_operations()
{
    RCLCPP_INFO(this->get_logger(), "\n=== 批量操作 ===");

    // 批量获取关节相关参数
    // 在实际应用中，可能需要处理多个相关联的参数
    std::vector<std::string> joint_params = {"joint_positions"};

    try {
        // 获取当前关节位置参数
        auto joint_parameters = param_client->get_parameters(joint_params);

        if (!joint_parameters.empty()) {
            // 提取关节位置数组
            auto joint_positions = joint_parameters[0].as_double_array();

            // 显示当前关节位置
            RCLCPP_INFO(this->get_logger(), "当前关节位置:");
            for (size_t i = 0; i < joint_positions.size(); ++i) {
                RCLCPP_INFO(this->get_logger(), "  关节 %zu: %.3f", i,
joint_positions[i]);
            }

            // 修改关节位置
            // 模拟关节运动：每个关节增加0.1弧度
            for (auto& pos : joint_positions) {
                pos += 0.1; // 每个关节增加0.1弧度
            }

            // 设置新的关节位置
            auto new_joint_param = rclcpp::Parameter("joint_positions",
joint_positions);
            auto result = param_client->set_parameters({new_joint_param});

            // 检查更新结果
            if (!result.empty() && result[0].successful) {
                RCLCPP_INFO(this->get_logger(), "✅ 关节位置已更新");

                // 显示更新后的关节位置
                RCLCPP_INFO(this->get_logger(), "新的关节位置:");
                for (size_t i = 0; i < joint_positions.size(); ++i) {
                    RCLCPP_INFO(this->get_logger(), "  关节 %zu: %.3f", i,
joint_positions[i]);
                }
            } else {
                RCLCPP_WARN(this->get_logger(), "❌ 关节位置更新失败");
            }
        }
    }
}

```

```

    }
} catch (const std::exception& e) {
    // 处理批量操作异常
    RCLCPP_ERROR(this->get_logger(), "批量操作失败: %s", e.what());
}
}

/**
 * @brief 参数值转字符串 - 将不同类型的参数值格式化为可读字符串
 *
 * @param param ROS参数对象
 * @return std::string 格式化后的参数值字符串
 *
 * 该函数是工具函数，用于：
 * 1. 根据参数类型选择合适的格式化方式
 * 2. 处理各种数据类型的显示格式
 * 3. 为数组类型提供可读的格式
 * 4. 为字符串类型添加引号标识
 *
 * 支持的参数类型：
 * - PARAMETER_BOOL: true/false
 * - PARAMETER_INTEGER: 整数值
 * - PARAMETER_DOUBLE: 浮点数值
 * - PARAMETER_STRING: 带引号的字符串
 * - PARAMETER_DOUBLE_ARRAY: [val1, val2, ...]格式的数组
 * - PARAMETER_STRING_ARRAY: ["str1", "str2", ...]格式的数组
 */
std::string parameter_value_to_string(const rclcpp::Parameter& param)
{
    // 根据参数类型进行不同的格式化处理
    switch (param.get_type()) {
        case rclcpp::ParameterType::PARAMETER_BOOL:
            // 布尔类型：转换为true/false字符串
            return param.as_bool() ? "true" : "false";

        case rclcpp::ParameterType::PARAMETER_INTEGER:
            // 整数类型：直接转换为字符串
            return std::to_string(param.as_int());

        case rclcpp::ParameterType::PARAMETER_DOUBLE:
            // 浮点类型：转换为字符串（默认精度）
            return std::to_string(param.as_double());

        case rclcpp::ParameterType::PARAMETER_STRING:
            // 字符串类型：添加引号表示这是字符串值
            return "\"" + param.as_string() + "\"";

        case rclcpp::ParameterType::PARAMETER_DOUBLE_ARRAY: {
            // 浮点数组类型：格式化为[val1, val2, ...]形式
            auto values = param.as_double_array();
            std::string result = "[";
            for (size_t i = 0; i < values.size(); ++i) {

```

```

        result += std::to_string(values[i]);
        if (i < values.size() - 1) result += ", ";
    }
    result += "];";
    return result;
}

case rclcpp::ParameterType::PARAMETER_STRING_ARRAY: {
    // 字符串数组类型: 格式化为["str1", "str2", ...]形式
    auto values = param.as_string_array();
    std::string result = "[";
    for (size_t i = 0; i < values.size(); ++i) {
        result += "\"" + values[i] + "\"";
        if (i < values.size() - 1) result += ", ";
    }
    result += "];";
    return result;
}

default:
    // 未知类型: 返回占位符
    return "unknown";
}
}

// ===== 成员变量定义 =====

/**
 * @brief 同步参数客户端对象
 *
 * SyncParametersClient提供同步的参数操作接口:
 * - 阻塞式服务调用, 等待操作完成
 * - 简化的API, 无需手动处理异步回调
 * - 适合简单的参数操作和调试工具
 *
 * 与AsyncParametersClient的区别:
 * - SyncParametersClient: 同步操作, 简单易用
 * - AsyncParametersClient: 异步操作, 适合高性能场景
 */
std::shared_ptr<rclcpp::SyncParametersClient> param_client_;
};

/**
 * @brief 主函数 - 参数客户端程序入口点
 * 主函数执行流程:
 * 1. 初始化ROS2系统
 * 2. 创建ParameterClient节点实例
 * 3. 参数操作在构造函数中自动完成
 * 4. 立即清理资源并退出
 *
 * 注意: 本示例是一次性执行的工具, 不需要进入spin循环
 * 在实际应用中, 参数客户端通常作为长期运行服务的一部分

```

```

*/
int main(int argc, char ** argv)
{
    // 初始化ROS2系统
    rclcpp::init(argc, argv);

    // 创建参数客户端节点
    // 所有参数操作在构造函数中完成
    auto node = std::make_shared<ParameterClient>();

    // 清理ROS2系统资源
    // 由于这是一次性工具，操作完成后直接退出
    rclcpp::shutdown();

    return 0; // 正常退出
}

```

高级参数客户端示例

```

/**
 * @file parameter_demo/src/advanced_parameter_client.cpp
 * @brief ROS2 高级参数客户端 - 测试高级参数节点的所有功能
 *
 * 本文件是专门为测试advanced_parameter_node.cpp而设计的客户端，包括：
 * 1. 测试分组参数的读取和设置
 * 2. 验证参数验证机制的工作
 * 3. 测试参数约束和边界条件
 * 4. 验证嵌套参数结构的处理
 * 5. 测试参数事件监听功能
 * 6. 验证参数保存和导出功能
 * 7. 压力测试和错误处理验证
 *
 * 这个客户端提供了全面的测试套件，确保高级参数节点的所有功能正常工作
 */

#include <rclcpp/rclcpp.hpp> // ROS2 C++ 客户端库
#include <rcl_interfaces/srv/get_parameters.hpp> // 获取参数服务接口
#include <rcl_interfaces/srv/set_parameters.hpp> // 设置参数服务接口
#include <rcl_interfaces/srv/list_parameters.hpp> // 列出参数服务接口
#include <rcl_interfaces/msg/parameter_event.hpp> // 参数事件消息类型
#include <chrono> // 时间处理库
#include <thread> // 线程库

/**
 * @class AdvancedParameterClient
 * @brief 高级参数客户端类 - 专门测试高级参数节点的功能
 *
 * 该类提供了完整的测试套件，用于验证高级参数节点的各项功能：
 * - 参数组管理：测试导航、传感器、控制参数组
 * - 参数验证：测试约束检查和错误处理
 * - 边界测试：验证参数范围限制

```

```

* - 类型测试：确保不同参数类型的正确处理
* - 事件监听：验证参数变更事件的产生和处理
* - 批量操作：测试复杂的参数操作场景
*/
class AdvancedParameterClient : public rclcpp::Node
{
public:
    /**
     * @brief 构造函数 - 初始化高级参数客户端
     *
     * 创建参数客户端连接到高级参数节点，并执行完整的测试套件
     */
    AdvancedParameterClient() : Node("advanced_parameter_client")
    {
        // 创建参数客户端，连接到高级参数节点
        param_client_ = std::make_shared<rclcpp::SyncParametersClient>(this,
"advanced_parameter_node");

        // 等待目标节点可用
        if (!param_client_>wait_for_service(std::chrono::seconds(10))) {
            RCLCPP_ERROR(this->get_logger(), "高级参数节点服务不可用，请先启动
advanced_parameter_node");
            return;
        }

        RCLCPP_INFO(this->get_logger(), "🚀 高级参数客户端已启动，开始测试...");

        // 执行全面的测试套件
        run_comprehensive_tests();

        RCLCPP_INFO(this->get_logger(), "✅ 所有测试完成!");
    }

private:
    /**
     * @brief 运行综合测试套件 - 按顺序执行所有测试用例
     *
     * 测试顺序设计：
     * 1. 基础功能测试（参数列表、读取）
     * 2. 参数组结构测试
     * 3. 参数设置和验证测试
     * 4. 边界条件和错误处理测试
     * 5. 高级功能测试（批量操作、事件监听）
     */
    void run_comprehensive_tests()
    {
        // 1. 基础功能测试
        test_basic_functionality();

        // 2. 参数组测试
        test_parameter_groups();
    }
}

```

```

// 3. 参数验证测试
test_parameter_validation();

// 4. 边界条件测试
test_boundary_conditions();

// 5. 批量操作测试
test_batch_operations();

// 6. 类型转换测试
test_parameter_types();

// 7. 错误处理测试
test_error_handling();
}

/**
 * @brief 测试基础功能 - 验证参数列表和基本读取功能
 *
 * 测试内容:
 * - 参数列表的获取
 * - 基本参数的读取
 * - 参数名称的验证
 */
void test_basic_functionality()
{
    RCLCPP_INFO(this->get_logger(), "\n📖 === 测试1: 基础功能测试 ===");

    try {
        // 获取所有参数列表
        auto result = param_client->list_parameters({}, 0);
        const auto& parameter_names = result.names;

        RCLCPP_INFO(this->get_logger(), "✅ 发现 %zu 个参数",
parameter_names.size());

        // 验证预期的参数组是否存在
        std::vector<std::string> expected_prefixes = {
            "navigation.", "sensors.", "control."
        };

        for (const auto& prefix : expected_prefixes) {
            bool found = false;
            for (const auto& name : parameter_names) {
                if (name.find(prefix) == 0) {
                    found = true;
                    break;
                }
            }

            if (found) {

```



```

        RCLCPP_INFO(this->get_logger(), "✅ 参数组 '%s' 存在",
prefix.c_str());
    } else {
        RCLCPP_WARN(this->get_logger(), "❌ 参数组 '%s' 不存在",
prefix.c_str());
    }
}

} catch (const std::exception& e) {
    RCLCPP_ERROR(this->get_logger(), "❌ 基础功能测试失败: %s", e.what());
}
}

/**
 * @brief 测试参数组 - 验证分组参数的组织和访问
 *
 * 测试内容:
 * - 导航参数组的完整性
 * - 传感器参数组的结构
 * - 控制参数组的配置
 */
void test_parameter_groups()
{
    RCLCPP_INFO(this->get_logger(), "\n🌸 === 测试2: 参数组测试 ===");

    // 测试导航参数组
    test_navigation_parameters();

    // 测试传感器参数组
    test_sensor_parameters();

    // 测试控制参数组
    test_control_parameters();
}

/**
 * @brief 测试导航参数 - 验证导航相关参数的完整性
 */
void test_navigation_parameters()
{
    RCLCPP_INFO(this->get_logger(), "🔧 测试导航参数组...");

    std::vector<std::string> nav_params = {
        "navigation.planning_algorithm",
        "navigation.max_planning_time",
        "navigation.planning_resolution",
        "navigation.use_obstacles",
        "navigation.max_linear_velocity",
        "navigation.max_angular_velocity"
    };

    try {

```

```

        auto parameters = param_client_>get_parameters(nav_params);

        for (size_t i = 0; i < parameters.size(); ++i) {
            const auto& param = parameters[i];
            RCLCPP_INFO(this->get_logger(),
                " 🚀 %s = %s",
                param.get_name().c_str(),
                parameter_value_to_string(param).c_str());
        }

        RCLCPP_INFO(this->get_logger(), "✅ 导航参数组测试通过");

    } catch (const std::exception& e) {
        RCLCPP_ERROR(this->get_logger(), "❌ 导航参数组测试失败: %s", e.what());
    }
}

/**
 * @brief 测试传感器参数 - 验证传感器配置参数
 */
void test_sensor_parameters()
{
    RCLCPP_INFO(this->get_logger(), "🔔 测试传感器参数组...");

    std::vector<std::string> sensor_params = {
        "sensors.lidar.topic",
        "sensors.lidar.frame_id",
        "sensors.lidar.min_range",
        "sensors.lidar.max_range",
        "sensors.camera.topic",
        "sensors.camera.width",
        "sensors.camera.height",
        "sensors.camera.fps"
    };

    try {
        auto parameters = param_client_>get_parameters(sensor_params);

        for (const auto& param : parameters) {
            RCLCPP_INFO(this->get_logger(),
                " 📡 %s = %s",
                param.get_name().c_str(),
                parameter_value_to_string(param).c_str());
        }

        RCLCPP_INFO(this->get_logger(), "✅ 传感器参数组测试通过");

    } catch (const std::exception& e) {
        RCLCPP_ERROR(this->get_logger(), "❌ 传感器参数组测试失败: %s", e.what());
    }
}

```

```

/**
 * @brief 测试控制参数 - 验证控制器参数配置
 */
void test_control_parameters()
{
    RCLCPP_INFO(this->get_logger(), "🎮 测试控制参数组...");

    std::vector<std::string> control_params = {
        "control.pid.kp",
        "control.pid.ki",
        "control.pid.kd",
        "control.max_output",
        "control.frequency"
    };

    try {
        auto parameters = param_client->get_parameters(control_params);

        for (const auto& param : parameters) {
            RCLCPP_INFO(this->get_logger(),
                " 🎮 %s = %s",
                param.get_name().c_str(),
                parameter_value_to_string(param).c_str());
        }

        RCLCPP_INFO(this->get_logger(), "✅ 控制参数组测试通过");

    } catch (const std::exception& e) {
        RCLCPP_ERROR(this->get_logger(), "❌ 控制参数组测试失败: %s", e.what());
    }
}

/**
 * @brief 测试参数验证 - 验证参数约束和验证机制
 *
 * 测试内容:
 * - 有效参数值的设置
 * - 无效参数值的拒绝
 * - 验证错误信息的正确性
 */
void test_parameter_validation()
{
    RCLCPP_INFO(this->get_logger(), "\n🔍 === 测试3: 参数验证测试 ===");

    // 测试有效参数设置
    test_valid_parameter_settings();

    // 测试无效参数设置
    test_invalid_parameter_settings();
}

/**

```

```

* @brief 测试有效参数设置 - 验证正常范围内的参数可以成功设置
*/
void test_valid_parameter_settings()
{
    RCLCPP_INFO(this->get_logger(), "✅ 测试有效参数设置...");

    std::vector<rclcpp::Parameter> valid_params = {
        rclcpp::Parameter("navigation.max_linear_velocity", 2.0), // 在有效范围内
        rclcpp::Parameter("navigation.max_angular_velocity", 1.5), // 在有效范围内
        rclcpp::Parameter("control.pid.kp", 1.5), // 非负值
        rclcpp::Parameter("sensors.camera.fps", 60.0), // 合理帧率
        rclcpp::Parameter("control.frequency", 100) // 合理频率
    };

    try {
        auto results = param_client->set_parameters(valid_params);

        for (size_t i = 0; i < results.size(); ++i) {
            const auto& param = valid_params[i];
            const auto& result = results[i];

            if (result.successful) {
                RCLCPP_INFO(this->get_logger(),
                    "✅ %s = %s (验证通过)",
                    param.get_name().c_str(),
                    parameter_value_to_string(param).c_str());
            } else {
                RCLCPP_WARN(this->get_logger(),
                    "❌ %s 设置失败: %s",
                    param.get_name().c_str(),
                    result.reason.c_str());
            }
        }
    } catch (const std::exception& e) {
        RCLCPP_ERROR(this->get_logger(), "❌ 有效参数测试失败: %s", e.what());
    }
}

/**
* @brief 测试无效参数设置 - 验证超出范围的参数被正确拒绝
*/
void test_invalid_parameter_settings()
{
    RCLCPP_INFO(this->get_logger(), "❌ 测试无效参数设置（应该被拒绝）...");

    std::vector<rclcpp::Parameter> invalid_params = {
        rclcpp::Parameter("navigation.max_linear_velocity", 10.0), // 超出范围
        rclcpp::Parameter("navigation.max_angular_velocity", 5.0), // 超出范围
    };
}

```

(>5.0)

(> π)

```

    rclcpp::Parameter("control.pid.kp", -1.0),           // 负值（不允许）
    rclcpp::Parameter("sensors.camera.fps", 200.0),     // 超出范围(>120)
    rclcpp::Parameter("control.frequency", 2000)        // 超出范围(>1000)
};

try {
    auto results = param_client->set_parameters(invalid_params);

    for (size_t i = 0; i < results.size(); ++i) {
        const auto& param = invalid_params[i];
        const auto& result = results[i];

        if (!result.successful) {
            RCLCPP_INFO(this->get_logger(),
                " ✓ %s 正确被拒绝: %s",
                param.get_name().c_str(),
                result.reason.c_str());
        } else {
            RCLCPP_WARN(this->get_logger(),
                " ✗ %s 应该被拒绝但被接受了!",
                param.get_name().c_str());
        }
    }

} catch (const std::exception& e) {
    RCLCPP_ERROR(this->get_logger(), " ✗ 无效参数测试失败: %s", e.what());
}

/**
 * @brief 测试边界条件 - 验证参数边界值的处理
 */
void test_boundary_conditions()
{
    RCLCPP_INFO(this->get_logger(), "\n🌀 === 测试4: 边界条件测试 ===");

    // 测试边界值
    std::vector<std::pair<rclcpp::Parameter, bool>> boundary_tests = {
        {rclcpp::Parameter("navigation.max_linear_velocity", 0.0), true}, // 下边界
        {rclcpp::Parameter("navigation.max_linear_velocity", 5.0), true}, // 上边界
        {rclcpp::Parameter("navigation.max_angular_velocity", 0.0), true}, // 下边界
        {rclcpp::Parameter("navigation.max_angular_velocity", 3.14), true}, // 上边界
        {rclcpp::Parameter("control.frequency", 1), true}, // 下边界
    };
}

```

上边界

```
{rclcpp::Parameter("control.frequency", 1000), true}, //
};

for (const auto& test : boundary_tests) {
    const auto& param = test.first;
    bool should_succeed = test.second;

    try {
        auto results = param_client->set_parameters({param});

        if (!results.empty()) {
            bool succeeded = results[0].successful;

            if (succeeded == should_succeed) {
                RCLCPP_INFO(this->get_logger(),
                    "  边界测试通过: %s = %s",
                    param.get_name().c_str(),
                    parameter_value_to_string(param).c_str());
            } else {
                RCLCPP_WARN(this->get_logger(),
                    "  边界测试失败: %s = %s (期望%s, 实际%s)",
                    param.get_name().c_str(),
                    parameter_value_to_string(param).c_str(),
                    should_succeed ? "成功" : "失败",
                    succeeded ? "成功" : "失败");
            }
        }
    }

    catch (const std::exception& e) {
        RCLCPP_ERROR(this->get_logger(), "  边界测试异常: %s", e.what());
    }
}

/**
 * @brief 测试批量操作 - 验证复杂的批量参数处理
 */
void test_batch_operations()
{
    RCLCPP_INFO(this->get_logger(), "\n === 测试5: 批量操作测试 ===");

    try {
        // 批量获取所有PID参数
        std::vector<std::string> pid_params = {
            "control.pid.kp", "control.pid.ki", "control.pid.kd"
        };

        auto original_params = param_client->get_parameters(pid_params);
        RCLCPP_INFO(this->get_logger(), " 获取原始PID参数:");
        for (const auto& param : original_params) {
            RCLCPP_INFO(this->get_logger(),
```

```

        " 🚩 %s = %s",
        param.get_name().c_str(),
        parameter_value_to_string(param).c_str());
    }

    // 批量修改PID参数
    std::vector<rclcpp::Parameter> new_pid_params = {
        rclcpp::Parameter("control.pid.kp", 2.0),
        rclcpp::Parameter("control.pid.ki", 0.2),
        rclcpp::Parameter("control.pid.kd", 0.1)
    };

    auto results = param_client->set_parameters(new_pid_params);

    RCLCPP_INFO(this->get_logger(), " 🚩 批量设置PID参数结果:");
    for (size_t i = 0; i < results.size(); ++i) {
        const auto& param = new_pid_params[i];
        const auto& result = results[i];

        if (result.successful) {
            RCLCPP_INFO(this->get_logger(),
                " ✅ %s = %s",
                param.get_name().c_str(),
                parameter_value_to_string(param).c_str());
        } else {
            RCLCPP_WARN(this->get_logger(),
                " ❌ %s 设置失败: %s",
                param.get_name().c_str(),
                result.reason.c_str());
        }
    }

    // 验证设置结果
    auto updated_params = param_client->get_parameters(pid_params);
    RCLCPP_INFO(this->get_logger(), " 🔍 验证更新后的PID参数:");
    for (const auto& param : updated_params) {
        RCLCPP_INFO(this->get_logger(),
            " 🚩 %s = %s",
            param.get_name().c_str(),
            parameter_value_to_string(param).c_str());
    }

} catch (const std::exception& e) {
    RCLCPP_ERROR(this->get_logger(), " ❌ 批量操作测试失败: %s", e.what());
}

/**
 * @brief 测试参数类型 - 验证不同数据类型的处理
 */
void test_parameter_types()
{

```

```

RCLCPP_INFO(this->get_logger(), "\n📄 === 测试6: 参数类型测试 ===");

// 测试不同类型的参数
std::vector<std::string> type_test_params = {
    "navigation.use_obstacles",           // 布尔类型
    "control.frequency",                 // 整数类型
    "navigation.max_linear_velocity",     // 浮点类型
    "navigation.planning_algorithm",      // 字符串类型
};

try {
    auto parameters = param_client->get_parameters(type_test_params);

    for (const auto& param : parameters) {
        RCLCPP_INFO(this->get_logger(),
            "📄 %s (%s) = %s",
            param.get_name().c_str(),
            param.get_type_name().c_str(),
            parameter_value_to_string(param).c_str());
    }

    RCLCPP_INFO(this->get_logger(), "✅ 参数类型测试通过");

} catch (const std::exception& e) {
    RCLCPP_ERROR(this->get_logger(), "❌ 参数类型测试失败: %s", e.what());
}

/**
 * @brief 测试错误处理 - 验证各种错误情况的处理
 */
void test_error_handling()
{
    RCLCPP_INFO(this->get_logger(), "\n⚠️ === 测试7: 错误处理测试 ===");

    // 测试不存在的参数
    try {
        std::vector<std::string> nonexistent_params = {"nonexistent.parameter"};
        auto parameters = param_client->get_parameters(nonexistent_params);

        RCLCPP_INFO(this->get_logger(), "📄 不存在参数的处理结果:");
        for (const auto& param : parameters) {
            RCLCPP_INFO(this->get_logger(),
                "🔪 %s (%s) = %s",
                param.get_name().c_str(),
                param.get_type_name().c_str(),
                parameter_value_to_string(param).c_str());
        }

    } catch (const std::exception& e) {
        RCLCPP_INFO(this->get_logger(), "✅ 正确捕获了不存在参数的异常: %s",
e.what());
    }
}

```



```

    }

    // 测试设置不存在的参数
    try {
        std::vector<rcpp::Parameter> invalid_params = {
            rcpp::Parameter("nonexistent.param", 123)
        };

        auto results = param_client->set_parameters(invalid_params);

        if (!results.empty() && !results[0].successful) {
            RCLCPP_INFO(this->get_logger(), "✅ 正确拒绝了不存在的参数设置: %s",
                results[0].reason.c_str());
        }

        } catch (const std::exception& e) {
            RCLCPP_INFO(this->get_logger(), "✅ 正确捕获了设置不存在参数的异常: %s",
                e.what());
        }
    }

    /**
     * @brief 参数值转字符串 - 将参数值格式化为可读字符串
     */
    std::string parameter_value_to_string(const rcpp::Parameter& param)
    {
        switch (param.get_type()) {
            case rcpp::ParameterType::PARAMETER_BOOL:
                return param.as_bool() ? "true" : "false";
            case rcpp::ParameterType::PARAMETER_INTEGER:
                return std::to_string(param.as_int());
            case rcpp::ParameterType::PARAMETER_DOUBLE:
                return std::to_string(param.as_double());
            case rcpp::ParameterType::PARAMETER_STRING:
                return "\"" + param.as_string() + "\"";
            default:
                return "unknown";
        }
    }

    // ===== 成员变量定义 =====

    /**
     * @brief 同步参数客户端对象 - 连接到高级参数节点
     */
    std::shared_ptr<rcpp::SyncParametersClient> param_client_;
};

/**
 * @brief 主函数 - 高级参数客户端程序入口点
 *
 * 使用方法:

```

```

* 1. 首先启动高级参数节点: ros2 run parameter_demo advanced_parameter_node
* 2. 然后运行此测试客户端: ros2 run parameter_demo advanced_parameter_client
*
* @param argc 命令行参数个数
* @param argv 命令行参数数组
* @return int 程序退出码
*/
int main(int argc, char ** argv)
{
    // 初始化ROS2系统
    rclcpp::init(argc, argv);

    // 输出使用说明
    std::cout << "\n✎ 高级参数节点测试客户端\n";
    std::cout << "📖 请确保已启动 advanced_parameter_node\n";
    std::cout << "💡 命令: ros2 run parameter_demo advanced_parameter_node\n\n";

    // 创建测试客户端
    auto node = std::make_shared<AdvancedParameterClient>();

    // 清理ROS2系统资源
    rclcpp::shutdown();

    return 0; // 正常退出
}

```

编译配置

CMakeLists.txt:

```

cmake_minimum_required(VERSION 3.8)
project(parameter_demo)

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "clang")
    add_compile_options(-Wall -Wextra -Wpedantic)
endif()

# 查找依赖包
find_package(ament_cmake REQUIRED)
find_package(rclcpp REQUIRED)
find_package(rcl_interfaces REQUIRED)
find_package(PkgConfig REQUIRED)
pkg_check_modules(YAML_CPP REQUIRED yaml-cpp)

# 创建可执行文件
add_executable(basic_parameter_node src/basic_parameter_node.cpp)
add_executable(advanced_parameter_node src/advanced_parameter_node.cpp)
add_executable(parameter_client src/parameter_client.cpp)

```

```

add_executable(advanced_parameter_client src/advanced_parameter_client.cpp)

# 设置依赖
ament_target_dependencies(basic_parameter_node rclcpp rcl_interfaces)
ament_target_dependencies(advanced_parameter_node rclcpp rcl_interfaces)
ament_target_dependencies(parameter_client rclcpp rcl_interfaces)
ament_target_dependencies(advanced_parameter_client rclcpp rcl_interfaces)

# 链接yaml-cpp库到advanced_parameter_node
target_link_libraries(advanced_parameter_node ${YAML_CPP_LIBRARIES})
target_include_directories(advanced_parameter_node PRIVATE ${YAML_CPP_INCLUDE_DIRS})

# 安装可执行文件
install(TARGETS
  basic_parameter_node
  advanced_parameter_node
  parameter_client
  advanced_parameter_client
  DESTINATION lib/${PROJECT_NAME})

# 安装配置文件
install(DIRECTORY config/
  DESTINATION share/${PROJECT_NAME}/config
  FILES_MATCHING PATTERN "*.yaml")

ament_package()

```

编译和运行

```

# 1. 编译参数演示包
cd ~/ros2_ws
colcon build --packages-select parameter_demo

# 2. 设置环境
source install/setup.bash

# 3.1 运行基础参数节点（终端1）
ros2 run parameter_demo basic_parameter_node

# 3.2 运行基础参数客户端（终端2）
ros2 run parameter_demo parameter_client

# 4.1 运行高级参数节点（终端1）
ros2 run parameter_demo advanced_parameter_node

# 4.2 运行高级参数客户端（终端2）
ros2 run parameter_demo advanced_parameter_client

```

运行效果演示

基础参数节点输出:

```
[INFO] [basic_parameter_node]: 基础参数节点已启动
[INFO] [basic_parameter_node]: === 当前参数值 ===
[INFO] [basic_parameter_node]: 调试模式: 禁用
[INFO] [basic_parameter_node]: 循环频率: 10 Hz
[INFO] [basic_parameter_node]: 最大速度: 1.00 m/s
[INFO] [basic_parameter_node]: 机器人名称: robot1
[INFO] [basic_parameter_node]: 关节位置: [0.000000, 0.000000, 0.000000, 0.000000, 0.000000, 0.000000]
[INFO] [basic_parameter_node]: 传感器: ["camera", "lidar", "imu"]
[INFO] [basic_parameter_node]: 机器人 robot1 正在以 1.00 m/s 的最大速度运行 (频率: 10 Hz)
```

参数客户端输出:

```
[INFO] [parameter_client]: 参数客户端已启动

=== 列出所有参数 ===
[INFO] [parameter_client]: 发现 6 个参数:
[INFO] [parameter_client]:   - debug_mode
[INFO] [parameter_client]:   - loop_rate
[INFO] [parameter_client]:   - max_speed
[INFO] [parameter_client]:   - robot_name
[INFO] [parameter_client]:   - joint_positions
[INFO] [parameter_client]:   - sensor_names

=== 获取特定参数 ===
[INFO] [parameter_client]: debug_mode = false (类型: bool)
[INFO] [parameter_client]: max_speed = 1.000000 (类型: double)
[INFO] [parameter_client]: robot_name = "robot1" (类型: string)
[INFO] [parameter_client]: joint_positions = [0.000000, 0.000000, 0.000000, 0.000000, 0.000000, 0.000000] (类型: double_array)

=== 设置参数 ===
[INFO] [parameter_client]:  成功设置 debug_mode = true
[INFO] [parameter_client]:  成功设置 max_speed = 2.500000
[INFO] [parameter_client]:  成功设置 robot_name = "super_robot"
[INFO] [parameter_client]:  成功设置 loop_rate = 20
```

高级参数节点输出:

```
[INFO] [1750316902.280422633] [advanced_parameter_node]: 高级参数节点已启动
[INFO] [1750316911.961570975] [advanced_parameter_node]: 参数验证通过:
navigation.max_linear_velocity
[INFO] [1750316911.961750119] [advanced_parameter_node]: 参数验证通过:
navigation.max_angular_velocity
[INFO] [1750316911.961926796] [advanced_parameter_node]: 参数验证通过: control.pid.kp
[INFO] [1750316911.962051959] [advanced_parameter_node]: 参数验证通过:
sensors.camera.fps
[INFO] [1750316911.962150629] [advanced_parameter_node]: 参数验证通过:
control.frequency
[INFO] [1750316911.962351037] [advanced_parameter_node]: 参数已变更:
navigation.max_linear_velocity (类型: 3)
[INFO] [1750316911.962428689] [advanced_parameter_node]: 参数已变更:
navigation.max_angular_velocity (类型: 3)
.....
```

高级参数客户端输出:

```
 高级参数节点测试客户端
 请确保已启动 advanced_parameter_node
 命令: ros2 run parameter_demo advanced_parameter_node

[INFO] [1750316911.956074169] [advanced_parameter_client]:  高级参数客户端已启动, 开
[INFO] [1750316911.956260710] [advanced_parameter_client]:
 === 测试1: 基础功能测试 ===
[INFO] [1750316911.956936324] [advanced_parameter_client]:  发现 25 个参数
[INFO] [1750316911.957022497] [advanced_parameter_client]:  参数组 'navigation.' 存
在
[INFO] [1750316911.957035137] [advanced_parameter_client]:  参数组 'sensors.' 存在
[INFO] [1750316911.957079523] [advanced_parameter_client]:  参数组 'control.' 存在
[INFO] [1750316911.957114904] [advanced_parameter_client]:
 === 测试2: 参数组测试 ===
[INFO] [1750316911.957159007] [advanced_parameter_client]:  测试导航参数组...
[INFO] [1750316911.958630492] [advanced_parameter_client]: 
navigation.planning_aA_star"
[INFO] [1750316911.958734003] [advanced_parameter_client]: 
navigation.max_planni.000000
[INFO] [1750316911.958744460] [advanced_parameter_client]: 
navigation.planning_r0.050000
[INFO] [1750316911.958751598] [advanced_parameter_client]: 
navigation.use_obstac
[INFO] [1750316911.958758125] [advanced_parameter_client]: 
navigation.max_linear1.000000
[INFO] [1750316911.958764476] [advanced_parameter_client]: 
navigation.max_angula 1.000000
.....
```

4.6.4 参数命令行操作详解

查看参数

```
# 列出所有节点的参数
ros2 param list

# 列出特定节点的参数
ros2 param list /basic_parameter_node

# 获取参数值
ros2 param get /basic_parameter_node debug_mode
ros2 param get /basic_parameter_node max_speed
ros2 param get /basic_parameter_node robot_name

# 获取数组参数
ros2 param get /basic_parameter_node joint_positions
ros2 param get /basic_parameter_node sensor_names

# 查看参数描述
ros2 param describe /basic_parameter_node max_speed
```

设置参数

```
# 设置基本类型参数
ros2 param set /basic_parameter_node debug_mode true
ros2 param set /basic_parameter_node max_speed 3.0
ros2 param set /basic_parameter_node robot_name "advanced_robot"
ros2 param set /basic_parameter_node loop_rate 25

# 设置数组参数
ros2 param set /basic_parameter_node joint_positions "[0.1, 0.2, 0.3, 0.4, 0.5, 0.6]"
ros2 param set /basic_parameter_node sensor_names "['camera', 'lidar', 'imu', 'gps']"

# 批量设置参数
ros2 param set /basic_parameter_node debug_mode false max_speed 1.5
```

参数文件操作

```
# 导出节点参数到文件，要节点启动的时候才能导出参数配置文件
ros2 param dump /basic_parameter_node >
src/parameter_demo/config/exported_basic_params.yaml

# 启动基础参数节点并加载配置
```

```

ros2 run parameter_demo basic_parameter_node --ros-args --params-file
src/parameter_demo/config/basic_parameter_node.yaml

# 启动高级参数节点并加载配
ros2 run parameter_demo advanced_parameter_node --ros-args --params-file
src/parameter_demo/config/robot_params.yaml

# 在启动时设置特定参数
ros2 run parameter_demo basic_parameter_node --ros-args \
  -p debug_mode:=true \
  -p max_speed:=2.5 \
  -p robot_name:="test_robot"

```

参数监控和调试

```

# 实时监控参数变更 有参数更新时就打印
ros2 topic echo /parameter_events

# 使用rqt参数编辑器
rqt_reconfigure # 或者
rqt --standalone rqt_reconfigure

# 查看参数类型信息
ros2 interface show rcl_interfaces/msg/Parameter

# 查看参数事件消息格式
ros2 interface show rcl_interfaces/msg/ParameterEvent

```

创建参数配置文件

config/robot_params.yaml:

```

basic_parameter_node:
  ros__parameters:
    debug_mode: true
    loop_rate: 15
    max_speed: 2.0
    robot_name: "config_robot"
    joint_positions: [0.1, 0.2, 0.3, 0.4, 0.5, 0.6]
    sensor_names: ["camera", "lidar", "imu", "encoder"]

advanced_parameter_node:
  ros__parameters:
    navigation:
      planning_algorithm: "RRT"
      max_planning_time: 15.0
      planning_resolution: 0.03
      use_obstacles: true
      max_linear_velocity: 1.5
      max_angular_velocity: 0.8

```

```

sensors:
  lidar:
    topic: "/scan"
    frame_id: "laser_frame"
    min_range: 0.08
    max_range: 12.0
  camera:
    topic: "/camera/image_raw"
    width: 1920
    height: 1080
    fps: 60.0

control:
  pid:
    kp: 1.5
    ki: 0.2
    kd: 0.08
  max_output: 2.0
  frequency: 100

```

高级命令行操作

```

# 使用命名空间
ros2 run parameter_demo basic_parameter_node --ros-args \
  -r __ns:=/robot1 \
  -p robot_name:="robot1"

# 参数重映射
ros2 run parameter_demo basic_parameter_node --ros-args \
  --remap robot_name:=custom_name \
  -p custom_name:="remapped_robot"

# 条件参数设置
ros2 run parameter_demo basic_parameter_node --ros-args \
  -p debug_mode:=true \
  -p max_speed:="$(eval '2.0 if True else 1.0')"

# 组合多个参数文件
ros2 run parameter_demo advanced_parameter_node --ros-args \
  --params-file config/base_params.yaml \
  --params-file config/sensor_params.yaml \
  -p navigation.max_speed:=3.0

```

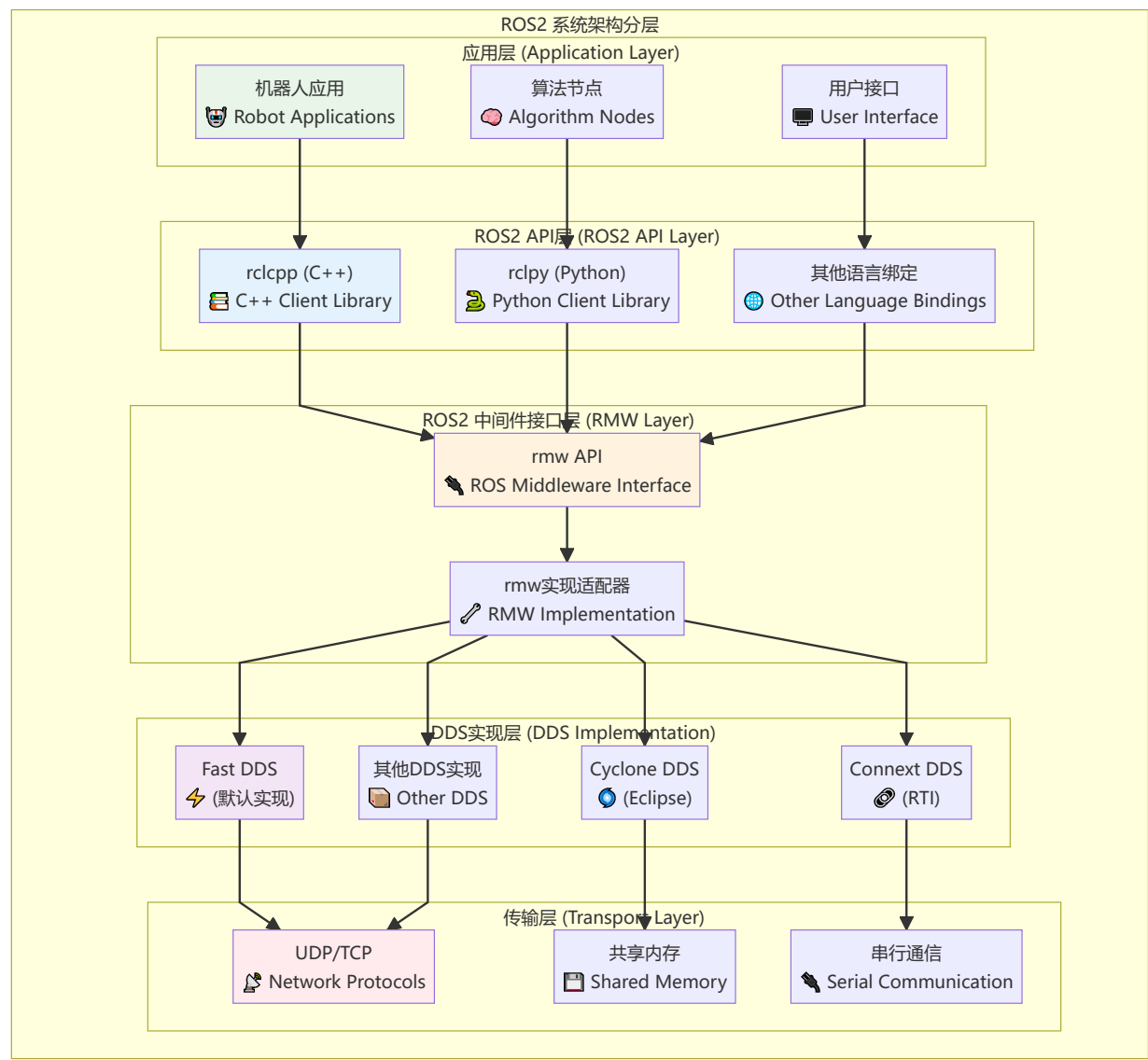
4.7 数据分发服务DDS

4.7.1 DDS是什么 (What is DDS)

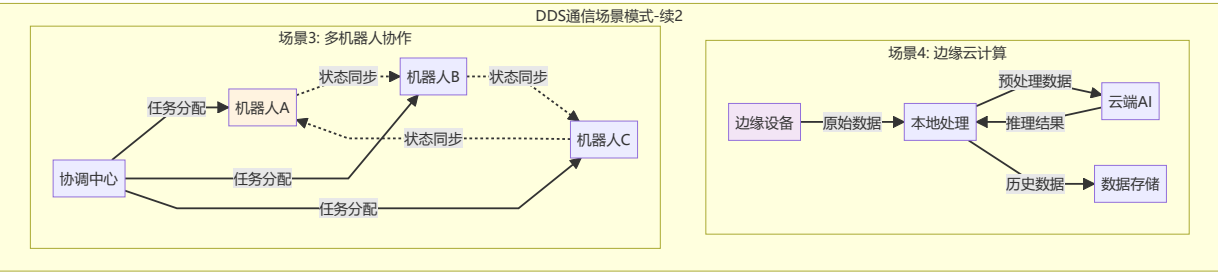
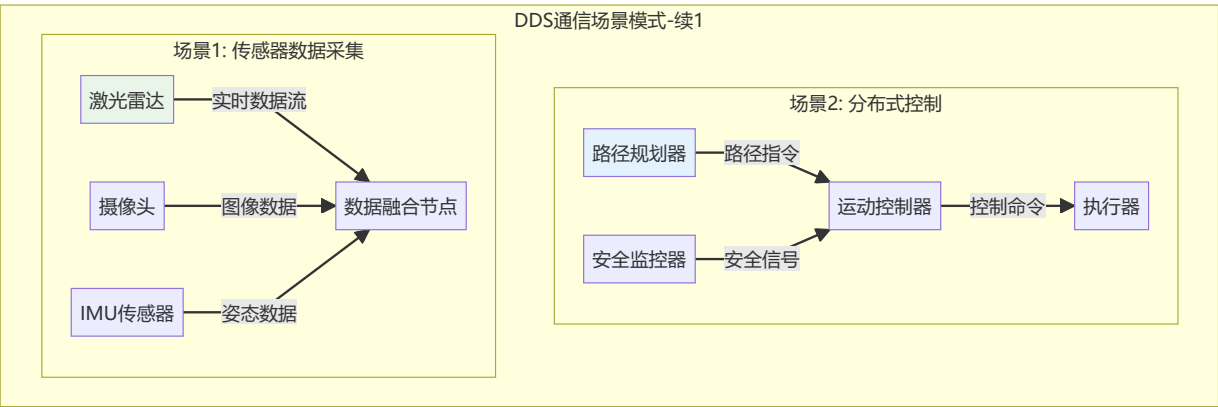
DDS概念和定义

- **DDS (Data Distribution Service)** ：数据分发服务，是ROS2的底层通信中间件
- **作用**：提供高性能、实时、可靠的分布式数据通信
- **标准**：基于OMG DDS规范，支持发布-订阅模式

ROS2系统架构分层图



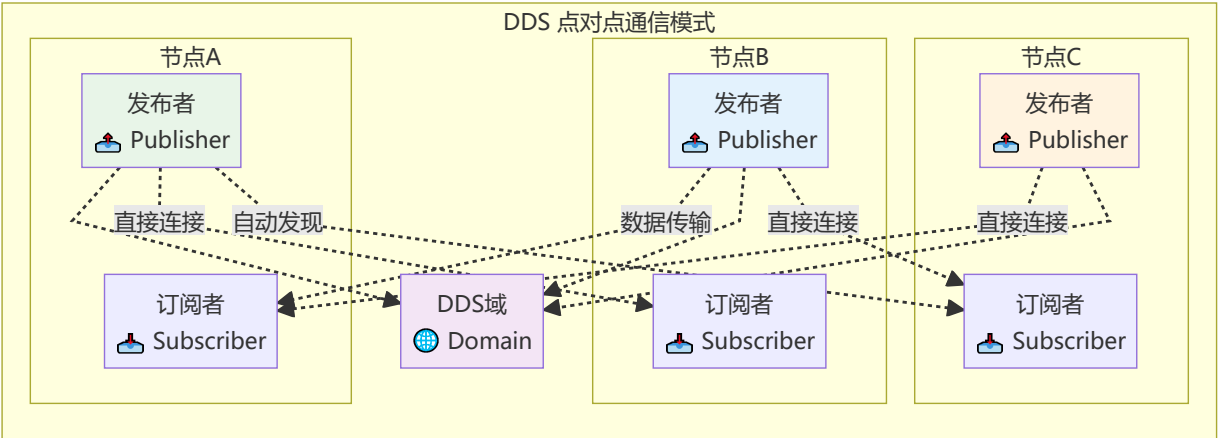
DDS在机器人系统中的通信场景



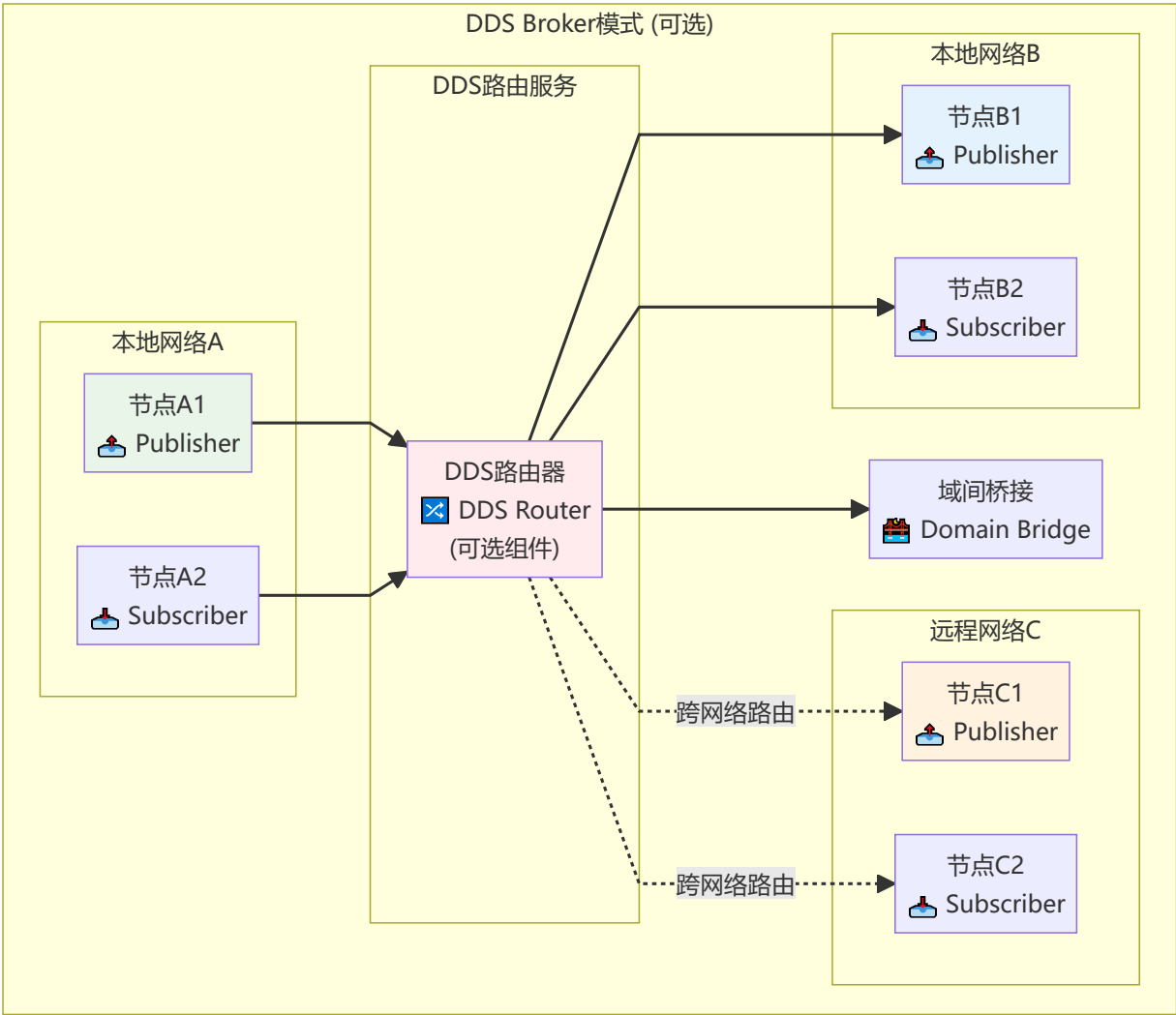
不局限以上场景。

4.7.2 DDS通信模型 (DDS Communication Models)

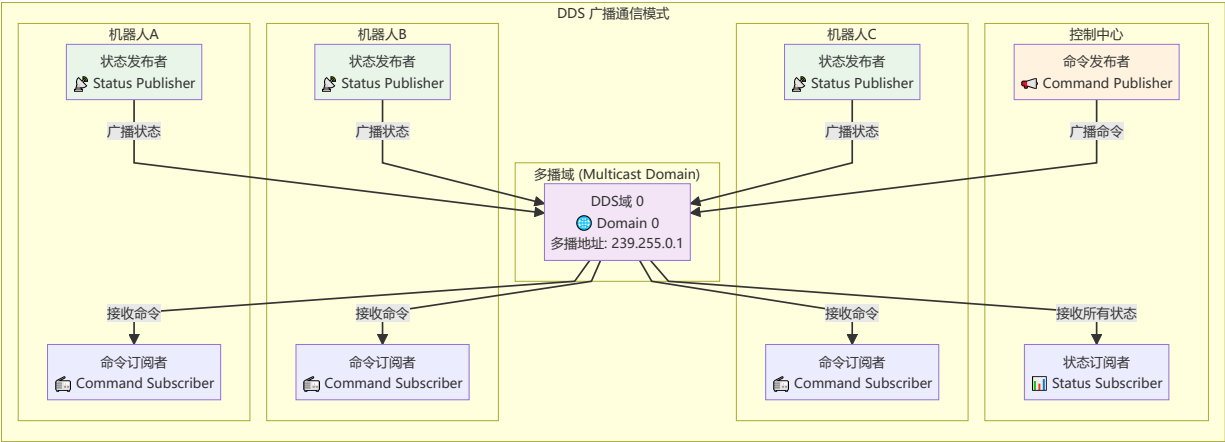
点对点模式 (Peer-to-Peer Model)



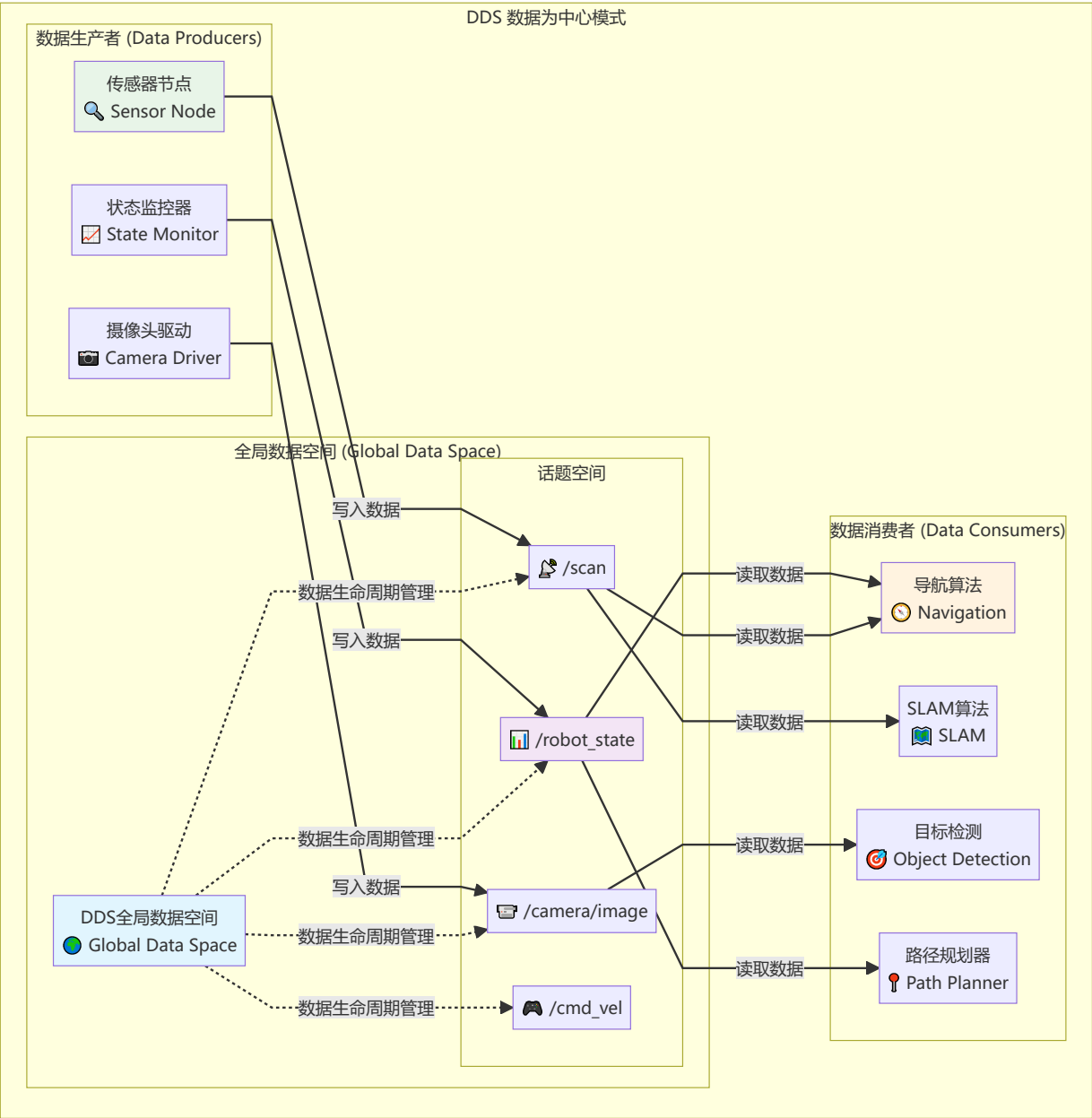
Broker模式 (Broker Model)



广播模式 (Broadcast Model)

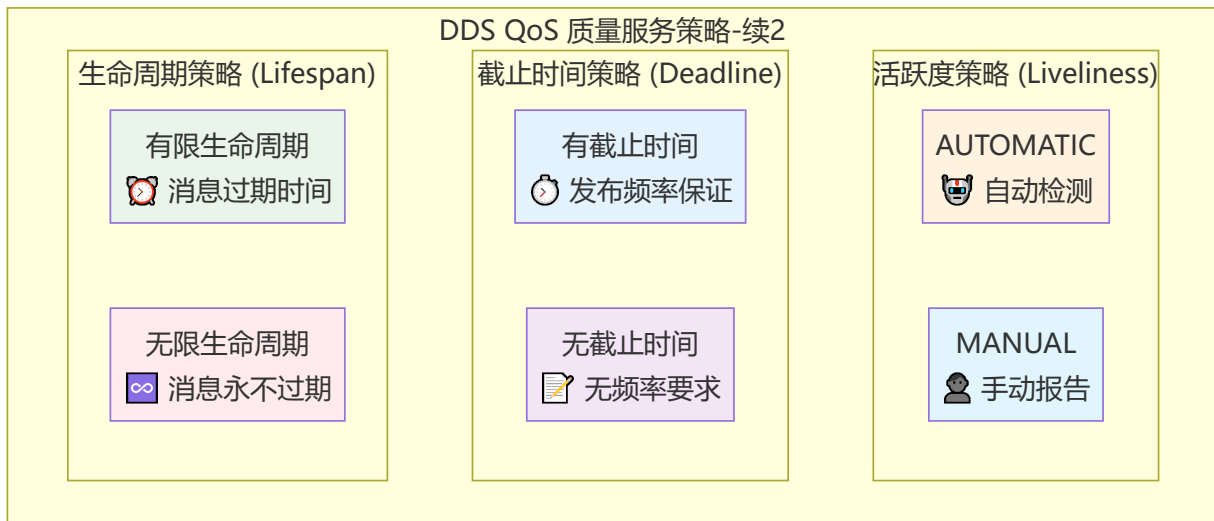
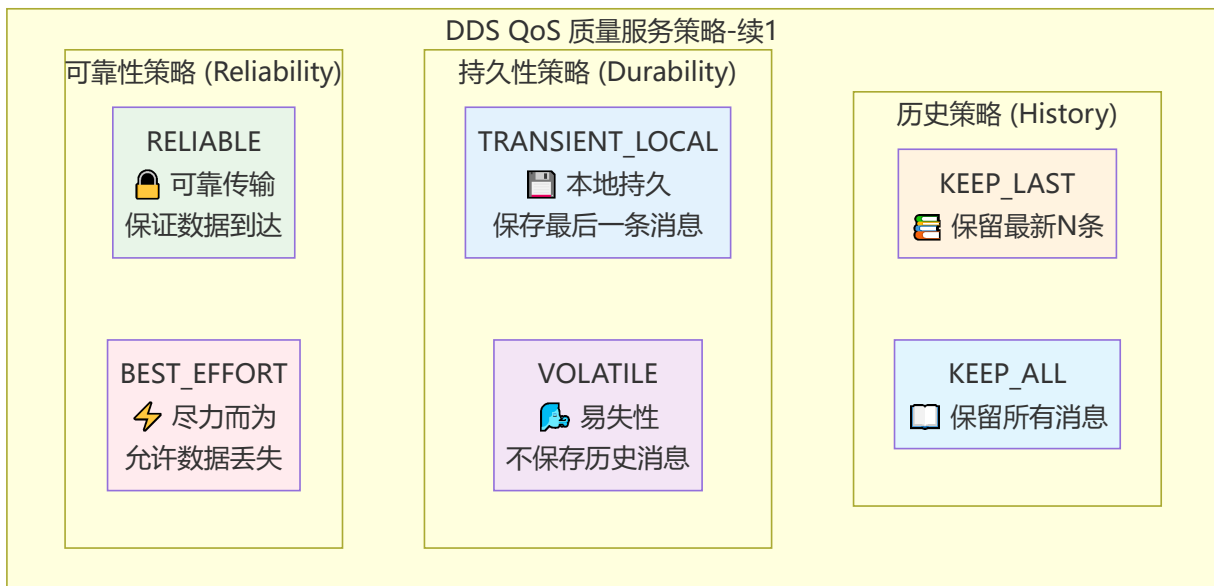


数据为中心模式 (Data-Centric Model)

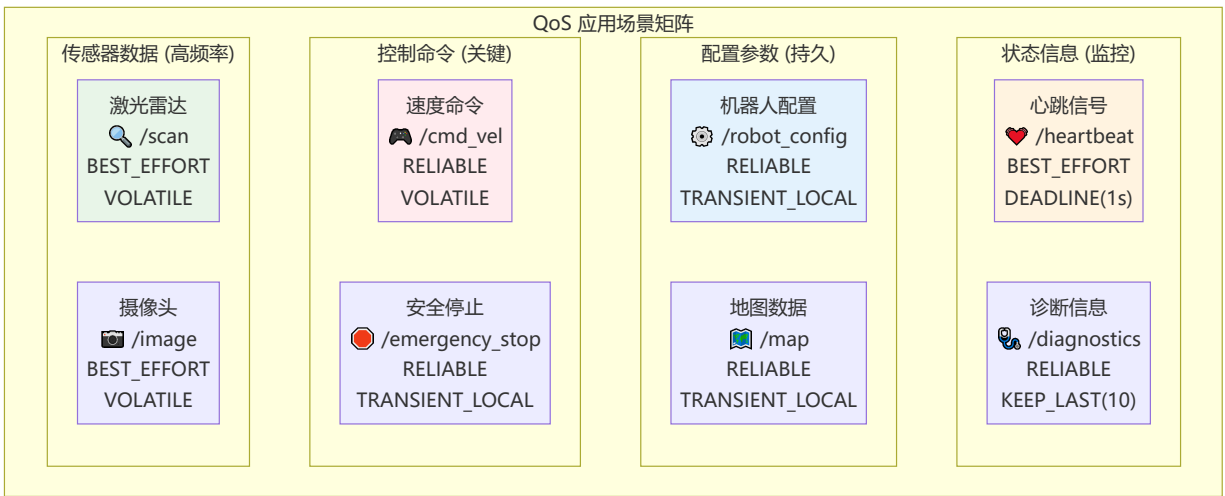


4.7.3 质量服务策略 (Quality of Service - QoS)

QoS策略概览图



QoS策略应用场景



4.7.4 命令行中配置DDS的QoS

创建QoS配置文件

config/qos_profiles.yaml:

```
# ROS2 QoS配置文件
qos_profile_sensor_data:
  history: keep_last
  depth: 5
  reliability: best_effort
  durability: volatile
  deadline:
    sec: 0
    nsec: 100000000 # 100ms
  lifespan:
    sec: 1
    nsec: 0
  liveliness: automatic
  liveliness_lease_duration:
    sec: 3
    nsec: 0

qos_profile_control_commands:
  history: keep_last
  depth: 1
  reliability: reliable
  durability: volatile
  deadline:
    sec: 0
    nsec: 50000000 # 50ms
  lifespan:
    sec: 0
    nsec: 50000000 # 500ms
  liveliness: automatic
```

```

liveliness_lease_duration:
  sec: 1
  nsec: 0

qos_profile_system_config:
  history: keep_last
  depth: 1
  reliability: reliable
  durability: transient_local
  deadline:
    sec: 2147483647
    nsec: 999999999 # 无限制
  lifespan:
    sec: 2147483647
    nsec: 999999999 # 无限制
  liveliness: automatic
  liveliness_lease_duration:
    sec: 10
    nsec: 0

qos_profile_diagnostics:
  history: keep_last
  depth: 50
  reliability: reliable
  durability: volatile
  deadline:
    sec: 5
    nsec: 0
  lifespan:
    sec: 30
    nsec: 0
  liveliness: manual_by_topic
  liveliness_lease_duration:
    sec: 10
    nsec: 0

```

命令行QoS操作

```

# 查看话题的QoS设置
ros2 topic info /scan --verbose
ros2 topic info /cmd_vel --verbose

# 使用特定QoS发布消息
ros2 topic pub /test_topic std_msgs/msg/String \
  '{data: "test message"}' \
  --qos-reliability reliable \
  --qos-durability transient_local \
  --qos-history keep_last \
  --qos-depth 10

# 查看QoS兼容性

```

```

ros2 topic info /scan --verbose | grep QoS

# 使用QoS配置文件启动节点
ros2 run my_package my_node --ros-args \
  --params-file config/qos_profiles.yaml

# 监控QoS匹配状态
ros2 doctor --report | grep -i qos

# 设置DDS详细日志
export RCUTILS_LOGGING_SEVERITY=DEBUG
export RMW_IMPLEMENTATION=rmw_fastrtps_cpp
export FASTRTPS_DEFAULT_PROFILES_FILE=config/dds_profiles.xml

```

DDS配置文件示例

config/dds_profiles.xml:

```

<?xml version="1.0" encoding="UTF-8" ?>
<profiles xmlns="http://www.eprosima.com/XMLSchemas/fastrtps_profiles">

  <!-- 传感器数据配置 -->
  <data_writer profile_name="sensor_data_writer">
    <qos>
      <reliability>
        <kind>BEST_EFFORT</kind>
      </reliability>
      <durability>
        <kind>VOLATILE</kind>
      </durability>
      <history>
        <kind>KEEP_LAST</kind>
        <depth>5</depth>
      </history>
      <resource_limits>
        <max_samples>100</max_samples>
        <max_instances>10</max_instances>
        <max_samples_per_instance>10</max_samples_per_instance>
      </resource_limits>
    </qos>
    <historyMemoryPolicy>PREALLOCATED_WITH_REALLOC</historyMemoryPolicy>
  </data_writer>

  <!-- 控制命令配置 -->
  <data_writer profile_name="control_command_writer">
    <qos>
      <reliability>
        <kind>RELIABLE</kind>
      </reliability>
      <max_blocking_time>
        <sec>0</sec>
        <nanosec>100000000</nanosec>
      </max_blocking_time>
    </qos>
  </data_writer>

```



```

        </reliability>
        <durability>
            <kind>VOLATILE</kind>
        </durability>
        <history>
            <kind>KEEP_LAST</kind>
            <depth>1</depth>
        </history>
        <deadline>
            <period>
                <sec>0</sec>
                <nanosec>50000000</nanosec>
            </period>
        </deadline>
    </qos>
</data_writer>

<!-- 配置数据配置 -->
<data_writer profile_name="config_data_writer">
    <qos>
        <reliability>
            <kind>RELIABLE</kind>
        </reliability>
        <durability>
            <kind>TRANSIENT_LOCAL</kind>
        </durability>
        <history>
            <kind>KEEP_LAST</kind>
            <depth>1</depth>
        </history>
    </qos>
</data_writer>

</profiles>

```

4.7.5 DDS C++示例

📁 完整项目结构

```

dds_example/
├── CMakeLists.txt           # 编译配置文件
├── package.xml             # 包信息和依赖
├── src/                    # C++源代码
│   ├── dds_publisher.cpp   # DDS发布者节点
│   └── dds_subscriber.cpp  # DDS订阅者节点
├── config/                # 配置文件
│   └── qos_profiles.yaml   # 多种QoS配置
├── launch/                # 启动文件
│   └── dds_demo.launch.py  # 基础演示启动

```

```
| └─ 🚀 dds_full_test.launch.py # 完整测试启动
|   └─ 📁 scripts/ # 测试脚本
|       └─ 📄 test_dds_demo.py # Python测试客户端
```

✅ 已实现的功能

1. 高级QoS配置演示

- 可靠传输 (RELIABLE)
- 持久化存储 (TRANSIENT_LOCAL)
- 自定义生命周期和截止时间
- 多种应用场景配置

2. 激光扫描数据模拟

- 360度激光雷达数据生成
- 10Hz发布频率
- 障碍物模拟和检测
- 数据完整性验证

创建工作空间和包

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_cmake dds_example --dependencies rclcpp
sensor_msgs std_msgs
```

DDS数据发布示例

```
/**
 * @file dds_example/src/dds_publisher.cpp
 * @brief ROS2 DDS发布者节点 - 展示高级QoS配置和激光扫描数据发布
 *
 * 本文件实现了一个DDS发布者节点，主要功能包括：
 * - 配置自定义QoS策略（可靠性、持久性、生命周期等）
 * - 模拟360度激光扫描数据生成
 * - 10Hz频率发布激光雷达数据
 * - 性能统计和连接状态监控
 * - 演示DDS的高级特性
 */

#include <chrono>
#include <memory>
#include <string>
#include <random>
```

```

#include "rclcpp/rclcpp.hpp"
#include "sensor_msgs/msg/laser_scan.hpp"
#include "std_msgs/msg/header.hpp"

using namespace std::chrono_literals;

/**
 * @class DDSPublisher
 * @brief DDS发布者类，继承自rclcpp::Node
 *
 * 该类展示了如何在ROS2中使用DDS的高级特性：
 * - 自定义QoS配置，包括可靠性传输和持久化存储
 * - 模拟真实的激光扫描数据生成
 * - 实时性能监控和统计
 * - 适用于工业级机器人系统的数据发布
 */
class DDSPublisher : public rclcpp::Node
{
public:
    /**
     * @brief 构造函数，初始化DDS发布者节点
     *
     * 在构造函数中完成以下初始化工作：
     * - 创建高级QoS配置文件
     * - 创建激光扫描数据发布者
     * - 设置10Hz的定时发布者
     * - 初始化随机数生成器用于数据模拟
     */
    DDSPublisher() : Node("dds_publisher"), count_(0)
    {
        // 输出节点启动信息
        RCLCPP_INFO(this->get_logger(), "DDS发布者节点启动");

        // 配置高级QoS策略，确保数据传输的可靠性和持久性
        auto qos_profile = createCustomQoSProfile();

        // 创建激光扫描数据发布者，使用自定义QoS配置
        laser_publisher_ = this->create_publisher<sensor_msgs::msg::LaserScan>(
            "sensor/laser_scan", qos_profile);

        // 创建定时器，以10Hz频率发布数据（100ms间隔）
        // 这个频率适合大多数激光雷达应用场景
        timer_ = this->create_wall_timer(
            100ms, std::bind(&DDSPublisher::publishLaserData, this));

        // 初始化随机数生成器，用于生成模拟的激光扫描数据
        // 使用当前时间作为种子，确保每次运行时数据不同

        random_generator_.seed(std::chrono::steady_clock::now().time_since_epoch().count());
    };

```

```

        RCLCPP_INFO(this->get_logger(), "激光扫描数据发布者已创建，QoS配置：可靠传输，保留最后5条消息");
    }

private:
    /**
     * @brief 创建自定义QoS配置文件
     * @return rclcpp::QoS 配置好的QoS策略
     *
     * 该方法配置了适合激光扫描数据传输的QoS策略：
     * - 可靠性：确保数据不丢失，适合重要的传感器数据
     * - 持久性：为延迟订阅者保留历史数据
     * - 历史策略：保留最近5条消息，平衡内存使用和数据完整性
     * - 生命周期：数据有效期5秒，超时后自动清理
     * - 截止时间：1秒处理超时，确保实时性
     */
    rclcpp::QoS createCustomQoSProfile()
    {
        // 创建QoS配置，设置历史队列深度为5
        rclcpp::QoS qos_profile(5); // 保留最近5条消息

        // 可靠性设置：RELIABLE确保数据传输的可靠性
        // 在网络不稳定时会重传丢失的数据包
        qos_profile.reliability(RMW_QOS_POLICY_RELIABILITY_RELIABLE);

        // 持久性设置：TRANSIENT_LOCAL为延迟订阅者保留数据
        // 即使订阅者晚于发布者启动，也能接收到之前的数据
        qos_profile.durability(RMW_QOS_POLICY_DURABILITY_TRANSIENT_LOCAL);

        // 生命周期设置：数据有效期5秒
        // 超过5秒的数据会被自动清理，防止内存泄漏
        qos_profile.lifespan(std::chrono::seconds(5));

        // 截止时间设置：处理超时时间1秒
        // 如果数据处理时间超过1秒，系统会发出警告
        qos_profile.deadline(std::chrono::seconds(1));

        return qos_profile;
    }

    /**
     * @brief 生成并发布模拟激光扫描数据
     *
     * 该方法模拟真实激光雷达的工作过程：
     * - 生成360度扫描数据（每度一个测量点）
     * - 设置真实的激光雷达参数（角度范围、距离范围等）
     * - 添加随机噪声模拟真实环境
     * - 插入特定模式的障碍物检测
     * - 记录发布性能统计
     */
    void publishLaserData()
    {

```

```

// 创建激光扫描消息对象
auto message = sensor_msgs::msg::LaserScan();

// 设置消息头信息
message.header.stamp = this->get_clock()->now(); // 当前时间戳
message.header.frame_id = "laser_frame";        // 坐标系ID

// 设置激光扫描仪的物理参数
message.angle_min = -M_PI;                      // 起始角度: -180度 (左侧)
message.angle_max = M_PI;                      // 结束角度: +180度 (右侧)
message.angle_increment = M_PI/180;            // 角度增量: 1度, 总共360个测量点
message.time_increment = 0.0;                  // 测量间时间间隔 (连续扫描为0)
message.scan_time = 0.1;                      // 完整扫描时间: 100ms
message.range_min = 0.1;                      // 最小有效测距: 10cm
message.range_max = 10.0;                      // 最大有效测距: 10m

// 生成360个方向的距离测量数据
size_t num_readings = 360;
message.ranges.resize(num_readings);           // 距离数据数组
message.intensities.resize(num_readings);       // 反射强度数组

// 创建随机数分布器, 模拟真实环境中的距离和强度变化
std::uniform_real_distribution<float> range_dist(0.5, 8.0); // 距离范围:
0.5-8.0米
std::uniform_real_distribution<float> intensity_dist(0.0, 100.0); // 强度范围: 0-100

// 为每个角度生成模拟的激光测距数据
for (size_t i = 0; i < num_readings; ++i) {
    // 模拟一些有趣的扫描模式, 增强演示效果
    if (i % 90 == 0) {
        // 每90度 (正前方、右侧、正后方、左侧) 放置近距离障碍物
        message.ranges[i] = 0.5; // 0.5米的近距离障碍物
    } else if (i % 45 == 0) {
        // 每45度方向设置远距离空旷区域
        message.ranges[i] = range_max; // 最大测距范围, 表示开阔空间
    } else {
        // 其他方向使用随机距离, 模拟复杂环境
        message.ranges[i] = range_dist(random_generator_);
    }

    // 为每个测量点生成随机的反射强度值
    // 反射强度取决于物体表面材质和距离
    message.intensities[i] = intensity_dist(random_generator_);
}

// 记录发布操作的开始时间, 用于性能统计
auto start_time = std::chrono::high_resolution_clock::now();

// 发布激光扫描数据到DDS网络
laser_publisher->publish(message);

```

```

// 记录发布操作的结束时间
auto end_time = std::chrono::high_resolution_clock::now();

// 计算本次发布操作的耗时（微秒级精度）
auto duration = std::chrono::duration_cast<std::chrono::microseconds>(
    end_time - start_time);

// 增加发布计数器
count_++;

// 每发布10条消息打印一次详细统计信息
// 这样可以监控系统性能，同时避免日志过于频繁
if (count_ % 10 == 0) {
    RCLCPP_INFO(this->get_logger(),
        "已发布第 %ld 条激光扫描数据，发布耗时：%ld 微秒，数据点数：%zu",
        count_, duration.count(), num_readings);

    // 打印QoS相关的统计信息
    printQoSStatistics();
}
}

/**
 * @brief 打印QoS统计信息和连接状态
 *
 * 该方法监控DDS连接的状态信息：
 * - 当前订阅者数量
 * - 话题名称确认
 * - QoS策略生效情况
 * - 网络连接健康状况
 */
void printQoSStatistics()
{
    try {
        // 获取当前发布的话题名称
        auto topic_info = laser_publisher->get_topic_name();

        // 获取当前连接的订阅者数量
        // 这个信息对于调试网络连接问题很有用
        RCLCPP_INFO(this->get_logger(),
            "话题：%s，当前订阅者数量：%zu",
            topic_info,
            laser_publisher->get_subscription_count());

    } catch (const std::exception& e) {
        // 如果获取统计信息失败，记录错误但不中断程序执行
        RCLCPP_WARN(this->get_logger(), "获取QoS统计信息失败：%s", e.what());
    }
}

// 成员变量定义

```

```

    rclcpp::Publisher<sensor_msgs::msg::LaserScan>::SharedPtr laser_publisher_;
    ///< 激光扫描数据发布者
    rclcpp::TimerBase::SharedPtr timer_;
    ///< 定时发布者
    size_t count_;
    ///< 发布消息计数器
    std::mt19937 random_generator_;
    ///< 随机数生成器，用于数据模拟
    const float range_max = 10.0;                                     ///<
    激光雷达最大测距范围
};

/**
 * @brief 主函数，程序入口点
 * @param argc 命令行参数个数
 * @param argv 命令行参数数组
 * @return int 程序退出状态码
 *
 * 主函数负责：
 * - 初始化ROS2系统
 * - 创建并运行DDS发布者节点
 * - 处理异常和优雅退出
 * - 清理系统资源
 */
int main(int argc, char * argv[])
{
    // 初始化ROS2系统，必须在使用任何ROS2功能之前调用
    rclcpp::init(argc, argv);

    RCLCPP_INFO(rclcpp::get_logger("main"), "启动DDS发布者示例");

    // 创建DDS发布者节点的共享指针
    auto node = std::make_shared<DDSPublisher>();

    try {
        // 进入ROS2的事件循环，持续处理回调函数
        // 这会阻塞直到节点被关闭（如收到Ctrl+C信号）
        rclcpp::spin(node);
    } catch (const std::exception& e) {
        // 捕获并记录运行时异常，但不会崩溃程序
        RCLCPP_ERROR(rclcpp::get_logger("main"), "节点运行异常： %s", e.what());
    }

    // 清理ROS2系统资源，释放内存和网络连接
    rclcpp::shutdown();
    RCLCPP_INFO(rclcpp::get_logger("main"), "DDS发布者节点已关闭");

    return 0;
}

```

DDS数据订阅示例

```
/**
 * @file dds_example/src/dds_subscriber.cpp
 * @brief ROS2 DDS订阅者节点 - 展示高级QoS配置和激光扫描数据分析
 *
 * 本文件实现了一个DDS订阅者节点，主要功能包括：
 * - 配置与发布者匹配的QoS策略
 * - 接收并分析激光扫描数据
 * - 实时性能监控和延迟统计
 * - 数据质量检测和异常报警
 * - 演示DDS的持久化数据恢复功能
 */

#include <chrono>
#include <memory>
#include <string>
#include <deque>

#include "rclcpp/rclcpp.hpp"
#include "sensor_msgs/msg/laser_scan.hpp"

using namespace std::chrono_literals;

/**
 * @class DDSSubscriber
 * @brief DDS订阅者类，继承自rclcpp::Node
 *
 * 该类展示了如何在ROS2中实现高性能的DDS数据订阅：
 * - 与发布者完全匹配的QoS配置
 * - 实时激光扫描数据处理和分析
 * - 高精度延迟测量和统计分析
 * - 数据质量监控和异常检测
 * - 适用于工业级机器人系统的数据接收
 */
class DDSSubscriber : public rclcpp::Node
{
public:
    /**
     * @brief 构造函数，初始化DDS订阅者节点
     *
     * 在构造函数中完成以下初始化工作：
     * - 创建与发布者匹配的QoS配置
     * - 创建激光扫描数据订阅者
     * - 设置统计信息定时打印机
     * - 初始化性能监控变量
     */
    DDSSubscriber() : Node("dds_subscriber"), message_count_(0)
    {
        RCLCPP_INFO(this->get_logger(), "DDS订阅者节点启动");
    }
};
```



```

// 配置与发布者完全匹配的QoS策略，确保连接成功
auto qos_profile = createMatchingQoSProfile();

// 创建激光扫描数据订阅者，使用回调函数处理接收到的数据
laser_subscription_ = this->create_subscription<sensor_msgs::msg::LaserScan>
(
    "sensor/laser_scan",
    qos_profile,
    std::bind(&DDSSubscriber::laserScanCallback, this,
std::placeholders::_1));

// 创建统计定时器，每5秒打印一次统计信息
// 这个间隔适合监控长期运行的系统性能
stats_timer_ = this->create_wall_timer(
    5s, std::bind(&DDSSubscriber::printStatistics, this));

RCLCPP_INFO(this->get_logger(), "激光扫描数据订阅者已创建，等待数据...");
}

private:
/**
 * @brief 创建与发布者匹配的QoS配置
 * @return rclcpp::QoS 配置好的QoS策略
 *
 * 该方法创建与发布者完全匹配的QoS配置：
 * - 可靠性：RELIABLE，确保接收所有重要数据
 * - 持久性：TRANSIENT_LOCAL，能够接收延迟订阅的历史数据
 * - 历史策略：保留最近5条消息，与发布者保持一致
 * - 生命周期：5秒数据有效期，超时数据自动丢弃
 * - 截止时间：1秒处理期限，确保实时性要求
 */
rclcpp::QoS createMatchingQoSProfile()
{
    // 创建QoS配置，历史队列深度与发布者保持一致
    rclcpp::QoS qos_profile(5); // 保留最近5条消息

    // 可靠性设置：与发布者匹配，确保数据完整接收
    // RELIABLE模式会要求重传丢失的数据包
    qos_profile.reliability(RMW_QOS_POLICY_RELIABILITY_RELIABLE);

    // 持久性设置：与发布者匹配，支持延迟订阅数据恢复
    // TRANSIENT_LOCAL允许接收订阅前发布的历史数据
    qos_profile.durability(RMW_QOS_POLICY_DURABILITY_TRANSIENT_LOCAL);

    // 生命周期设置：与发布者保持一致，5秒数据有效期
    // 超过生命周期的数据将被自动丢弃
    qos_profile.lifespan(std::chrono::seconds(5));

    // 截止时间设置：处理超时时间1秒
    // 如果数据处理超过1秒，系统会发出性能警告
    qos_profile.deadline(std::chrono::seconds(1));
}

```

```

        return qos_profile;
    }

/**
 * @brief 激光扫描数据回调函数
 * @param msg 接收到的激光扫描消息
 *
 * 该回调函数处理每条接收到的激光扫描数据：
 * - 计算端到端传输延迟
 * - 分析激光扫描数据质量
 * - 检测障碍物和异常情况
 * - 维护性能统计信息
 * - 定期输出详细分析结果
 */
void laserScanCallback(const sensor_msgs::msg::LaserScan::SharedPtr msg)
{
    // 计算从发布到接收的端到端传输延迟
    auto publish_time = rclcpp::Time(msg->header.stamp); // 消息发布时间戳
    auto current_time = this->get_clock()->now();         // 当前接收时间
    auto latency = current_time - publish_time;           // 传输延迟

    // 调用专门的激光扫描数据分析函数
    auto analysis = analyzeLaserScan(msg);

    // 更新接收消息计数器
    message_count_++;

    // 记录延迟数据用于统计分析
    latencies_.push_back(latency.seconds());
    if (latencies_.size() > 100) { // 保留最近100个延迟数据，避免内存过度使用
        latencies_.pop_front();
    }

    // 每10条消息打印一次详细信息，平衡信息量和日志频率
    if (message_count_ % 10 == 0) {
        RCLCPP_INFO(this->get_logger(),
            "收到第 %ld 条激光扫描数据 | 延迟: %.3f ms | 有效点数: %zu/%zu | 最近距离: %.2f m | 平均距离: %.2f m",
            message_count_,
            latency.seconds() * 1000, // 转换为毫秒显示
            analysis.valid_points,
            analysis.total_points,
            analysis.min_range,
            analysis.avg_range);
    }

    // 检测数据质量问题和潜在的系统异常
    checkDataQuality(msg, analysis);
}

/**

```

```

* @struct LaserScanAnalysis
* @brief 激光扫描数据分析结果结构体
*
* 该结构体包含激光扫描数据的各种统计信息：
* - 数据点统计（总数、有效数）
* - 距离统计（最小值、最大值、平均值）
* - 障碍物检测统计
*/
struct LaserScanAnalysis {
    size_t total_points;    ///< 总测量点数
    size_t valid_points;    ///< 有效测量点数
    float min_range;        ///< 最小测距值
    float max_range;        ///< 最大测距值
    float avg_range;        ///< 平均测距值
    size_t obstacle_count;  ///< 近距离障碍物数量（距离<1米）
};

/**
* @brief 分析激光扫描数据的详细信息
* @param msg 激光扫描消息
* @return LaserScanAnalysis 分析结果结构体
*
* 该方法对激光扫描数据进行深度分析：
* - 统计有效和无效数据点
* - 计算距离的统计特征
* - 检测近距离障碍物
* - 验证数据的完整性和准确性
*/
LaserScanAnalysis analyzeLaserScan(const sensor_msgs::msg::LaserScan::SharedPtr
msg)
{
    // 初始化分析结果结构体
    LaserScanAnalysis analysis = {};
    analysis.total_points = msg->ranges.size();
    analysis.min_range = std::numeric_limits<float>::max(); // 初始化为最大值
    analysis.max_range = 0.0f;

    float range_sum = 0.0f;    // 用于计算平均距离
    size_t valid_count = 0;    // 有效数据点计数器

    // 遍历所有激光测距数据点
    for (size_t i = 0; i < msg->ranges.size(); ++i) {
        float range = msg->ranges[i];

        // 检查数据有效性：在有效范围内且不是NaN或无穷大
        if (range >= msg->range_min && range <= msg->range_max &&
            !std::isnan(range) && !std::isinf(range)) {

            // 统计有效数据点
            analysis.valid_points++;
            range_sum += range;
            valid_count++;
        }
    }
}

```

```

        // 更新最小和最大距离值
        analysis.min_range = std::min(analysis.min_range, range);
        analysis.max_range = std::max(analysis.max_range, range);

        // 检测近距离障碍物（距离小于1米）
        // 这对于机器人导航和安全防护很重要
        if (range < 1.0f) {
            analysis.obstacle_count++;
        }
    }

    // 计算平均距离，避免除零错误
    analysis.avg_range = valid_count > 0 ? range_sum / valid_count : 0.0f;

    return analysis;
}

/**
 * @brief 检查数据质量并发出相应警告
 * @param msg 激光扫描消息
 * @param analysis 数据分析结果
 *
 * 该方法监控激光扫描数据的质量：
 * - 检查数据完整性（有效数据比例）
 * - 监控障碍物密度（安全性考虑）
 * - 验证时间戳新鲜度（实时性检查）
 * - 发出相应的警告信息
 */
void checkDataQuality(const sensor_msgs::msg::LaserScan::SharedPtr msg,
                     const LaserScanAnalysis& analysis)
{
    // 检查数据完整性：计算有效数据比例
    float valid_ratio = static_cast<float>(analysis.valid_points) /
analysis.total_points;
    if (valid_ratio < 0.8f) { // 有效数据少于80%被认为是质量问题
        RCLCPP_WARN(this->get_logger(),
                    "数据质量警告：有效数据比例仅为 %.1f%%", valid_ratio * 100);
    }

    // 检查障碍物密度：过高的障碍物密度可能表示环境危险
    float obstacle_ratio = static_cast<float>(analysis.obstacle_count) /
analysis.valid_points;
    if (obstacle_ratio > 0.3f) { // 30%以上的点检测到近距离障碍物
        RCLCPP_WARN(this->get_logger(),
                    "障碍物密度高：%.1f%% 的有效点检测到近距离障碍物", obstacle_ratio * 100);
    }

    // 检查时间戳新鲜度：过旧的数据可能表示网络延迟或系统问题
    auto msg_age = this->get_clock()->now() - rclcpp::Time(msg->header.stamp);
    if (msg_age.seconds() > 0.5) { // 数据超过500ms被认为过旧

```

```

        RCLCPP_WARN(this->get_logger(),
            "数据时效性警告: 消息时间戳过旧 %.3f 秒", msg_age.seconds());
    }
}

/**
 * @brief 定期打印系统统计信息
 *
 * 该方法每5秒调用一次，提供系统性能的综合报告：
 * - 消息接收统计
 * - 延迟性能分析
 * - QoS连接状态
 * - 系统健康度评估
 */
void printStatistics()
{
    // 检查是否接收到任何数据
    if (message_count_ == 0) {
        RCLCPP_INFO(this->get_logger(), "暂未接收到任何数据");
        return;
    }

    // 计算详细的延迟统计信息
    if (!latencies_.empty()) {
        double avg_latency = 0.0; // 平均延迟
        double max_latency = 0.0; // 最大延迟
        double min_latency = std::numeric_limits<double>::max(); // 最小延迟

        // 遍历所有延迟数据计算统计值
        for (auto latency : latencies_) {
            avg_latency += latency;
            max_latency = std::max(max_latency, latency);
            min_latency = std::min(min_latency, latency);
        }
        avg_latency /= latencies_.size(); // 计算平均值

        // 打印详细的性能统计报告
        RCLCPP_INFO(this->get_logger(),
            "=== DDS性能统计 (最近5秒) ===");
        RCLCPP_INFO(this->get_logger(),
            "总消息数: %ld", message_count_);
        RCLCPP_INFO(this->get_logger(),
            "平均延迟: %.3f ms | 最小延迟: %.3f ms | 最大延迟: %.3f ms",
            avg_latency * 1000, min_latency * 1000, max_latency * 1000);
        RCLCPP_INFO(this->get_logger(),
            "延迟样本数: %zu", latencies_.size());
    }

    // 打印QoS连接信息和网络状态
    printQoSConnectionInfo();
}

```

```

/**
 * @brief 打印QoS连接信息和网络状态
 *
 * 该方法显示DDS连接的详细状态：
 * - 订阅话题信息
 * - QoS策略配置确认
 * - 网络连接健康状况
 * - 错误处理和异常报告
 */
void printQoSConnectionInfo()
{
    try {
        // 获取当前订阅的话题名称
        auto topic_info = laser_subscription->get_topic_name();
        RCLCPP_INFO(this->get_logger(),
            "订阅话题: %s", topic_info);

        // 打印当前生效的QoS配置信息
        // 注意：这里可以扩展更多QoS统计信息的获取
        RCLCPP_INFO(this->get_logger(),
            "QoS配置：可靠传输，持久化存储，5秒数据生命周期");

    } catch (const std::exception& e) {
        // 如果获取连接信息失败，记录警告但不中断程序运行
        RCLCPP_WARN(this->get_logger(), "获取连接信息失败: %s", e.what());
    }
}

// 成员变量定义
rclcpp::Subscription<sensor_msgs::msg::LaserScan>::SharedPtr
laser_subscription_; ///< 激光扫描数据订阅者
rclcpp::TimerBase::SharedPtr stats_timer_;
///< 统计信息定时器
size_t message_count_;
///< 接收消息计数器
std::deque<double> latencies_;
///< 延迟统计队列，使用deque便于维护固定大小
};

/**
 * @brief 主函数，程序入口点
 * @param argc 命令行参数个数
 * @param argv 命令行参数数组
 * @return int 程序退出状态码
 *
 * 主函数负责：
 * - 初始化ROS2系统
 * - 创建并运行DDS订阅者节点
 * - 处理用户中断和异常情况
 * - 优雅关闭系统资源
 */
int main(int argc, char * argv[])

```

```

{
    // 初始化ROS2系统，这是使用任何ROS2功能的前提
    rclcpp::init(argc, argv);

    RCLCPP_INFO(rclcpp::get_logger("main"), "启动DDS订阅者示例");

    // 创建DDS订阅者节点的智能指针
    auto node = std::make_shared<DDSSubscriber>();

    try {
        // 进入ROS2事件循环，持续处理消息和回调
        // 这个函数会阻塞，直到节点被外部信号中断
        rclcpp::spin(node);
    } catch (const std::exception& e) {
        // 捕获并记录运行时异常，确保程序不会意外崩溃
        RCLCPP_ERROR(rclcpp::get_logger("main"), "节点运行异常: %s", e.what());
    }

    // 清理ROS2系统资源，包括网络连接和内存
    rclcpp::shutdown();
    RCLCPP_INFO(rclcpp::get_logger("main"), "DDS订阅者节点已关闭");

    return 0;
}

```

编译配置

CMakeLists.txt

```

# 节选部分
# 安装配置文件
install(DIRECTORY config/
  DESTINATION share/${PROJECT_NAME}/config
  FILES_MATCHING PATTERN "*.yaml"
)

# 安装启动文件
install(DIRECTORY launch/
  DESTINATION share/${PROJECT_NAME}/launch
  FILES_MATCHING PATTERN "*.py"
)
....

```

config/qos_profiles.yaml

```
# @file dds_example/config/qos_profiles.yaml
# @brief ROS2 DDS QoS服务质量配置文件 - 针对不同应用场景的专业配置
#
# 本配置文件定义了6种不同应用场景的QoS策略，每种配置都经过精心设计
# 以满足特定的性能、可靠性和实时性要求。QoS配置是DDS系统的核心，
# 直接影响数据传输的可靠性、延迟、带宽使用和系统行为。
#
# QoS参数说明：
# - history: 历史策略 (keep_last/keep_all)
# - depth: 队列深度，影响内存使用和数据完整性
# - reliability: 可靠性 (reliable/best_effort)
# - durability: 持久性 (volatile/transient_local/transient/persistent)
# - deadline: 数据发布/接收的最大时间间隔
# - lifespan: 数据的有效生命周期
# - liveliness: 发布者存活检测机制
# - liveliness_lease_duration: 存活检测的租约时间

# 配置1: 高可靠性传感器数据配置
# 设计目标: 确保重要传感器数据的完整传输和历史数据恢复
# 典型应用: 激光雷达、摄像头、IMU等关键传感器
sensor_data_reliable:
  # 历史策略: 保留最近的数据，平衡内存使用和数据完整性
  history: keep_last
  # 队列深度: 10条消息，足够缓冲网络抖动期间的数据
  # 设计考虑: 太小会丢失数据，太大会占用过多内存
  depth: 10
  # 可靠性: RELIABLE确保数据不丢失，适合关键传感器数据
  # 设计原因: 传感器数据对机器人决策至关重要，不能容忍数据丢失
  reliability: reliable
  # 持久性: TRANSIENT_LOCAL支持延迟订阅者接收历史数据
  # 设计原因: 新启动的节点需要获取当前传感器状态，避免等待下一次数据
  durability: transient_local
  # 截止时间: 1秒，传感器数据更新频率的合理期望
  # 设计考虑: 大多数传感器以1-50Hz更新，1秒是安全的上限
  deadline:
    sec: 1
    nsec: 0
  # 生命周期: 5秒，防止过时数据影响决策
  # 设计原因: 传感器数据的时效性很重要，5秒后的数据通常无效
  lifespan:
    sec: 5
    nsec: 0
  # 存活检测: 自动模式，系统自动维护连接状态
  # 设计原因: 减轻应用程序负担，系统自动处理网络异常
  liveliness: automatic
  # 存活租约: 2秒，检测发布者是否还活跃
  # 设计考虑: 既能及时发现故障，又不会因短暂网络问题误判
  liveliness_lease_duration:
    sec: 2
    nsec: 0
```



```
# 配置2: 实时控制命令配置
# 设计目标: 最低延迟的控制命令传输, 优先实时性而非历史数据
# 典型应用: 电机控制、舵机控制、紧急停止等实时控制
control_commands:
  # 历史策略: 只保留最新命令, 控制命令只关心最新状态
  history: keep_last
  # 队列深度: 1, 只保留最新控制命令
  # 设计原因: 旧的控制命令没有意义, 只需要最新的控制指令
  depth: 1
  # 可靠性: RELIABLE, 控制命令必须可靠到达
  # 设计原因: 丢失控制命令可能导致机器人行为异常甚至危险
  reliability: reliable
  # 持久性: VOLATILE, 控制命令不需要历史数据
  # 设计原因: 控制系统只关心当前命令, 历史命令没有价值
  durability: volatile
  # 截止时间: 100ms, 控制系统需要高实时性
  # 设计考虑: 100ms是大多数控制系统可接受的最大延迟
  deadline:
    sec: 0
    nsec: 100000000 # 100ms
  # 生命周期: 1秒, 控制命令的短暂有效期
  # 设计原因: 超过1秒的控制命令通常已经过时
  lifespan:
    sec: 1
    nsec: 0
  # 存活检测: 自动模式, 确保控制器连接状态
  liveliness: automatic
  # 存活租约: 1秒, 快速检测控制器故障
  # 设计考虑: 控制系统需要快速故障检测以确保安全
  liveliness_lease_duration:
    sec: 1
    nsec: 0

# 配置3: 高频传感器数据配置
# 设计目标: 处理高频率数据流, 优化带宽和CPU使用
# 典型应用: 高频IMU、编码器、力传感器等高频数据源
high_frequency_sensors:
  # 历史策略: 保留少量最新数据, 减少内存开销
  history: keep_last
  # 队列深度: 5, 平衡数据完整性和内存使用
  # 设计考虑: 高频数据会快速填满队列, 小队列减少延迟
  depth: 5
  # 可靠性: BEST_EFFORT, 优先低延迟而非完全可靠
  # 设计原因: 高频数据允许偶尔丢失, 换取更低的传输延迟
  reliability: best_effort
  # 持久性: VOLATILE, 高频数据不需要历史恢复
  # 设计原因: 高频数据更新快, 历史数据价值有限
  durability: volatile
  # 截止时间: 50ms, 高频数据的严格实时要求
  # 设计考虑: 高频传感器通常以20-100Hz运行, 50ms是合理上限
  deadline:
```

```
sec: 0
nsec: 50000000 # 50ms
# 生命周期: 500ms, 高频数据的短暂有效期
# 设计原因: 高频数据更新快, 500ms后的数据通常已被新数据替代
lifespan:
  sec: 0
  nsec: 500000000 # 500ms
# 存活检测: 自动模式, 监控高频数据源状态
liveliness: automatic
# 存活租约: 500ms, 快速检测高频传感器故障
# 设计考虑: 高频传感器故障需要快速发现, 避免使用过时数据
liveliness_lease_duration:
  sec: 0
  nsec: 500000000

# 配置4: 系统状态监控配置
# 设计目标: 完整记录系统状态变化, 支持故障分析和调试
# 典型应用: 系统健康监控、电池状态、温度监控、错误日志
system_monitoring:
  # 历史策略: 保留所有历史数据, 用于故障分析
  history: keep_all
  # 队列深度: 100, 存储足够的历史状态信息
  # 设计考虑: 监控数据更新频率低, 可以保存更多历史记录
  depth: 100
  # 可靠性: RELIABLE, 确保监控数据的完整性
  # 设计原因: 监控数据用于故障诊断, 必须保证数据完整
  reliability: reliable
  # 持久性: TRANSIENT_LOCAL, 支持监控数据的历史查询
  # 设计原因: 新启动的监控节点需要了解系统历史状态
  durability: transient_local
  # 截止时间: 10秒, 监控数据更新频率较低
  # 设计考虑: 系统状态变化相对缓慢, 10秒是合理的更新间隔
  deadline:
    sec: 10
    nsec: 0
  # 生命周期: 60秒, 监控数据的中等有效期
  # 设计原因: 系统状态信息在一定时间内仍有参考价值
  lifespan:
    sec: 60
    nsec: 0
  # 存活检测: 手动主题模式, 由应用程序控制存活状态
  # 设计原因: 监控系统可能间歇性运行, 需要应用程序明确控制
  liveliness: manual_by_topic
  # 存活租约: 30秒, 给监控系统足够的响应时间
  # 设计考虑: 监控系统可能有复杂的检查逻辑, 需要较长的处理时间
  liveliness_lease_duration:
    sec: 30
    nsec: 0

# 配置5: 紧急停止配置
# 设计目标: 最高优先级的安全信号传输, 零容忍的可靠性要求
# 典型应用: 紧急停止按钮、安全系统、碰撞检测、越界报警
```

emergency_stop:

```
# 历史策略: 只保留最新状态, 紧急信号只关心当前状态
history: keep_last
# 队列深度: 1, 紧急信号不需要缓冲
# 设计原因: 紧急情况下, 只有最新的安全状态才有意义
depth: 1
# 可靠性: RELIABLE, 紧急信号绝对不能丢失
# 设计原因: 安全相关信号的丢失可能导致严重事故
reliability: reliable
# 持久性: TRANSIENT_LOCAL, 确保新连接的节点立即获知安全状态
# 设计原因: 新启动的安全系统必须立即了解当前的紧急状态
durability: transient_local
# 截止时间: 10ms, 紧急信号需要极低延迟
# 设计考虑: 安全系统的响应时间直接关系到人员和设备安全
deadline:
  sec: 0
  nsec: 10000000 # 10ms
# 生命周期: 2秒, 紧急状态的有效期
# 设计原因: 紧急状态应该定期刷新, 确保系统了解最新安全状况
lifespan:
  sec: 2
  nsec: 0
# 存活检测: 自动模式, 持续监控安全系统状态
liveliness: automatic
# 存活租约: 500ms, 快速检测安全系统故障
# 设计考虑: 安全系统的故障必须在最短时间内被发现
liveliness_lease_duration:
  sec: 0
  nsec: 500000000
```

配置6: 日志记录配置

```
# 设计目标: 大容量、高可靠的日志数据存储和传输
# 典型应用: 调试日志、操作记录、数据采集、系统审计
```

logging:

```
# 历史策略: 保留所有日志数据, 用于完整的历史回溯
history: keep_all
# 队列深度: 1000, 支持大量日志数据的缓冲
# 设计考虑: 日志数据量大, 需要足够的缓冲空间避免丢失
depth: 1000
# 可靠性: RELIABLE, 确保日志数据的完整性
# 设计原因: 日志数据对于故障分析和调试至关重要
reliability: reliable
# 持久性: TRANSIENT_LOCAL, 支持日志数据的持久化存储
# 设计原因: 日志查看器需要能够访问历史日志数据
durability: transient_local
# 截止时间: 5秒, 日志系统的宽松时间要求
# 设计考虑: 日志记录不是实时性要求, 可以有较宽松的时间限制
deadline:
  sec: 5
  nsec: 0
# 生命周期: 300秒(5分钟), 日志数据的长期有效性
# 设计原因: 日志数据具有长期价值, 用于历史分析和调试
```

```

lifespan:
  sec: 300
  nsec: 0
# 存活检测: 自动模式, 监控日志系统状态
liveliness: automatic
# 存活租约: 60秒, 日志系统的宽松存活检测
# 设计考虑: 日志系统可能有批处理逻辑, 需要较长的处理时间
liveliness_lease_duration:
  sec: 60
  nsec: 0

# 设计总结和使用指导:
#
# 1. sensor_data_reliable: 平衡可靠性和性能, 适用于大多数传感器
# 2. control_commands: 优先实时性, 适用于控制回路
# 3. high_frequency_sensors: 优化高频数据传输, 减少系统负载
# 4. system_monitoring: 支持完整的系统状态跟踪和分析
# 5. emergency_stop: 最高安全等级, 零容忍的可靠性要求
# 6. logging: 大容量数据存储, 支持长期的历史分析
#
# 选择原则:
# - 安全关键系统选择 emergency_stop 或 sensor_data_reliable
# - 实时控制系统选择 control_commands
# - 高频数据流选择 high_frequency_sensors
# - 监控和诊断选择 system_monitoring
# - 数据记录和审计选择 logging
#
# 自定义建议:
# 根据具体应用场景, 可以基于这些配置进行微调, 主要考虑因素包括:
# - 网络带宽限制
# - 内存使用约束
# - 实时性要求
# - 可靠性需求
# - 数据重要性等级

```

launch

dds_example/launch/dds_demo.launch.py

dds_example/launch/dds_full_test.launch.py

配置的启动文件, 具体见源码

编译和运行

第一步：编译项目

```
# 进入工作空间
cd ~/ros2_ws

# 编译DDS示例包
colcon build --packages-select dds_example

# 设置环境变量
source install/setup.bash
```

第二步：运行演示

方法1：基础演示（推荐新手）

```
ros2 launch dds_example dds_demo.launch.py
```

方法2：完整测试（推荐专业用户）

```
# 运行自动化测试
ros2 launch dds_example dds_full_test.launch.py

# 自定义测试时长
ros2 launch dds_example dds_full_test.launch.py test_duration:=60

# 仅启动节点，不进行测试
ros2 launch dds_example dds_full_test.launch.py enable_test:=false
```

方法3：手动启动节点

```
# 终端1：启动发布者
ros2 run dds_example dds_publisher

# 终端2：启动订阅者
ros2 run dds_example dds_subscriber

# 终端3：运行测试客户端
ros2 run dds_example test_dds_demo.py
```

测试和验证命令

监控命令

```
# 查看话题列表
ros2 topic list

# 实时监听数据
ros2 topic echo /sensor/laser_scan

# 检查发布频率
ros2 topic hz /sensor/laser_scan

# 查看详细话题信息
ros2 topic info /sensor/laser_scan --verbose

# 测试延迟
ros2 topic delay /sensor/laser_scan

# 测试带宽
ros2 topic bw /sensor/laser_scan
```

高级测试场景

测试1：持久化数据恢复

```
# 1. 启动发布者
ros2 run dds_example dds_publisher

# 2. 等待3秒后启动订阅者
ros2 run dds_example dds_subscriber

# 3. 强制终止订阅者 (Ctrl+C)
# 4. 等待几秒后重启订阅者
# 5. 观察是否恢复历史数据
```

QoS配置详解

项目包含6种预定义的QoS配置（`config/qos_profiles.yaml`）：

配置名称	应用场景	特点
<code>sensor_data_reliable</code>	传感器数据	高可靠，持久化
<code>control_commands</code>	控制命令	低延迟，实时性
<code>high_frequency_sensors</code>	高频数据	高频率，低开销
<code>system_monitoring</code>	状态监控	完整历史，长期存储

配置名称	应用场景	特点
<code>emergency_stop</code>	紧急信号	极低延迟，高优先级
<code>logging</code>	日志记录	大容量，可靠存储

4.8 分布式通信

ROS 2 核心模块关系解析及图表，涵盖节点、话题、服务、动作、参数和 DDS 的交互逻辑：

4.8.1 核心模块解析

模块	作用	通信模型	依赖关系
节点 (Node)	基本执行单元（进程）	-	依赖 DDS 实现通信
话题 (Topic)	发布/订阅模型，异步数据传输（如传感器数据）	Pub/Sub	通过 DDS 传输消息
服务 (Service)	请求/响应模型，同步调用（如开关控制）	Client/Server	依赖 DDS 的 RPC 机制
动作 (Action)	长任务管理（目标+反馈+结果，如导航）	组合模型（服务+话题）	依赖 DDS 和服务/话题
参数 (Parameter)	动态配置节点运行时参数	客户端/服务端	通过 DDS 访问参数服务器
DDS	底层通信中间件，处理发现、序列化、路由	分布式数据空间	为所有通信模块提供支持

4.8.2 分布式图示

(1) ROS 2 整体架构

```
graph TD
    subgraph Application["应用层 - Application Layer"]
        A["👤 用户节点<br/>User Node<br/>(C++/Python)"]
    end

    subgraph API["API 层 - Client API Layer"]
        B1["📄 rclcpp<br/>(C++ API)"]
        B2["🐍 rclpy<br/>(Python API)"]
        B3["📦 其他语言绑定<br/>(Rust, Java, etc.)"]
    end
```

```

subgraph Middleware["中间件层 - Middleware Layer"]
    C["🌀 RMW<br/>ROS Middleware<br/>(DDS 抽象层)"]
end

subgraph DDS["DDS 实现层 - DDS Implementation"]
    D1["⚡ Fast DDS<br/>(eProsima)"]
    D2["🌀 Cyclone DDS<br/>(Eclipse)"]
    D3["🔧 RTI Connex DDS<br/>(RTI)"]
end

subgraph Transport["传输层 - Transport Layer"]
    E1["🌐 UDP<br/>(跨网络)"]
    E2["🖱️ TCP<br/>(可靠传输)"]
    E3["💾 共享内存<br/>(本地高效)"]
    E4["🔌 串行通信<br/>(嵌入式设备)"]
end

%% 连接关系
A -->|"调用 API"| B1
A -->|"调用 API"| B2
A -.->|"可选"| B3

B1 -->|"统一接口"| C
B2 -->|"统一接口"| C
B3 -.->|"统一接口"| C

C -->|"适配器模式"| D1
C -->|"适配器模式"| D2
C -.->|"企业级"| D3

D1 -->|"底层协议"| E1
D1 -->|"底层协议"| E2
D1 -->|"底层协议"| E3

D2 -->|"底层协议"| E1
D2 -->|"底层协议"| E2
D2 -->|"底层协议"| E3

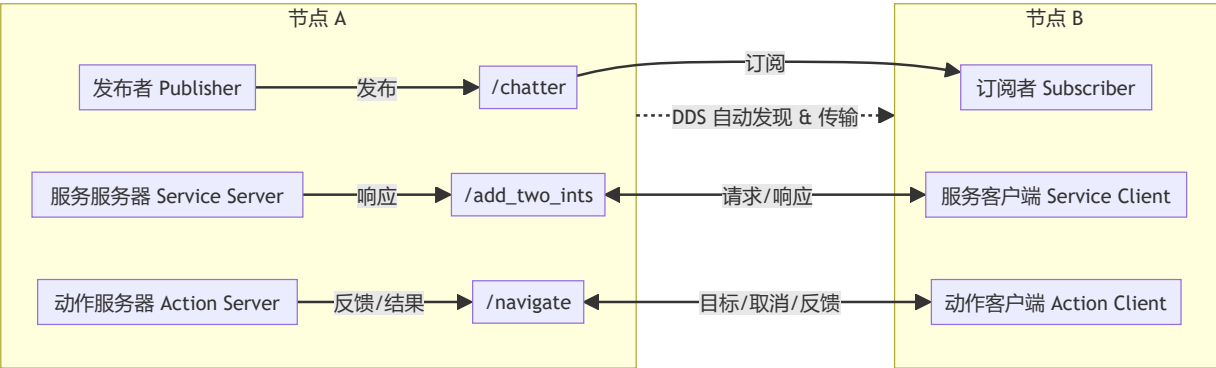
D3 -.->|"底层协议"| E4

%% 样式定义
classDef appLayer fill:#e1f5fe,stroke:#0277bd,stroke-width:3px
classDef apiLayer fill:#f3e5f5,stroke:#7b1fa2,stroke-width:2px
classDef middlewareLayer fill:#fff3e0,stroke:#ef6c00,stroke-width:3px
classDef ddsLayer fill:#e8f5e8,stroke:#2e7d32,stroke-width:2px
classDef transportLayer fill:#fce4ec,stroke:#c2185b,stroke-width:2px

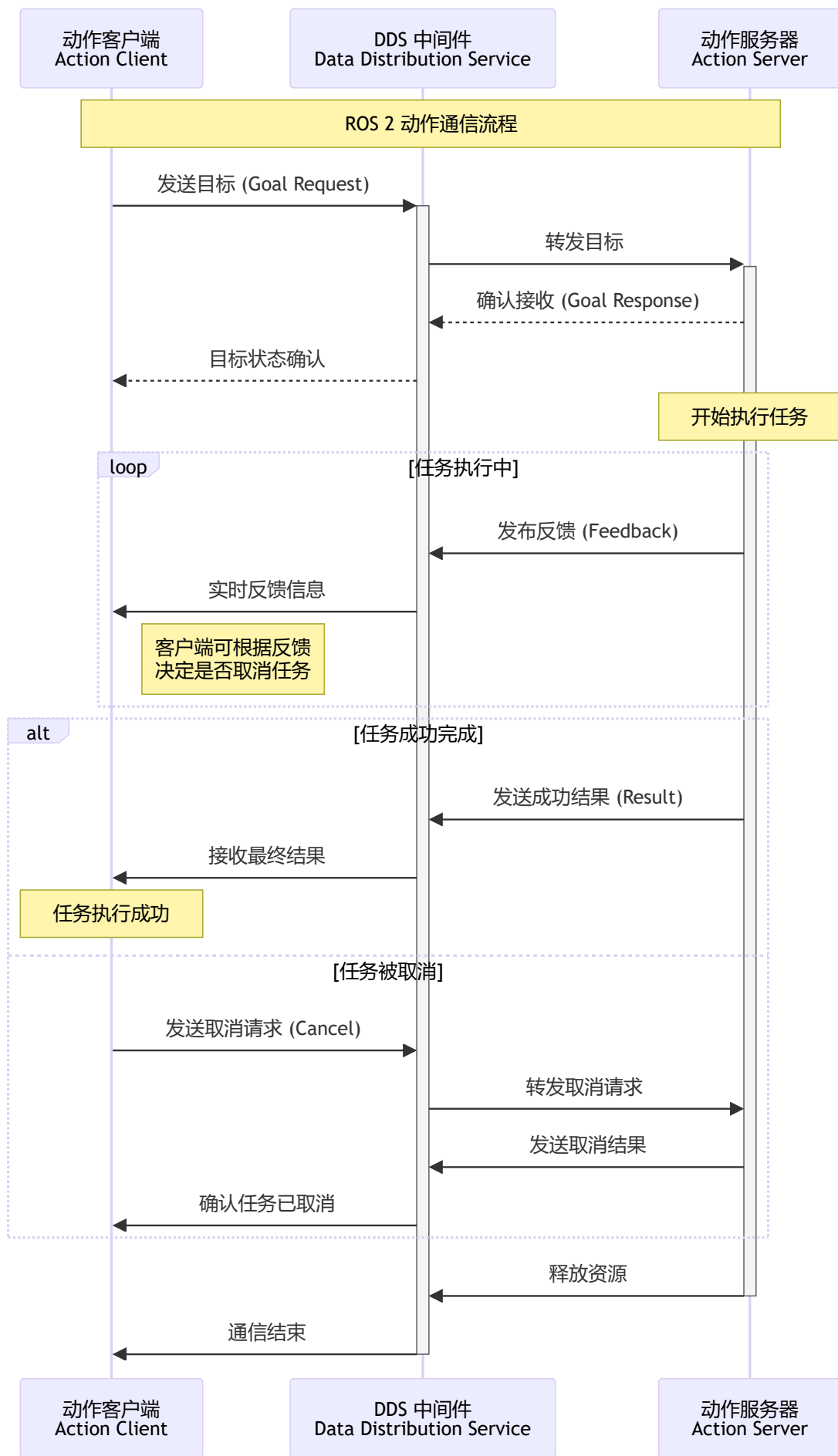
class A appLayer
class B1,B2,B3 apiLayer
class C middlewareLayer
class D1,D2,D3 ddsLayer
class E1,E2,E3,E4 transportLayer

```

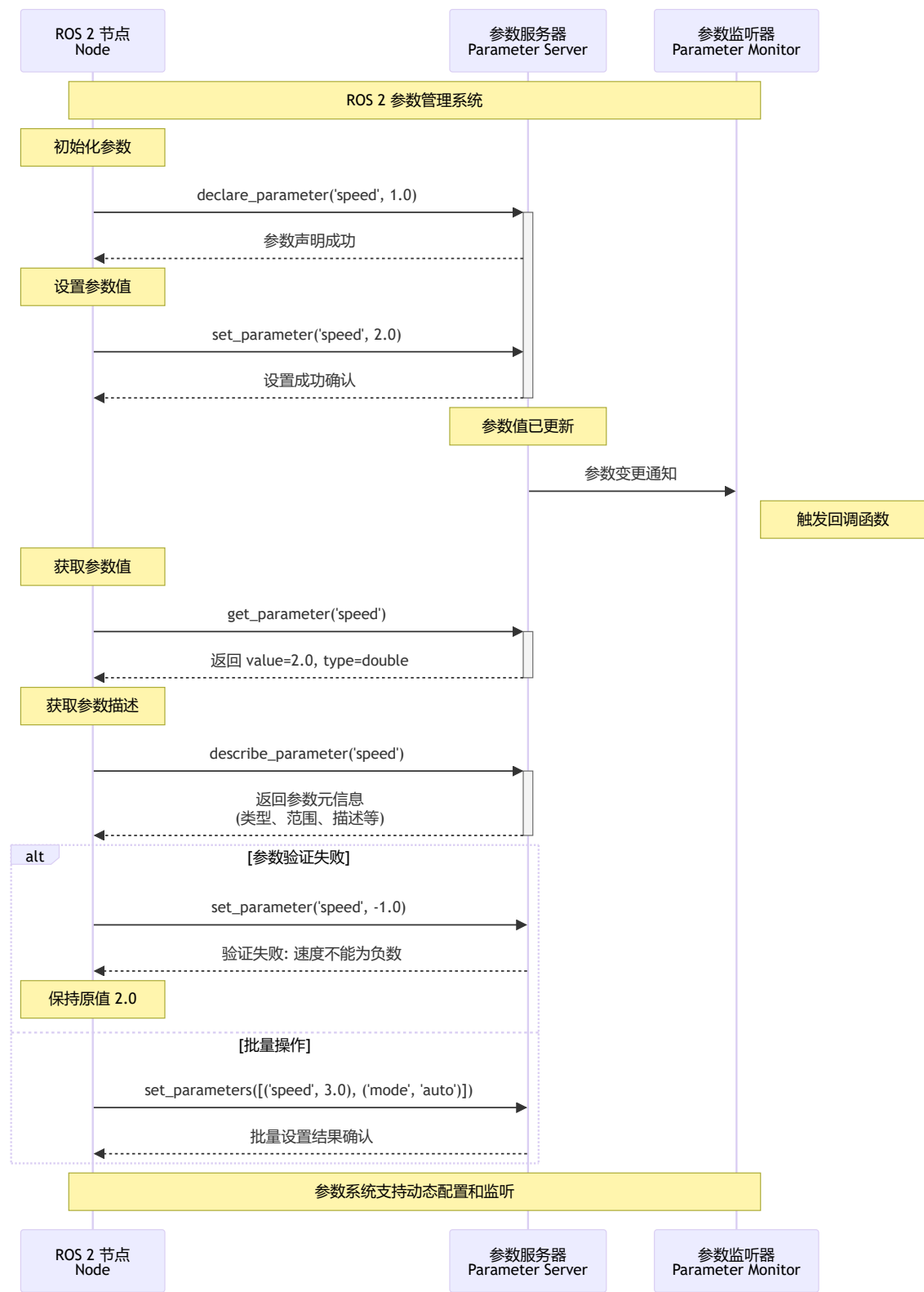

(2) 节点间通信关系



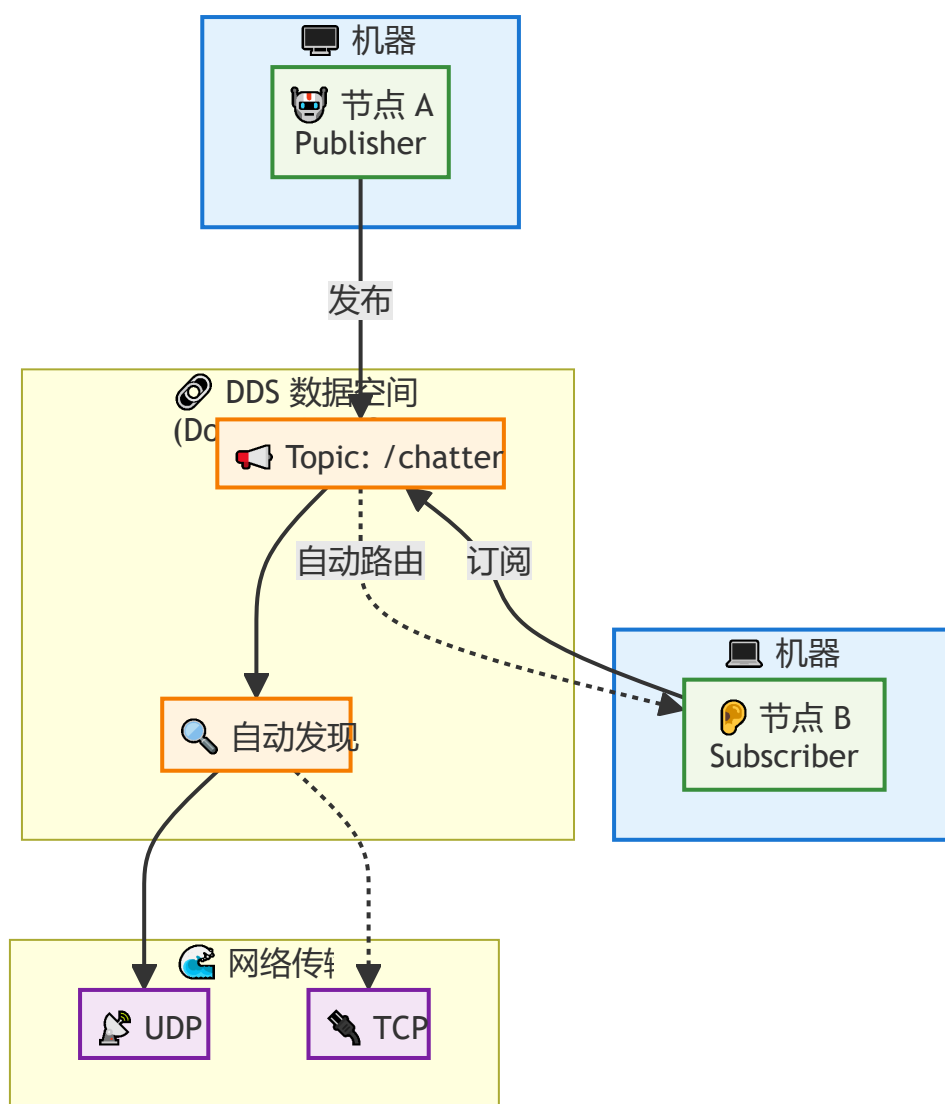
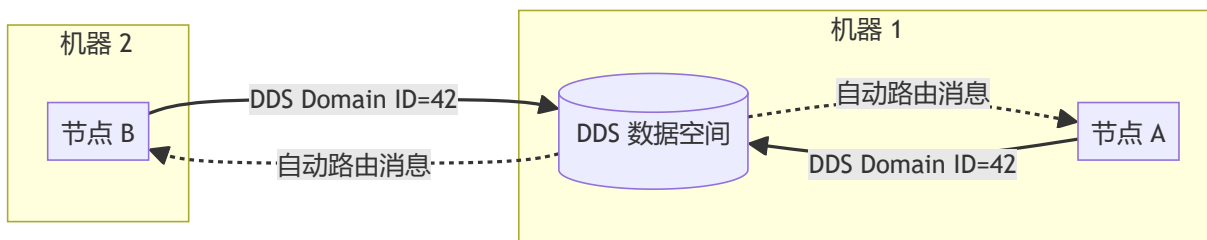
(3) 动作 (Action) 详细流程

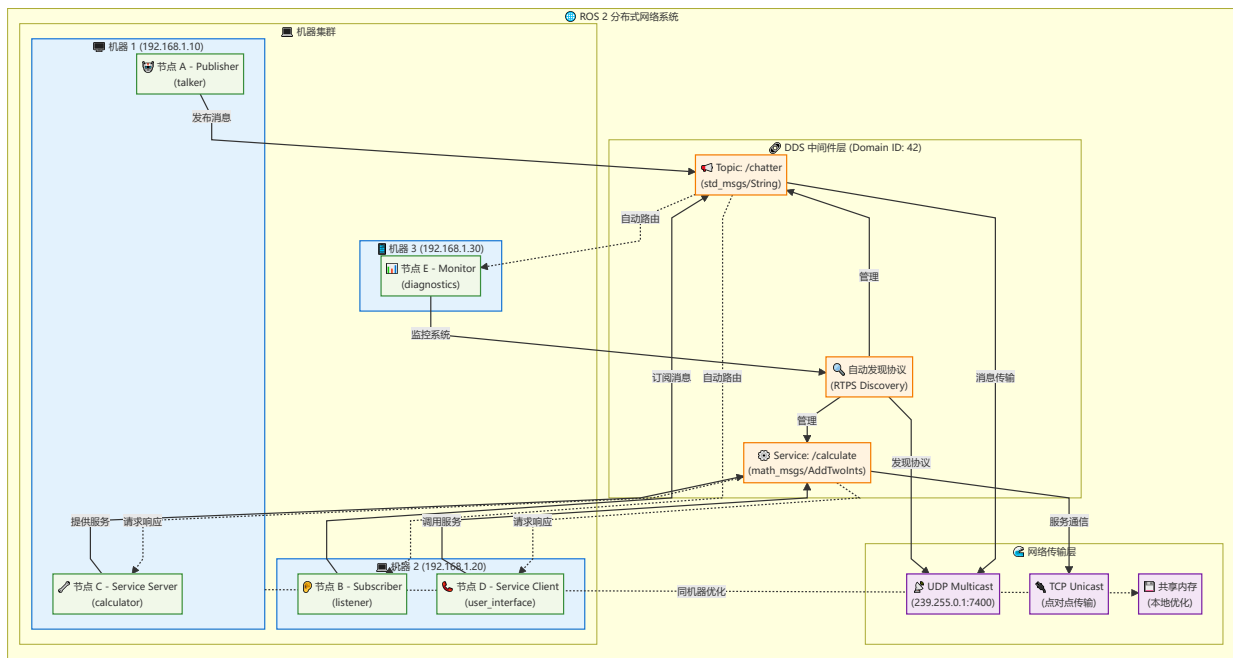


(4) 参数服务器交互



(5) 分布式通信 (跨机器)





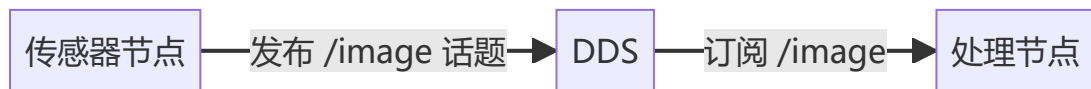
4.8.4 关键交互说明

1. DDS 的核心作用：

- 所有通信模块（话题/服务/动作/参数）均通过 DDS 实现数据传输。
- 提供 **自动发现**（节点无需手动配置连接）、**序列化**（跨语言兼容）、**QoS 策略**（可靠性配置）。

2. 模块协作示例：

- 传感器数据处理：



- 跨机器控制：



